

Federated Learning for Financial Fraud Detection

Mariana Cardoso Barata Lima

Thesis to obtain the Master of Science Degree in

Aerospace Engineering

Supervisors: Prof. Mário Alexandre Teles de Figueiredo
Dr. Pedro dos Santos Saleiro da Cruz

Examination Committee

Chairperson: Prof. Paulo Sérgio De Brito André
Supervisor: Dr. Pedro dos Santos Saleiro da Cruz
Member of the Committee: Prof. Cláudia Alexandra Magalhães Soares

December 2023

“Sempre chega a hora em que descobrimos
que sabíamos muito mais do que antes julgávamos.”

– *Ensaio sobre a lucidez*, José Saramago

Para a minha Mãe. Em memória do José.

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

First of all, I am deeply grateful to Feedzai, namely Pedro Bizarro, for this remarkable opportunity that began with the Women in Science MSc. Scholarship. Thank you for this extraordinary initiative and for letting me experience what it is like to be a Feedzaian. Working on this project has indeed been an incredibly rewarding experience.

I would also like to express my heartfelt gratitude to my supervisors, Dr. Pedro Saleiro, Prof. Mário Figueiredo, and Dr. Jacopo Bono, for their invaluable guidance and expertise. I am also deeply thankful to Sérgio Jesus, who tirelessly helped me whenever I needed it. Their unwavering support has been invaluable to me.

To the entire Research AI team at Feedzai, I extend my sincere appreciation for the warm welcome and approachability. A special thank you goes out to the old FATE team: my buddy Vladimir, Sérgio, João, Mário, and Ricardo. It was a true pleasure working with individuals as talented and supportive as you.

Additionally, I would like to convey a huge thanks to my fellow interns, Jean and Tiago. Your presence has made this journey far more enjoyable, and your friendship has never let me feel alone.

To all my friends and family, thank you for your support and for being an integral part of my life. A special thanks to my mother, my role model and the strongest woman I know, and to my father, the kindest and most hardworking man. Thank you for providing me with all the opportunities you never had. I am incredibly proud to call you my parents.

I would also like to extend this thanks to the wonderful people I met in Stuttgart. Thank you for sharing in one of the most overwhelming experiences of my life. An expression of sincere gratitude to Faiz for his kind-hearted friendship.

To Rita, Diogo, João, and Emauz, thank you for the cherished memories we have created together during these past five years. My journey at Técnico would not have been as pleasing without your presence and friendship.

Last but not least, I want to thank Simão for his unconditional support, understanding, and love throughout all these years. Thank you for always believing in me, even when I do not. I genuinely could not have achieved this without you, and I am eternally grateful.

Obrigada, Aero, e obrigada.

This work was carried out in Feedzai's AI Research Group.

Resumo

O aumento de fraude financeira tem desafiado a estabilidade dos sistemas financeiros mundiais, que tradicionalmente abordam este problema através do desenvolvimento de modelos de aprendizagem automática (AA) baseados nos próprios dados de transações de clientes. Apesar de diferentes instituições apresentarem semelhantes padrões de fraude e esquemas de dados, diretrizes regulamentares impedem a partilha de dados de clientes entre as mesmas, tornando os modelos centralizados inadequados para a detecção colaborativa de fraude. Este trabalho explora aprendizagem federada (AF) como uma abordagem alternativa, que permite o treino de um modelo partilhado entre várias instituições, com o objetivo de melhorar o desempenho preditivo em relação a modelos locais. A aplicação de AF na detecção de fraude financeira carece ainda de estudo relativamente à compreensão da interação entre o desempenho, a equidade, e a privacidade em contextos operacionais dinâmicos. Para colmatar esta lacuna, propõe-se uma abordagem experimental para aplicação de AF na detecção de fraude na abertura de contas bancárias. Esta consiste na comparação entre aprendizagem centralizada e distribuída, em contextos IID e não IID, na avaliação de equidade, e na incorporação de privacidade diferencial (PD). Os resultados experimentais demonstram que AF não compromete o desempenho nem a equidade quando comparada com modelos centralizados. Contudo, revelam-se os compromissos inerentes à obtenção simultânea de desempenho, equidade e privacidade robusta quando PD é integrada na estrutura de AF. Em suma, conclui-se que AF surge como uma via promissora na detecção de fraude em cenários em que a centralização de dados é limitada por restrições legais/ regulamentares.

Palavras-chave: aprendizagem federada, aprendizagem automática, equidade, privacidade diferencial, criminalidade financeira

Abstract

Financial fraud is on the rise and poses challenges to the stability of global financial systems. Individual financial institutions typically address this through the development of proprietary machine learning (ML) models based on their own customer transaction data. Despite facing similar fraud patterns and using similar data schemas, these institutions are often restricted by regulatory guidelines that prevent the sharing of customer data, making traditional centralised ML models unsuitable for collaborative fraud detection. This work explores federated learning (FL) as an alternative approach. FL, a subfield of ML, enables the training of a shared model across multiple institutions, aiming to improve predictive performance over localised models trained on individual datasets. The application of FL in financial fraud detection remains under-researched, particularly in understanding the interplay between predictive performance, fairness, and privacy in dynamic operational settings. To overcome this gap, we develop a thorough experimental framework for applying FL to bank account opening fraud detection consisting of evaluating centralised versus distributed learning in both IID and non-IID settings, evaluating fairness and incorporating differential privacy (DP). Experimental findings indicate that FL does not adversely affect either performance or fairness when compared to centralised models. However, the study also uncovers trade-offs in achieving performance, fairness, and robust privacy simultaneously when DP is integrated into the FL framework. The conclusion drawn is that FL offers a viable path for fraud detection in scenarios where data centralisation is restricted due to legal or regulatory requirements.

Keywords: federated learning, machine learning, fairness, differential privacy, financial fraud

Contents

Acknowledgments	vii
Resumo	ix
Abstract	xi
List of Tables	xv
List of Figures	xvii
Acronyms	xix
1 Introduction	1
1.1 Motivation	1
1.2 Objectives	2
1.3 Contributions	2
1.4 Thesis Outline	3
2 Related Work	5
2.1 Supervised Machine Learning for Decision-Making	5
2.2 Fairness in Machine Learning Algorithms	7
2.2.1 Fairness-Enhancing Mechanisms	8
2.3 Federated Learning	11
2.3.1 Challenges of Federated Learning	12
2.3.2 Federated Learning for Financial Fraud Detection	16
2.3.3 Federated Learning Applications in Aerospace	17
3 Theoretical Background	19
3.1 Evaluation Metrics	19
3.1.1 Performance Metrics	19
3.1.2 Observational Fairness Metrics	20
3.2 Centralised Learning	22
3.2.1 Multi-Layer Perceptron	22
3.3 Federated Learning	25
3.3.1 Problem Formulation	25
3.3.2 Federated Averaging Method	26
3.3.3 Alternative Methods in Federated Optimisation	28

3.4	Differential Privacy	31
3.4.1	Sample-Level Differential Privacy	31
4	Centralised vs. Federated Learning on IID Settings	35
4.1	Bank Account Fraud Dataset Suite	35
4.1.1	BAF IID	36
4.2	Methodology	36
4.2.1	Centralised Model	36
4.2.2	Federated Learning Approach	38
4.3	Results	41
4.3.1	Performance Metrics	41
4.3.2	Federated Averaging	41
5	Real-World Federated Learning	47
5.1	BAF Base Dataset	47
5.2	Methodology	48
5.2.1	Centralised Model and FedAvg	48
5.2.2	FedProx and FedOpt	49
5.3	Results	51
6	Fairness and Privacy in Federated Learning	57
6.1	Fairness Evaluation	57
6.1.1	Fairness metrics	59
6.1.2	Results	59
6.2	Differentially Private Federated Learning	62
6.2.1	Implementation	62
6.2.2	Results	63
7	Conclusions	67
7.1	Future Work	68
	Bibliography	69
A	Hyperparameter Search Spaces	79
B	Dissertation Workflow	81

List of Tables

2.1	Related work on supervised machine learning for decision-making.	7
2.2	Related work on fairness-enhancing techniques.	11
2.3	Related work on addressing statistical heterogeneity in FL.	14
2.4	Related work on privacy-preserving technologies.	16
3.1	Binary classification confusion matrix.	19
4.1	Summary of the BAF IID dataset.	36
4.2	Multi-layer perceptron architecture for the IID settings.	37
4.3	Number of samples and fraud rate across clients for BAF IID dataset.	39
4.4	Different configurations of the FedAvg model for the experiments on IID settings.	40
4.5	Key differences between both IID experiments.	42
5.1	Summary of the BAF Base dataset.	48
5.2	Multi-layer perceptron architecture for the non-IID settings.	48
5.3	Number of samples and fraud rate across clients for BAF Base dataset.	49
5.4	Different configurations of the FedAvg model for the experiments on non-IID settings.	49
5.5	Different configurations of the FedProx model for the experiments on non-IID settings.	50
5.6	Different configurations of the FedOpt model for the experiments on non-IID settings.	51
5.7	Key differences between the IID and the non-IID experiments.	51
6.1	Fairness overview of the BAF IID dataset.	58
6.2	Fairness overview of the BAF Base dataset.	58
6.3	Fairness overview across clients on non-IID settings.	58
6.4	FPR parity for the FedAvg algorithm on the IID uniform settings, with $K = 5$ clients.	60
6.5	FPR parity for the FedAvg algorithm on the IID uniform settings, with $K = 8$ clients.	60
6.6	FPR parity for the FedAvg algorithm on the IID uniform settings, with $K = 16$ clients.	60
6.7	FPR parity for the FedAvg algorithm on the IID stratified settings.	61
6.8	FPR parity for FedAvg, FedProx, and FedOpt algorithms on non-IID settings.	61
A.1	Centralised model hyperparameter search space.	79
A.2	Differential Privacy hyperparameter search space.	80

List of Figures

3.1	Multi-layer perceptron architecture (adapted from [91]).	23
3.2	Diagram of the federated training process (adapted from [9]).	26
3.3	Visual depiction of the DP-SGD algorithm [97].	33
4.1	Flower core framework architecture with Virtual Client Engine (adapted from [101]).	39
4.2	Performance evaluation for the FedAvg algorithm on the IID uniform setting.	43
4.3	Performance evaluation for the FedAvg algorithm on the IID stratified setting.	44
4.4	Comparison of the performance of the FedAvg algorithm on both IID settings.	44
5.1	Performance evaluation for the FedAvg algorithm on non-IID settings.	52
5.2	Performance evaluation for the FedProx algorithm non-IID settings.	53
5.3	Performance evaluation for the FedAdam algorithm, with $\tau = 10^{-4}$, on non-IID settings.	54
5.4	Performance evaluation for the FedAdam algorithm, with $\eta = 10^{-1.5}$, on non-IID settings.	54
5.5	Comparison of the performance of FedAvg, FedProx, and FedOpt algorithms.	55
6.1	Comparison of the performance and fairness of FedAvg, FedProx, and FedOpt algorithms.	62
6.2	Influence of δ on the performance, fairness and privacy level of the DP-FedProx algorithm.	65
6.3	Influence of Γ on the performance, fairness and privacy level of the DP-FedProx algorithm.	65
6.4	Influence of σ on the performance, fairness and privacy level of the DP-FedProx algorithm.	65
B.1	Flowchart of the dissertation workflow.	82

Acronyms

AI Artificial Intelligence

ANN Artificial Neural Network

AUC Area Under the ROC Curve

Adagrad Adaptive Gradient

Adam Adaptive Moment Estimation

BAF Bank Account Fraud

BCE Binary Cross-Entropy

CNN Convolutional Neural Network

COMPAS Correctional Offender Management Profiling for Alternative Sanctions

DL Deep Learning

DP-FedProx Differentially Private FedProx

DP-SGD Differentially Private Stochastic Gradient Descent

DP Differential Privacy

FDS Fraud Detection System

FI Financial Institution

FL Federated Learning

FNR False Negative Rate

FN False Negative

FPR False Positive Rate

FP False Positive

FTL Federated Transfer Learning

FedAvg Federated Averaging

FedSGD Federated Stochastic Gradient Descent

GBDT Gradient Boosted Decision Trees

GD Gradient Descent

GM Gaussian Mechanism

HFL Horizontal Federated Learning

IID Independent and Identically Distributed

MLP Multi-Layer Perceptron

ML Machine Learning

NN Neural Network

PHM Prognostic and Health Management

PPV Positive Predicted Value

RNN Recurrent Neural Network

ROC Receiver Operating Characteristic

RUL Remaining Useful Lifetime

ReLU Rectified Linear Unit

SGD Stochastic Gradient Descent

SMC Secure Multiparty Computation

SMOTE Synthetic Minority Oversampling Technique

SVM Support Vector Machine

SaaS Software as a Service

TN True Negative

TPR True Positive Rate

TP True Positive

TanH Hyperbolic Tangent

UAV Unmanned Aerial Vehicle

VFL Vertical Federated Learning

Chapter 1

Introduction

1.1 Motivation

The advent of machine learning (ML) has led to the widespread integration of artificial intelligence (AI) into high-stakes decision-making contexts. These scenarios encompass critical domains, including: finance, for the prevention of financial fraud [1]; healthcare, for disease diagnosis [2]; aerospace, for anomaly detection and collision avoidance [3]; and even criminal justice [4].

State-of-the-art ML algorithms excel in extracting patterns and statistical insights from extensive data. However, they often operate as opaque black boxes, not providing explanations behind their decisions. This lack of transparency has resulted in instances where ML models, in applications with significant human and societal consequences, have exhibited discrimination against underprivileged groups [5]. This occurs because these models may inadvertently amplify and perpetuate biases that may be present in the data itself [6]. The ensuing awareness of this problem has encouraged progress in the field of Fair ML, which centres on measuring and enhancing fairness [7, 8].

Feedzai operates as a software as a service (SaaS) company, providing a platform that leverages ML and AI to detect and prevent a range of financial crimes, including transaction frauds, bank account opening frauds, and money laundering. For companies like Feedzai, the development of an effective fraud detection system (FDS) is of the utmost importance, as it plays a critical role in mitigating the negative consequences of these crimes, which can include financial losses and blocked credit cards.

Data obtained from financial institutions (FIs) is often characterised by its high level of sensitivity; as a result, it is typically stored and processed locally within each FI. Consequently, it is a prevalent practice for each FI to access and use its data resources to train reliable ML models designed for the detection of fraudulent activities. This approach is referred to as centralised learning and involves local storage of data but also local hosting of the individual ML models themselves.

Nevertheless, centralised FDSs may lack efficiency and generate numerous false alarms due to insufficient data and skewed data distributions [9]. Moreover, financial fraud patterns and data schemas often show similarities across different FIs, located within the same geographical region. Hence, a potential approach for enhancing FDSs could be for FIs to exchange their data amongst themselves or

with a centralised server. This collaborative data exchange approach could empower a centralised joint learning model to identify additional instances of fraudulent activities originating from diverse institutions. However, due to regulatory guidelines regarding the privacy of customer data within fraud detection datasets, FIs are generally prohibited from engaging in data sharing. Despite local storage and apparent anonymisation of the datasets, privacy concerns may persist.

Federated learning (FL), a subfield of ML introduced by McMahan et al. [10] in 2016, has emerged as a viable alternative to centralised learning. In FL, multiple parties collaboratively train a consortium model under the orchestration of a central server while keeping the training data decentralised. FL aims to enhance predictive performance when compared to individual local models trained on a single source and mitigate many of the privacy risks and costs resulting from traditional, centralised ML approaches [11].

Despite its potential, the adoption of FL within the banking industry is an under-researched area, particularly in understanding the complex interplay between predictive performance, fairness, and privacy in dynamic operational settings. Ensuring fairness is of the utmost importance to prevent any particular group (for instance, defined by race, gender, or age) from being disproportionately affected by the errors of the automated system. An incorrect fraud prediction typically results in negative consequences for end-customers, the decision subjects.

1.2 Objectives

Given the aforementioned considerations, the main objective of this dissertation is to develop and evaluate an FL approach for an imbalanced binary classification task, specifically in the context of financial fraud detection, considering tabular datasets. To accomplish this goal, we investigate state-of-the-art FL algorithms and assess their performance and fairness. Additionally, a privacy-preserving technique is employed to ensure the protection of the datasets instances.

1.3 Contributions

In the pursuit of the above-mentioned research objectives, this thesis offers noteworthy contributions to the field of FL and its implementation in the domain of detecting financial fraud. First, an FL approach to identify fraudulent bank account openings is developed based on a tabular dataset derived from a real-world fraud detection use case.

Secondly, a comprehensive comparison is conducted, assessing both the performance and fairness of the FL approach in contrast to a centralised learning method. This evaluation takes into account several parameters, including the amount of local computation and the number of participating clients.

Furthermore, this comparative evaluation is carried out across three different data scenarios. The first stage begins in a controlled setting where data stands fully independent and identically distributed (IID), followed by an exploration of a scenario in which the original dataset maintains IID properties but

its distribution among clients is not IID. Finally, the study expands to a realistic, fully non-IID setting, comprehensively acknowledging the consequences and impacts of non-IID data in the FL approach.

Alongside these contributions, we implement and investigate various state-of-the-art FL algorithms, including Federated Averaging (FedAvg), FedProx, and FedOpt. The latter two are specifically tailored to address the statistical heterogeneity challenge within the FL framework.

Finally, the implemented methods are fortified with a privacy-preserving mechanism, namely differential privacy (DP). DP is employed to hide the contribution of individual instances, ensuring the protection of end-customers privacy and preventing inferences about specific individuals from the model using privacy attacks. The consequences of employing this privacy technique are additionally and thoroughly studied concerning the models' performance and fairness.

1.4 Thesis Outline

The remainder of this work is structured as follows:

- **Chapter 2 (Related Work):** offers a comprehensive overview of the state-of-the-art in FL, focusing on its challenges and including a literature review on algorithmic fairness. Additionally, this chapter highlights notable literature within the aerospace field that leverages FL for various applications.
- **Chapter 3 (Theoretical Background):** provides the necessary theoretical background on the key concepts crucial to this dissertation. It encompasses an exploration of evaluation measures for ML models, an introduction to observational fairness metrics, and the formalisation of the centralised learning approach. Additionally, this chapter formulates the federated optimisation problem, presents the implemented FL algorithms, and offers a theoretical introduction to DP. Together, these components lay the groundwork for a comprehensive understanding of the topics addressed in this dissertation.
- **Chapter 4 (Centralised vs. Federated Learning on IID Settings):** reveals the detailed implementation of the proposed FL methodology, specifically designed to detect financial fraud in IID data distribution settings. This chapter further delivers a comprehensive overview of the dataset used in this study and presents empirical results that offer valuable insights into the performance of the framework.
- **Chapter 5 (Real-World Federated Learning):** extends the foundations laid in the previous chapter and focuses on the distinctive challenge of non-IID data distributions. It provides insights regarding the adaptability and effectiveness of FL within the intricate framework of non-IID data and delves into the practical implications and strategies for addressing the complexities inherent in these settings. It also provides a description of the dataset employed for this purpose.
- **Chapter 6 (Fairness and Privacy in Federated Learning):** delves into the domain of responsible AI, addressing concerns related to fairness and ethics. This chapter includes a comprehensive

analysis of the employed algorithms' fairness, considering both IID and non-IID settings. Additionally, it tackles the privacy issue within FL by introducing the implementation of a privacy-preserving technique into the training pipeline to protect sensitive data. The underlying consequences on the models' performance are also analysed.

- **Chapter 7 (Conclusions):** provides some concluding remarks and suggestions for future work.

Chapter 2

Related Work

This chapter explores the existing literature to provide insights and the context that support this dissertation. First, a brief summary of the applications of supervised ML in decision-making is offered for historical background. Then, an overview of research topics in the field of observational fairness is presented, including some fairness-enhancing mechanisms. Furthermore, an exhaustive literature review of FL is carried out, covering state-of-the-art algorithms, core challenges, and applications. Significant attention is directed towards prior research that has employed FL techniques in the domains of fraud detection and aerospace.

2.1 Supervised Machine Learning for Decision-Making

Supervised learning is a subcategory of ML that aims to learn a function that maps an input to an output based on a sample input-output pair. It uses labelled historical data to infer an unknown outcome, based on a given set of relevant features. Due to its multiple characteristics such as scalability, generalisation, and consistency, supervised learning is a handy tool in decision-making, where informed data-driven predictions are required.

Some relevant domains and fields employ supervised learning for decision-making. Within finance, ML has been leveraged to detect fraudulent credit card transactions, identity theft and fake account registration, and even anti-money laundering [1, 12]. A wide range of approaches, from simple regression and decision trees to sophisticated neural networks (NNs), have been used. For instance, Fu et al. [13] propose a convolutional neural network (CNN) to capture the inherent patterns of fraud behaviours and demonstrate its superior performance compared with traditional techniques that use rule-based expert systems. In addition, Desrousseaux et al. [14] suggest an ML system to detect theft on e-banking account opening, using browsing history to characterise user behaviour.

In the healthcare sector, ML has been extensively used, from extracting information from medical documents to diagnosing diseases [2]. For example, Rajpurkar et al. [15] design a CNN that is able to diagnose pneumonia from chest X-rays with higher accuracy than practising radiologists, and Bharat et al. [16] demonstrate that a support vector machine (SVM) is a suitable technique to predict recurrence

of breast cancer. More recently, Kushwaha et al. [17] investigate how ML algorithms can be used to combat the COVID-19 virus and the pandemic.

On the other hand, ML has played a controversial role in aiding judges in deciding whether prisoners should be released pretrial and at what level should bail be set. Through the use of criminal risk assessment software, such as the Correctional Offender Management Profiling for Alternative Sanctions (COMPAS) system, it is possible to predict a person's risk of recidivism [4].

Furthermore, areas included in the field of aerospace engineering, such as thermodynamics, aerodynamics, flight mechanics, acoustics, and guidance and attitude control systems also increasingly rely on ML techniques [3]. Anomaly detection, for instance, plays a pivotal role in spacecraft flight by aiding in the recognition of uncommon occurrences or suspicious observations. It serves the purpose of continuously monitoring the spacecraft's condition and predicting potential malfunctions. Su et al. [18] propose omnianomaly, a stochastic recurrent neural network (RNN) for multivariate time series anomaly detection. Based on reconstruction probability, their approach provides an intuitive and effective way to interpret detected anomalies and demonstrates its robustness, outperforming the best baseline method by 0.09 on the overall F1-score. In predicting the remaining useful lifetime (RUL) of aircraft subsystems, Azevedo et al. [19] present a web-based application that models a prognostics and health management system, intending to forecast the RUL of specific aircraft components through the application of various ML approaches, including NNs.

Another sector of aerospace engineering where ML algorithms are of the utmost importance is collision avoidance. Collisions at orbital velocities can create debris and initiate a chain reaction of subsequent collisions, posing a threat to the entire satellite population. This scenario is identified as the Kessler syndrome and its name has led to the development of a Python package for ML applied to collision avoidance by Acciarini et al. [20]. Kessler offers capabilities for importing and exporting conjunction data messages, as well as predicting the progression of conjunction events using trained Bayesian RNNs. Finally, AI and ML can also be very useful in preventing aviation incidents by enabling a deeper understanding of the complex factors involved. The research conducted by Cankaya et al. [21] aims to identify the key variables and their probabilistic relationships that influence the extent of aircraft damage. The authors develop a fully automatic ML prediction-based decision support system for assessing aircraft damage severity. Their methodology reveals the probability-based interrelations between parameters including air terminal, flight, flight crew, and aircraft-related characteristics, providing insight on the underlying interactions that contribute to accident severeness.

To summarise the ML applications within the domains employing supervised learning, Table 2.1 details the mentioned research papers, their respective domains and main scope of application, and some of the algorithms employed.

The aforementioned supervised ML applications often rely on black-box models, such as deep learning (DL) networks. These models boast remarkable predictive capabilities, however, due to their nested non-linear structure, they generate outcomes without justified explanations of how the results are obtained. This lack of explainability and transparency can be a major drawback since it does not guarantee fairness-aware decisions [5, 22, 23]. The algorithms' ability to learn patterns from data also enables

Table 2.1: Related work on supervised machine learning for decision-making.

References	Domain	Scope	Algorithms
[1, 12–14]	Finance	Credit card fraud detection and identity theft on e-banking account opening detection.	Logistic Regression, Decision Tree, NN, CNN, SVM, and Random Forest, among others.
[2, 15–17]	Healthcare	Diagnosis and prediction of diseases. Combating the COVID-19 pandemic.	CNN, SVM, among others.
[4]	Criminal Justice	Prediction of potential recidivism risk.	COMPAS software.
[3, 18–21]	Aerospace Engineering	Anomaly detection, prediction of the RUL of aircraft components, collision avoidance, and prevention of aviation incidents.	NN, RNN, and Bayesian RNN, among others.

them to incorporate biases embedded within and make decisions that disproportionately harm certain social groups [7, 8]. The awareness of this issue has encouraged advancements in the field of fair ML.

2.2 Fairness in Machine Learning Algorithms

Discrimination involves treating an individual unfairly based on their affiliation with a specific group, defined by a protected attribute, such as race or gender [8]. As ML becomes more prominent in critical decision-making procedures, there is a rising concern about its potential to exacerbate pre-existing social inequities [24].

Angwin et al. [6] reveal a significant bias in the COMPAS software, where black defendants have a considerably higher probability of being classified as high risk, even though race is not explicitly used as an input. Moreover, among defendants who did not re-offend, blacks are almost twice as likely as whites to be assigned a higher risk (45% vs. 24%). On the reverse side, whites are more likely than blacks to be considered lower risk among those who have re-offended.

Within the financial field, Bartlett et al. [25] estimate racial/ethnic discrimination in the US consumer lending market using data from two major mortgage financing enterprises, Fannie Mae and Freddie Mac, on mortgage credit risk pricing. The authors find that Latinx and African-American borrowers are charged 7.9 and 3.6 basis points more for purchase and refinance mortgages than other borrowers.

To address these issues, fair ML research focuses on approaches to measure and mitigate unfairness. Barocas et al. [26] provide observational fairness metrics, which can be categorised into two types: individual fairness refers to fairness among single instances, implying that a model is fair if people with similar attributes face similar model predictions; group fairness evaluates whether the results are fair when considering the overall outcomes for the protected group. Common group fairness metrics include statistical parity [27], equalised odds, equality of opportunity [28], and predictive equality [29]. More

recently, Martinez et al. [30] introduce minimax group fairness, which demands that the model optimise prediction performance for the most disadvantaged demographic group without unduly compromising the performance of other demographic groups. This principle is often referred to as no-harm fairness [31].

Nevertheless, while multiple fairness metrics have been proposed, determining the suitable metrics for specific applications can be a non-trivial decision. Therefore, Saleiro et al. [32] propose Aequitas, an open-source bias and fairness audit toolkit that allows users to test models for multiple fairness measures in relation to different population sub-groups. Additionally, Ghani et al. [33] attempt to bridge the research-practice gap in the field of algorithmic fairness, by providing a tutorial that empowers data scientists and practitioners to effectively navigate and address bias-related issues throughout the entire life cycle of real-world data science projects, using the Aequitas toolkit.

2.2.1 Fairness-Enhancing Mechanisms

The literature on Fair ML covers a wide range of approaches aimed at mitigating bias and unfairness in ML algorithms. However, there is an evident trade-off between fairness and accuracy. Corbett-Davies et al. [29] conduct a theoretical analysis on this topic. As such, the primary objective of Fair ML concerning this trade-off is to develop algorithms that prioritise fairness while minimising the compromise on accuracy.

On an intuitive level, one can consider the inclusion of fairness constraints in the optimisation problem. Nevertheless, it is reasonable to expect that introducing these constraints could result in comparable or even reduced accuracy when compared to a model solely focused on maximising accuracy. There are, however, some exceptions. For instance, Zhao and Gordon [34] develop an algorithm that returns a randomised classifier that is both fully accurate and entirely fair, under the statistical parity constraint.

Fairness-aware approaches enclose three broad categories, aligning with actions taken at different stages of analysis: pre-processing, in-processing, and post-processing. These align with interventions on the training data, model optimisation, and model output, respectively. In the work developed by Lamba et al. [35], the authors present a comprehensive empirical evaluation of distinct bias mitigation techniques over real-world problems originating from public policy and social welfare domains.

Pre-processing methods

Pre-processing methods assume that bias in the data is the primary cause of unfair predictions and therefore involve altering the training data before model learning. These modifications consist of either eliminating the sensitive variable or adjusting the data distribution related to the sensitive variable through techniques like sampling [35].

Exclusion of sensitive attributes (for instance, race or gender) has been extensively studied in the past [36]. Approaches based on this technique claim that if a ML model does not have access to the protected variable as a feature, the resulting model will not exhibit bias towards that variable, thereby achieving fairness. However, this assumption is often too optimistic and ineffective in real-world sce-

narios, since the classifier is still able to learn the discriminatory patterns present in the training data through the representation of the protected attribute by proxy variables [37]. A significant body of research demonstrates that omission of sensitive variables not only fails to reduce bias but can also lead to reduced model accuracy [38]. Furthermore, this exclusion can complicate the task of reducing discrimination when employing in-processing or post-processing techniques at later stages of the pipeline [39].

On the other hand, resampling entails modifying the distribution of the training data by either under or oversampling examples, with the aim of mitigating disparities when this modified data is used in model training. For instance, Salazar et al. [40] present FAWOS, a fairness-aware oversampling algorithm that creates new synthetic data points based on the distributions of multiple sensitive attributes' instances. Experimental results on different learning classifiers show that FAWOS can significantly enhance the fairness outcomes of the models without compromising classification accuracy.

More recently, in 2023, Pombal et al. [41] propose FADO, a fairness-aware data valuation framework that can be employed to integrate fairness concerns into various ML tasks, such as data pre-processing. The authors present an entropy-based data valuation measure that aligns with their dual objective of optimising both performance and fairness, offering improved computational efficiency compared to existing metrics. Experimental results demonstrate the application of FADO within unfairness mitigation pre-processing techniques, revealing its superior performance compared to existing methods in the literature when considering fairness-accuracy trade-offs.

To conclude, the primary benefit of pre-processing approaches is that they offer a high degree of flexibility as they directly manipulate the training data, allowing for the implementation of different configurations for subsequent models.

In-processing methods

In-processing methods use ML models or methods explicitly designed to address bias during the model-building process, incorporating either regularisation techniques, that penalise unfair solutions in the objective function, or constrained optimisation, which enforces fairness as a strict constraint, rejecting solutions that do not satisfy fairness criteria.

For example, in the work developed by Kamishima et al. [42], the authors present a regularisation method that penalises an increase in conditional probability for a specific subgroup based on the mutual information between the sensitive attribute and the predictions. Zafar et al. [43] enhance the previous research by integrating fairness constraints into the objective function of the underlying classifier.

More recently, Cruz et al. [44] introduce FairGBM, an in-processing fair ML technique for tabular data, designed to address fairness concerns in high-stakes domains using gradient boosted decision trees (GBDT). In contrast with other existing fairness methods for GBDT, FairGBM shows an order of magnitude speedup in training time and little to no impact on predictive performance when compared to unconstrained GBDT. To handle non-differentiable observational fairness metrics, the authors propose smooth convex error rate proxies for common fairness criteria.

Furthermore, in 2023, Kose and Shen [45], develop FairGAT, a fairness-aware graph-based learning

framework, that employs a novel attention design in graph attention networks. The experiments conducted on real-world networks show that FairGAT enhances group fairness measures while providing utility comparable to fairness-aware baselines in tasks such as node classification and link prediction.

Finally, Han et al. [46] present the fair fairness benchmark (FFB), which serves as a standardised and open-source framework for evaluating in-processing group fairness methods. The authors provide flexible and research-oriented open-source code, establishing unified benchmarking pipelines for fairness methods. Overall, this work aims to facilitate the growth and development of the fairness research community by offering a valuable resource for evaluating and advancing in-processing group fairness techniques.

When compared to pre-processing methods, in-processing ones are expected to have a higher likelihood of satisfying fairness concerns, as they directly incorporate fairness metrics into the design.

Post-processing methods

Post-processing methods typically do not depend on the specific ML models employed; instead, they focus on adjusting scores, thresholds, or model selection to enhance fairness in predictions.

Hardt et al. [28] propose a post-processing approach to achieve either equalised odds or equality of opportunity by adjusting the scores generated by trained classifiers. Rather than employing a global threshold, this method explores the receiver operating characteristic (ROC) space to identify group-specific thresholds that meet the desired fairness metrics. More recently, building upon this research, Rodolfa et al. [47, 48] expand the application of similar techniques to various policy domains and observe minimal to no compromise in model accuracy.

Another post-processing fairness-enhancing technique, suggested by Dwork et al. [49], involves decoupling the training and learning a different classifier for each group. However, decoupling the models might reduce the available training data, especially for small groups. Hence, the authors also explore transfer learning techniques between groups that involve assigning different weights to training examples from the protected group and the rest of the population.

Finally, the simple process of model selection can also be an approach to incorporate fairness in a post-processing manner. However, relying solely on model selection can seem arbitrary as it seeks models that perform well on both fairness and accuracy metrics without actually improving fairness [35].

These post-processing methods are known for their adaptability, relying on prediction and sensitive attribute data, which is particularly beneficial in high-stakes decision-making settings where data access before model training is restricted, and modifying model optimisation strategies is not possible.

These fairness-enhancing techniques embody distinct perspectives regarding the origins of algorithmic unfairness. Pre-processing posits that bias in the data is the primary cause, whereas in- and post-processing approaches attribute the responsibility to modelling choices and criteria. With researchers divided on whether to blame the ML algorithms or the data on which they are trained, interactions between the two have received scant attention. Therefore, in the work of Pombal et al. [7], the authors assert that algorithmic unfairness arises from the interplay between model choices and biases in the data, rather than being solely due to one of them in isolation. Their experiments in a large, real-world,

Table 2.2: Related work on fairness-enhancing techniques.

References	Approach	Description	Advantages
[36–41]	Pre-processing	Assume the bias in the data as the primary cause of unfairness and involve changing the training data before model construction.	High degree of flexibility.
[42–46]	In-processing	Methods that address bias during the model-building process. Usually incorporate regularisation techniques or constrained optimisation.	Higher likelihood of satisfying fairness concerns.
[28, 47–49]	Post-processing	Focus on adjusting scores, thresholds, or model selection to enhance fairness.	Adaptability, useful when data access is restricted and modifying the models is impossible.

bank-account-opening fraud dataset illustrate how an algorithm that demonstrates fairness under specific data biases may exhibit unfairness in different scenarios. In that study, the authors find that simple unfairness mitigation techniques effectively balance error rates and emphasise the importance of selecting the right fairness metric by revealing the limitations of using ratios.

A summary of the unfairness mitigation approaches discussed in this section is provided in Table 2.2.

2.3 Federated Learning

Many of the ML algorithms mentioned in the previous sections require substantial amounts of data. However, in several areas of application, data is often distributed across multiple organisations under the protection of privacy restrictions. As a result, since it was introduced in 2016 by McMahan et al. [10], FL has emerged as a prominent research area within ML, as it enables multiple parties to collaboratively train a model without exchanging their local data, frequently under the coordination of a central server.

Based on the distribution characteristics of the data, FL has been categorised into different architectures: horizontal FL (HFL), vertical FL (VFL) and federated transfer learning (FTL) [50].

In HFL, the most common scenario in FL, datasets share the same feature space but different samples. For example, two regional FIs might have distinct user groups from their respective regions but the same feature space due to the similarity of businesses. Usually, the HFL training process involves client selection, broadcasting, local computations using the client’s private data, and global aggregation on the server. On the other hand, VFL addresses situations where data owners differ in feature space but share the same sample ID space. It also matches many real-world scenarios, for instance, a patient may have medical records in multiple hospitals, each with a different set of features. Lastly, FTL deals with circumstances where the datasets have both distinct feature and sample ID spaces. In such scenarios, transfer learning techniques can be employed to offer solutions that consider the entire sample

and feature spaces within a federation. In the context of this work, HFL is considered. Henceforth, when FL is mentioned, it specifically refers to the HFL architecture.

Regarding the scale of the federation, two settings are contemplated – cross-silo and cross-device FL. Cross-silo scenarios are usually associated with parties being organisations, where there is a limited number of clients with large computational ability. When it comes to cross-device settings, the clients are a very large number of devices with small computation power. Another key difference between these settings lies in client reliability. While in cross-silo there are relatively few failures, in cross-device settings clients are highly unreliable, with an expected failure or dropout rate of 5% or more among the participating clients in a round [11].

2.3.1 Challenges of Federated Learning

Having introduced FL and its categorisation, it is important to delve into the obstacles this approach faces. Therefore, this section introduces some of the core challenges associated with FL, which have been highlighted and explained in the work of Li et al. [51]. These challenges distinguish FL from traditional problems, such as distributed learning in data centre settings or conventional private data analyses.

Communication-Efficiency

In federated networks, it is imperative to develop communication-efficient methods able to send small model updates as part of the training process, instead of sending the entire dataset over the network. This work focuses on the most common communication topology in FL: the star network, in which a central server is connected to a network of clients and the communication between them takes place in rounds. Due to its proven success in various DL applications, stochastic gradient descent (SGD) serves as an initial step for federated optimisation. Its implementation involves performing a single batch gradient computation for each client per round. This approach, called federated SGD (FedSGD), is computationally efficient but requires a substantial number of training rounds to produce high-quality models.

Therefore, McMahan et al. [10] develop FedAvg, the most widely used FL framework based on iterative model averaging. FedAvg adds more computation to each client by iterating the local update multiple times before the averaging step, thus reducing the communication cost. This method is shown to train high-quality ML models, such as multi-layer perceptrons (MLPs) or CNNs, using relatively few rounds of communication.

Statistical Heterogeneity

A core challenge in FL is the heterogeneity of local data. Unlike a typical distributed optimisation problem, the data in federated learning is often non-IID, which means that a specific client's local dataset may not be representative of the population distribution, and unbalanced, meaning varying amounts of local training data. This heterogeneity may significantly degrade the performance of the FL methods,

since the local models are optimised towards their local optima, which can vary widely, resulting in a global model diverging from the global optimum.

Li et al. [52] show that methods such as FedAvg lack convergence guarantees and can diverge in practice when the data is heterogeneous. Therefore, the authors propose FedProx, a method that introduces an additional proximal term in the local objective to restrict the amount of local changes. This approach also tackles the issue of system heterogeneity by allowing for variable amounts of computation locally across clients. Compared to FedAvg, FedProx exhibits more stable and accurate convergence patterns, improving absolute test accuracy by 22 percentage points on average.

Due to the notable success of adaptive optimisation methods to combat these convergence issues in non-federated settings, Reddi et al. [53] propose FedOpt, a family of federated versions of adaptive optimisers, including FedAdam, FedAdagrad, and FedYogi. Their experiments on heterogeneous data emphasise the relationship between client heterogeneity and communication efficiency and show that adaptive optimisers can remarkably improve the performance of FL in nonconvex settings.

Moreover, Karimireddy et al. [54] present SCAFFOLD, a stochastic controlled averaging algorithm, which uses a control variate regularisation term that induces variance reductions on local updates based on the updates of other clients to correct for the client-drift. The authors demonstrate the convergence of SCAFFOLD for heterogeneous data as well as its ability to reduce the communication required when dealing with similar clients.

The aforementioned studies aim to reduce the loss across the entire training data under the non-IID setting. Another approach is to create personalised FL algorithms, where each party learns a personalised model that performs effectively on their local data. For instance, Li et al. [55] propose Ditto, a multi-task learning objective for FL that provides personalisation to achieve fairness and robustness to attacks in statistically heterogeneous networks. It incorporates a regularisation term that encourages the personalised models to be close to the optimal global model. This regularisation term resembles FedProx, however, in Ditto, the goal is to acquire personalised models, while FedProx generated a single global model. For instance, when the regularisation hyperparameter is set to zero, Ditto reduces to learning separate local models, whereas FedProx would revert to FedAvg. The authors show that their approach not only achieves competitive performance relative to recent personalisation methods, but also enables more accurate, robust, and fair models relative to state-of-the-art fair or robust baselines.

More recently, Yang et al. [56] introduce G-FML, a group-based federated meta-learning framework that addresses the inefficient training issue of the current federated meta-learning approaches on highly skewed non-IID data. G-FML adaptively divides the clients into groups based on the similarity of their data distribution, and the personalised models are obtained with meta-learning within each group. Although this approach requires a pre-processing step for training an autoencoder, it achieves the desired level of accuracy in fewer communication rounds when compared to FedAvg.

Table 2.3 provides a summary of the algorithms discussed in this section, which are designed to tackle the challenge of statistical heterogeneity in FL.

Table 2.3: Related work on addressing statistical heterogeneity in FL.

References	Algorithm	Description	Client Models
Li et al. [52] (2020)	FedProx	Adds a proximal regularisation term in the local objective of FedAvg to restrict the local updates to be closer to the initial (global) model.	Shared Global
Reddi et al. [53] (2020)	FedOpt	Introduces an FL version of adaptive optimisers (FedAdam, FedAdagrad, and FedYogi) for use on the server side during the aggregation step.	Shared Global
Karimireddy et al. [54] (2020)	SCAFFOLD	Uses a control variate regularisation term that induces variance reductions on local updates based on the updates of other clients.	Shared Global
Li et al. [55] (2021)	Ditto	Multi-task learning objective for personalised FL that provides fairness and robustness benefits through the use of a regularisation term.	Personalised
Yang et al. [56] (2023)	G-FML	Federated Meta-Learning framework that divides the clients into groups based on their data distribution similarity and obtains personalised models within each group through met-learning.	Personalised

Fairness

The presence of data heterogeneity among clients in FL may also lead to the emergence or worsening of discrimination against groups defined by protected attributes. Although there is growing interest in fairness-aware ML algorithms, the emphasis has predominantly been on centralised ML, with limited exploration of decentralised approaches, such as FL.

Most of the work concentrated on fair ML models in FL settings has predominantly aimed at achieving fairness on a client level, which focuses on building algorithms that ensure models exhibit similar performance across different clients. The aforementioned Ditto algorithm [55] is an example of this concept of client fairness.

Papadaki et al. [31] introduce minimax demographic group fairness in FL settings, in which certain clients may have limited access to specific demographic groups during the training phase, and show under what conditions minimax group fairness is equivalent to minimax client fairness. The authors propose the FedMinMax optimisation algorithm, designed to address the described problem and demonstrate that it outperforms existing FL methods in terms of group fairness.

In 2023, Salazar et al. [57] present Fair-FATE, an innovative fair FL algorithm designed to achieve group fairness while preserving high utility through a fairness-aware aggregation technique. Their approach is the first to use a momentum term to estimate a fair global model update, which mitigates the oscillations of unfair gradients. Experimental results illustrate that Fair-FATE outperforms state-of-the-art fair FL approaches under different degrees of data heterogeneity.

Privacy Concerns

Although FL does not reveal local data, the shared model parameters during the training process may still potentially disclose sensitive information about the data. Numerous attacks have targeted ML models, aiming to potentially infer raw data by exploiting model access. As such, multiple privacy-preserving mechanisms, including cryptographic methods and DP, offer distinct privacy guarantees [58].

Cryptographic methods include homomorphic encryption [59], which allows to secure the learning process by performing computations on encrypted data. However, it has currently been applied in limited settings [51]. Additionally, secure multiparty computation (SMC) [60] ensures that the parties cannot learn anything except the output, which can be used to securely aggregate the transferred gradients. SMC can maintain the original accuracy while providing a very strong privacy guarantee. Nevertheless, the additional encryption and decryption operations introduce substantial additional communication costs [51].

On the other hand, DP, introduced by Dwork [61], is the state-of-the-art model for quantifying and limiting information disclosure about individuals, due to its simple implementation and minor systems overhead. DP guarantees that one single record does not influence much the output of a function. This implies introducing a level of uncertainty into the released model in a way that it is impossible to determine whether a particular sample is used in the learning process. Gradient-based learning techniques often adopt a common strategy of introducing DP by adding random perturbations (for example, Gaussian noise) to the intermediate output during each iteration.

Nevertheless, when extended to FL settings, the idea of user- or client-level DP, introduced by McMahhan et al. [62] is commonly applied. Here, the goal is to protect all the records of a user in the training set, rather than a single instance (referred to as sample-level DP). It is important to note that user-level DP offers stronger privacy guarantees compared to sample-level DP, given that typically a single user may contribute numerous records [11].

There is, however, an inherent trade-off between DP and model accuracy, as introducing noise enhances privacy but can potentially lead to a decrease in accuracy. To tackle this issue, Geyer et al. [63] propose an algorithm for client-level DP preserving federated optimisation where the goal is to hide the individual contributions of clients during training, balancing the privacy-performance trade-off. The authors show that DP on a user level is feasible at only a minor cost in model accuracy with a sufficiently large number of participating clients.

Moreover, the pioneering work of Noble et al. [64] introduces DP-SCAFFOLD, an FL approach that addresses both data heterogeneity and client privacy challenges by incorporating DP constraints into the widely used SCAFFOLD algorithm. Their research prominently illustrates the privacy-utility trade-off, showcasing the superior performance of DP-SCAFFOLD over state-of-the-art algorithms in scenarios with increasing local updates and levels of data heterogeneity.

To conclude, Table 2.4 summarises the privacy-preserving technologies discussed in this section, based on [11].

Table 2.4: Related work on privacy-preserving technologies, adapted from [11].

References	Technology	Description
[59]	Homomorphic Encryption	Enables a party to perform mathematical operations on encrypted data, thus allowing them to compute functions of data without needing access to the plaintext data itself.
[60]	Secure Multiparty Computation	Two or more parties collaboratively simulate, through cryptography, a fully trusted third party. This third party can calculate a function based on inputs from all the parties and disclose the result without any of the parties gaining additional information.
[61–64]	Differential Privacy	Quantifies how much information about an individual could be inferred from the output of an analysis on data that includes the individual. Algorithms that employ DP incorporate some randomness or noise, which can be tweaked to disguise the individual’s impact on the result.

2.3.2 Federated Learning for Financial Fraud Detection

In recent years, as the field of FL has experienced substantial growth, its application in detecting financial fraud has received increased attention. While there is limited literature on FDSs built upon FL algorithms, this section presents a comprehensive overview of recent studies and advancements in this particular research area.

In 2019, Yang et al. [9] propose FFD, an FL framework for credit card fraud detection that allows FIs to learn a model using data distributed within their local databases. To balance the real-world credit card transactions dataset used, the authors combine their method with a synthetic minority oversampling technique (SMOTE) [65] that is specifically designed to tackle imbalanced datasets by generating synthetic samples for the minority class. FFD makes a slight modification to FedAvg in the update aggregation step, considering the combination of data size and detection model performance on each FI for the weighted average. The experimental results show that this method achieves an average test AUC 10 percentage points higher than state-of-the-art FDSs.

Similarly, Jansson and Axelsson [66] develop a framework for training a fraud detection model with FL. Their approach uses FedAvg in three distinct data settings: skewed data, non-IID data, and SMOTE non-IID data, where the latter involves oversampling fraudulent examples using the SMOTE technique. Overall, the results show that FedAvg can outperform the centralised model when detecting credit card fraud. Furthermore, the authors find that FedAvg, when trained on SMOTE non-IID data, demonstrates improved fraud detection capabilities but exhibits a worse ability to differentiate between classes compared to its training on non-IID data.

Zheng et al. [67] introduce a fraudulent credit card detection approach based on federated meta-learning. Their work presents an innovative meta-learning-based classifier to learn the discriminative feature extraction on unseen class samples, by allowing joint comparison with K negative samples in each mini-batch. Experimental results demonstrate that the proposed approach achieves higher performance compared to state-of-the-art baselines, with at least 9.3, 8.8, and 4.1 percentage points

higher in terms of accuracy, recall and F1-score, respectively.

With regard to privacy concerns, Byrd and Polychroniadou [68] propose a differentially private SMC protocol for FL and test it using logistic regression on a real-world financial dataset. They show that with added layers of protection, private data recovery becomes difficult or impossible even with adversarial actors participating in the system.

2.3.3 Federated Learning Applications in Aerospace

While the primary focus of this dissertation revolves around financial fraud detection, it is worth mentioning that FL holds significant promise and has found applications across various domains, including aerospace. This section briefly delves into some notable FL applications within the aerospace industry, showcasing the wide-ranging potential of this technology.

Rosero et al. [69] conduct a comparison of distributed collaborative learning methods for prognostic and health management (PHM) systems, with a specific focus on predictive aircraft maintenance. PHM techniques integrate diagnostics and prognostics of machinery component failures. While diagnostics detects and isolates failures, prognostics predicts future state or RUL. Focusing on prognostics, the authors compare the prediction error minimisation of different FL algorithms, such as FedAvg and FedProx, using feed-forward NNs. The experiments using FedAvg reveal similar prediction errors to the centralised approach, although with unclear predictions throughout the aggregation stages. Regarding FedProx, the prediction error curve decreases steadily over the aggregation iterations if μ is chosen appropriately.

Similarly, Li et al. [70] develop a clustering FL method consisting of a deep NN with a self-attention mechanism for bearing fault diagnosis. This strategy overcomes the limitations of traditional fault diagnosis techniques that fail to collect sufficient data for model training due to data security, economic conflict, and relative laws, among other reasons. Their method proves to be effective and superior when compared to other popular fault diagnosis schemes and ensures data privacy safety.

There is an extensive body of literature concerning the implementation of FL techniques in the context of unmanned aerial vehicles (UAVs). FL allows multiple UAVs to collaboratively train ML models without sharing raw personal data, such as UAV's localisation and identity, thus ensuring the preservation of privacy. Brik et al. [71] conduct a study that highlights multiple federated DL applications in UAVs-enabled wireless networks.

The work of Zhang and Hanzo [72] proposes FL-aided multi-UAV systems for image classification in area exploration scenarios. In their approach, a ground fusion centre coordinates multiple UAVs to assemble a cooperative multi-UAV network. Each UAV has an onboard camera that collects images to learn a local model and then sends the local updates to the ground fusion centre via fading wireless channels. FL is hence used to perform a global update and to send the corresponding results back to the UAVs, for the next round. The results show that, for a fixed target test accuracy, the presented method can operate at a significantly lower communication cost than the centralised approach.

Yao and Ansari [73] present a secure FL in fog-aided Internet of drones networks to counteract

eavesdroppers. The authors design the Power Control in Secure FL algorithm, which is shown to outperform previous work in multiple simulations. Furthermore, Wang et al. [74] propose SFAC, a secure FL framework for UAV-assisted mobile crowdsensing. The authors also design a privacy-preserving algorithm to protect UAVs by applying local DP. To overcome the need in FL for a central entity in charge of ML model aggregation, a novel approach named Decentralised Federated Learning for UAV Networks is developed by Qu et al. [75] and enables FL within UAV networks without the need of a central server. The work validates the feasibility and effectiveness of the architecture through preliminary simulations.

More recently, Moustafa et al. [76] propose DFSat, an FL-based framework for intrusion detection, to identify and mitigate cyberattacks from Internet of things-integrated satellite networks, using an RNN. The results demonstrate the effectiveness of the DFSat, outperforming existing state-of-the-art intrusion detection methods. Finally, Chen et al. [77] discuss how FL can address the challenges in low earth orbit-based satellite communication and So et al. [78] employ FL in which ground stations and satellites cooperate to train a global ML model without sharing the satellite-captured images. The authors also introduce an optimisation problem that captures a staleness-idleness trade-off and propose FedSpace, the first FL framework running at satellites and ground stations that dynamically schedules model aggregation based on deterministic and time-varying satellite orbit connectivity. Extensive evaluations on real-world satellite images and networks reveal FedSpace's capacity to reduce the training time by 38.6 percentage points when compared to state-of-the-art FL algorithms.

Chapter 3

Theoretical Background

This chapter reviews some theoretical aspects needed through this dissertation. First, the evaluation measures for ML models, namely performance and observational fairness metrics, are presented. Afterwards, the centralised training approach, notably the MLP, is discussed. Finally, the fundamental concepts related to FL are covered, including widely used algorithms as well as differential privacy.

3.1 Evaluation Metrics

To be able to evaluate an ML model, it is necessary to define the metrics that quantify the model's correctness, accuracy, and fairness. This section presents both performance and observational fairness metrics and discusses some inherent trade-offs associated with the latter.

3.1.1 Performance Metrics

When dealing with a binary classifier, the most comprehensive evaluation of its performance is provided by a confusion matrix, represented in Table 3.1, which shows the four possible outcomes for the predictions. From the confusion matrix, it is possible to determine the accuracy of the model, which is the proportion of correct predictions to all predictions made:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} .$$

However, accuracy is not the most suitable metric for fraud detection models due to data imbalance,

Table 3.1: Binary classification confusion matrix. In the context of fraud detection, positive corresponds to fraudulent samples while negative correspond to legitimate ones.

		Ground-truth label	
		0 (Negative)	1 (Positive)
Predicted label	0	True Negative (TN)	False Negative (FN)
	1	False Positive (FP)	True Positive (TP)

i.e., the fact that fraud is rare. Even if the FDS incorrectly predicts all instances as legitimate, accuracy will still be very highly valued. Instead, the common performance metrics used for FDSs include the area under the ROC curve (AUC) and a cost-sensitive learning approach [79].

The ROC curve depicts the performance of a classification model by showing the true positive rate (TPR) versus the false positive rate (FPR) across all classification thresholds. The TPR, or recall, defined as

$$\text{Recall} = \frac{TP}{TP + FN},$$

measures the proportion of fraudulent samples that are correctly predicted as such. On the other hand, the FPR refers to the fraction of negative cases incorrectly classified as positive and is given by

$$\text{FPR} = \frac{FP}{FP + TN}.$$

The AUC score is used to measure the model's ability to distinguish between the classes and can be interpreted as the probability of a classifier ranking frauds higher than legitimate instances [79]. Another common performance evaluation metric is the F1-score, which combines precision and recall by taking their harmonic mean, yielding:

$$\text{F1-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}},$$

where precision is the fraction of predicted positives that are actual fraud:

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (3.1)$$

The second usual method for measuring FDSs' performance is cost-sensitive learning [80]. Through the use of a cost matrix, which assigns a cost to each element of the confusion matrix, cost-sensitive metrics are able to quantify the loss of a fraud. The domain and application determine the specific values of the cost matrix. For example, false positives (false alarms) in medicine are typically less costly than false negatives (undetected diseases).

However, it might be difficult to collect a cost matrix from FIs, needed to develop a good cost-sensitive learning method for fraud detection. Hence, a Neyman-Pearson criterion [81] can be used to specify the optimisation goal, such as maximising recall subject to a fixed FPR. In these circumstances, several threshold values should be tested in the model validation set, and the corresponding predicted FPR should be determined for each value. The threshold value with the predicted FPR closest to, but below the intended fixed value is then selected.

3.1.2 Observational Fairness Metrics

In the context of this work, the focus is on assessing observational group fairness metrics, rather than individual fairness, since the latter relies on measuring similarity between individuals and may induce bias. Therefore, to gauge the equitability of the algorithms' predicted outcomes across groups defined

by the sensitive attribute S , the following observational group fairness metrics are considered.

Statistical parity, or demographic parity (Definition 3.1), states that both protected and unprotected groups have an equal probability of being assigned to the positive (or negative) predicted class, regardless of the underlying target prevalence. However, statistical parity might fail to take into account the bias introduced by the ground-truth label itself. For example, in the context of fraud detection, if one group has a much higher fraud rate than another, a predictor can be expected to predict more positive outcomes for the former group than the latter.

Definition 3.1 (Statistical Parity [27]). A predictor $\hat{Y}$ has statistical parity with respect to S if

$$\mathbb{P}(\hat{Y} = 1 \mid S = 0) = \mathbb{P}(\hat{Y} = 1 \mid S = 1).$$

Equality of opportunity, in Definition 3.2, refers to parity in the TPR, i.e., the probability of being correctly classified as positive is the same across different groups. It is important to note that, unlike statistical parity, this fairness concept considers the target prevalence.

Definition 3.2 (Equality of Opportunity [28]). A binary classifier $\hat{Y}$ satisfies equality of opportunity with respect to S and Y if

$$\mathbb{P}(\hat{Y} = 1 \mid Y = 1, S = 0) = \mathbb{P}(\hat{Y} = 1 \mid Y = 1, S = 1).$$

Finally, predictive equality, as stated in Definition 3.3, refers to parity in the FPR, i.e., the probability of being incorrectly classified as positive does not change with the group.

Definition 3.3 (Predictive Equality [29]). A binary classifier $\hat{Y}$ satisfies predictive equality with respect to S and Y if

$$\mathbb{P}(\hat{Y} = 1 \mid Y = 0, S = 0) = \mathbb{P}(\hat{Y} = 1 \mid Y = 0, S = 1).$$

As stated in Chapter 2, each application requires a distinct strategy when it comes to fairness metrics. As such, it is relevant to the topic to introduce some inherent trade-offs in observational fairness.

Fairness-Fairness Trade-offs

Fairness often involves internal conflicts. Multiple works have demonstrated the challenge of simultaneously meeting multiple fairness criteria. For instance, Chouldechova [82] demonstrates that when a classifier score is calibrated and there are disparities in group-wise prevalences, achieving a balance between FPR and false negative rate (FNR) becomes unattainable. Additionally, Speicher et al. [83] compare between-group and in-group fairness metrics and show that optimising for one can potentially have a detrimental impact on the other.

To better understand the aforementioned trade-offs and how they emerge from the interplay between the data and the classifier, Chouldechova [82] shows that

$$\text{FPR} = \frac{p}{1-p} \frac{1 - \text{PPV}}{\text{PPV}} (1 - \text{FNR}), \quad (3.2)$$

where $\text{FNR} = 1 - \text{TPR}$, p is the prevalence, and PPV stands for positive predicted value, also referred to as precision, given by (3.1). From (3.2), it is clear that when prevalence p varies between groups but PPV is balanced, achieving FPR parity must imply sacrificing FNR parity. The only way to counteract this is to ensure that either all the groups have the same prevalence of the target variable or the classifier is perfect [84].

In conclusion, although different methods achieve distinct trade-offs between fairness and accuracy, they consistently exhibit a common pattern: the group with a higher prevalence experiences a disproportionate impact from false positives, resulting in a lesser impact from false negatives [7]. Since prevalence is only related to the data itself, examining group-wise prevalence is crucial for making well-informed decisions regarding fairness metrics selection. Therefore, as explored in this dissertation, fraud detection requires ensuring that the FDSs implemented do not unduly discriminate against specific protected sub-groups of the population, either directly or indirectly. Thus, and considering what was mentioned above, in this work, fairness with respect to the label positives is assessed by measuring the ratio of group-wise TPR, defined as

$$\text{TPR Parity} = \min \left(\frac{\text{TPR}_{S=0}}{\text{TPR}_{S=1}}, \frac{\text{TPR}_{S=1}}{\text{TPR}_{S=0}} \right).$$

Similarly, fairness with respect to the label negatives is determined as

$$\text{FPR Parity} = \min \left(\frac{\text{FPR}_{S=0}}{\text{FPR}_{S=1}}, \frac{\text{FPR}_{S=1}}{\text{FPR}_{S=0}} \right), \quad (3.3)$$

i.e., the ratio between group-wise FPR. Equalising the two TPR and FPR parities to one ensures both equality of opportunity and predictive equality. This means that an approximately equal proportion of fraud is detected for each sub-group, while simultaneously preventing any sub-group from being disproportionately denied access to banking services [7].

3.2 Centralised Learning

As stated in Chapter 2, FDSs are traditionally trained in a centralised way, meaning that the ML models used by FIs to find patterns and detect fraud are trained using data stored in a central server. An effective supervised learning technique used is artificial neural networks (ANNs) due to their strong learning and predictive capacities [85]. In particular, the MLP has been proven to be effective in fraud detection, outperforming numerous other well-known ML classification methods [86, 87]. Thus, the MLP is selected as the baseline for this work, and the next section discusses the underlying theoretical concepts, based on [88–90].

3.2.1 Multi-Layer Perceptron

An MLP is a fully connected feedforward ANN, which is usually trained by gradient backpropagation. It consists of three different types of layers: one input layer, comprised of nodes corresponding to the components of a vector x (feature vector); one or more hidden layers of neurons; one output layer,

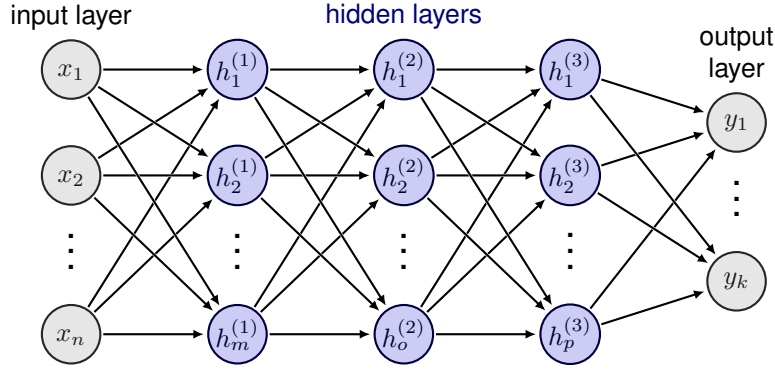


Figure 3.1: Multi-layer perceptron architecture (adapted from [91]).

consisting of units whose values are the output of the network. Figure 3.1 illustrates the architecture of an MLP with three hidden layers.

These models are termed feedforward because there are no backward connections, i.e., feedback, in the network. The goal of an MLP is to approximate some function f . In the case of a classifier, for instance, $y = f(x; \theta)$ maps an input x to a corresponding category y , by learning the θ parameters (weights and biases) to achieve the most accurate function approximation.

In each node of each layer of the network, the values of the previous layer's nodes are combined with the weights, typically through a pre-activation function, $z : \mathbb{R}^n \rightarrow \mathbb{R}$, consisting of a linear aggregator and a bias term. The output of z is called activation potential. Considering the j -th unit of the l -th layer, $z_j^{(l)}$ is defined as follows

$$z_j^{(l)} = \mathbf{w}_j^{(l-1)\top} \mathbf{h}^{(l-1)} + b_j^{(l)} = \sum_{i=1}^n w_{j,i}^{(l-1)} h_i^{(l-1)} + b_j^{(l)},$$

where $\mathbf{w}_j^{(l-1)} \in \mathbb{R}^n$ are the connecting weights, $b_j^{(l)} \in \mathbb{R}$ is the bias, and $\mathbf{h}^{(l-1)} \in \mathbb{R}^n$ are the outputs of the n units in the preceding layer. Then, a non-linear function named the activation function, $\sigma : \mathbb{R} \rightarrow \mathbb{R}$, is applied to the activation potential to get the output of the unit. Thus, the result of the node can be represented as

$$h_j^{(l)} = \sigma(z_j^{(l)}) = \sigma\left(\sum_{i=1}^n w_{j,i}^{(l-1)} h_i^{(l-1)} + b_j^{(l)}\right).$$

The previous equation can alternatively be expressed in matrix form to represent all the m units in the l -th layer, yielding

$$\begin{pmatrix} h_1^{(l)} \\ h_2^{(l)} \\ \vdots \\ h_m^{(l)} \end{pmatrix} = \sigma \left[\begin{pmatrix} w_{1,1}^{(l-1)} & w_{1,2}^{(l-1)} & \dots & w_{1,n}^{(l-1)} \\ w_{2,1}^{(l-1)} & w_{2,2}^{(l-1)} & \dots & w_{2,n}^{(l-1)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,1}^{(l-1)} & w_{m,2}^{(l-1)} & \dots & w_{m,n}^{(l-1)} \end{pmatrix} \begin{pmatrix} h_1^{(l-1)} \\ h_2^{(l-1)} \\ \vdots \\ h_n^{(l-1)} \end{pmatrix} + \begin{pmatrix} b_1^{(l)} \\ b_2^{(l)} \\ \vdots \\ b_m^{(l)} \end{pmatrix} \right],$$

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l-1)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}).$$

It is important to note that when designing the MLP, the architecture, i.e., the number of layers and neurons, must be carefully selected and rigorously tested to avoid both over- and underfitting.

Activation functions

There are multiple types of activation functions that should be used according to the problem the NN is working on. For classification problems, the hidden layers commonly use activation functions such as Rectified Linear Unit (ReLU), leaky ReLU, and hyperbolic tangent (TanH). These functions are defined below:

$$\begin{aligned} \text{ReLU: } \sigma(x) &= \max(0, x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}, \\ \text{Leaky ReLU: } \sigma(x) &= \max(0.01x, x) = \begin{cases} 0.01x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}, \\ \text{Tanh: } \sigma(x) &= \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}. \end{aligned} \quad (3.4)$$

The output layer of a binary classifier usually contains only one neuron and the sigmoid activation function, defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad (3.5)$$

is normally applied, since its output ranges between zero and one and may thus be interpreted as the probability of being assigned to the first class.

Training and backpropagation

Following a forward pass through the network, the parameters $\theta = \{W, b\}$ must be optimised using a training set of labelled patterns $\mathcal{T} = \{(x_i, y_i), i = 1, \dots, n\}$, which is referred to as training the model. The optimisation goal is to minimise a total loss (cost) function as the network learns from the training data. A common loss function for binary classification is the binary cross-entropy (BCE) loss, defined as

$$\mathcal{L}_{BCE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{m} \sum_{i=1}^m \ell(y_i, \hat{y}_i) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)], \quad (3.6)$$

where $m < n$ is the batch size, y_i corresponds to the ground-truth label for sample i , and $\hat{y}_i = f(x_i; \theta)$ stands for its prediction. The cost function can be minimised with respect to the network parameters θ using a version of the gradient descent (GD), for example, mini-batch GD:

$$\theta_{t+1} = \theta_t + \Delta\theta_t, \quad \text{where} \quad \Delta\theta_t = -\eta \nabla_{\theta} \mathcal{L}(\mathbf{x}, \mathbf{y}; \theta_t) = -\frac{\eta}{m} \sum_{i=1}^m \frac{\partial \ell(x_i, y_i; \theta_t)}{\partial \theta_t}.$$

The parameter $\eta \in \mathbb{R}^+$ denotes the learning rate and controls the update step size. One iteration corresponds to using a mini-batch to update the parameters. After all the mini-batches have been iterated, an epoch has been performed.

The computation of these gradients, i.e., the training process, can be achieved using the backpropagation procedure. This technique consists of a practical application of the chain rule for derivatives. The critical insight is that the derivative of the cost function with respect to a layer's input may be calculated

by working backwards from the gradient with respect to that layer's output. The backpropagation can be applied repeatedly to propagate gradients across all layers, starting from the network's output to the input layer. It is then simple to compute the gradients with respect to the weights and biases of each layer once these gradients have been obtained.

An alternative to the GD algorithm could be the Adaptive Moment Estimation (Adam), an optimiser for first-order gradient-based optimisation of stochastic cost functions [92]. Adam estimates the first and second moments of the gradients to compute individual adaptive learning rates for different parameters. This method is computationally efficient and insensitive to gradient rescaling, requires little memory, and is ideally suited for problems with huge amounts of data and/or parameters.

3.3 Federated Learning

FL has emerged as a viable alternative to the centralised approach, enabling individual clients (co-ordinated by a central server) to collaboratively learn a shared model. This methodology maintains the decentralisation and confidentiality of their local training data, ensuring it remains secluded from the central server and other participants. This section introduces the fundamental principles of HFL, including the algorithms employed in this dissertation, based on [10, 52, 53]. Furthermore, the definition of differential privacy (the privacy-enhancing technique) is provided.

3.3.1 Problem Formulation

This section concerns the formulation of the FL framework, based on the work by McMahan et al. [10]. In this setting, the optimisation problem is referred to as federated optimisation and, among all the characteristics that set it apart from a conventional distributed optimisation problem, this thesis focuses specifically on its non-IID and unbalanced properties.

Considering that the communication between the server and clients occurs in rounds within a synchronous update scheme and that there is a fixed set of K clients, each with a local dataset, the typical horizontal federated training procedure can be described as follows:

- i.* A random fraction C of clients is selected at the beginning of each round;
- ii.* The server sends the current global model (model parameters) to each of these clients;
- iii.* Each client performs local computation based on the shared model and its local dataset and sends the update to the server;
- iv.* The server collects and aggregates the client updates;
- v.* Finally, the global model is updated based on the previously aggregated updates and the process repeats.

Figure 3.2 depicts a diagram of this training process.

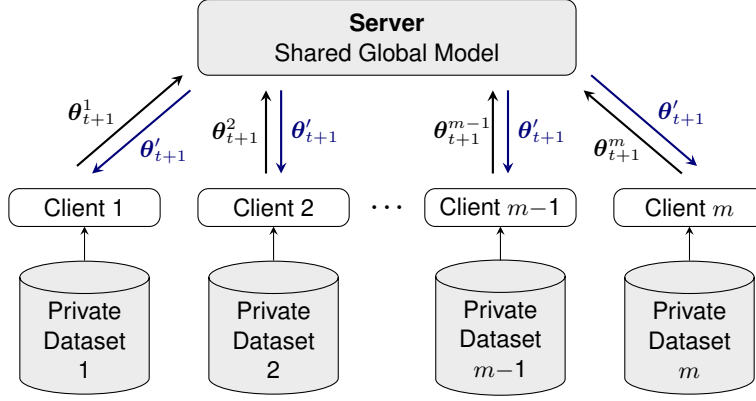


Figure 3.2: Diagram of the federated training process (adapted from [9]). θ^k_{t+1} corresponds to the local update of client k sent to the server and θ'_{t+1} are the model parameters for the next round, after aggregation by the server.

Considering non-convex NN objective functions, the explored FL approaches apply to any finite-sum objective of the form

$$\min_{\theta} f(\theta), \quad \text{where} \quad f(\theta) := \frac{1}{n} \sum_{i=1}^n f_i(\theta). \quad (3.7)$$

For ML problems, $f_i(\theta) = \ell(x_i, y_i; \theta)$ corresponds to the loss of the prediction on sample (x_i, y_i) using the model parameters θ . Considering that the data is partitioned over K clients, with $\mathcal{P}_k$ the set of indexes of data points on client k and $n_k = |\mathcal{P}_k|$, the objective (3.7) may be rewritten as

$$f(\theta) = \sum_{k=1}^K \frac{n_k}{n} F_k(\theta), \quad \text{where} \quad F_k(\theta) = \frac{1}{n_k} \sum_{i \in \mathcal{P}_k} f_i(\theta).$$

The IID assumption that distributed optimisation methods usually make lies in the fact that $\mathcal{P}_k$ is formed by distributing the training data uniformly at random among the clients. Thus, the expectation over the set of examples assigned to a fixed client k is $\mathbb{E}_{\mathcal{P}_k} [F_k(\theta)] = f(\theta)$. The case where this does not hold, i.e., F_k could be an arbitrarily bad approximation to f , is referred to as the non-IID setting.

3.3.2 Federated Averaging Method

As stated in Chapter 2, FedAvg is the most prevalent FL algorithm and is an extension of FedSGD that allows for reducing the communication cost by increasing computation on each client. This section describes the mathematical principles of the FedAvg algorithm, building upon the work introduced by McMahan et al. [10].

FedSGD selects a C -fraction of clients on each round of communication and performs a single batch gradient computation of the loss over all the data held by these clients. Therefore, C determines the global batch size, with $C = 1$ corresponding to full-batch (non-stochastic) GD.

Considering $C = 1$ and a fixed learning rate η , FedSGD consists of the clients computing the average gradients on its local data, $g_k = \nabla F_k(\theta_t)$, using the current model parameters θ_t , and then the server

aggregates these gradients and performs the update

$$\theta_{t+1} \leftarrow \theta_t - \eta \sum_{k=1}^K \frac{n_k}{n} g_k, \quad (3.8)$$

since $\sum_{k=1}^K \frac{n_k}{n} g_k = \nabla f(\theta_t)$. It is feasible to reformulate (3.8) so that each client k locally performs a GD step on the current model using its local data, and the server then computes a weighted average of the resulting models, yielding

$$\theta_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} \theta_{t+1}^k, \quad \text{where} \quad \theta_{t+1}^k \leftarrow \theta_t - \eta g_k. \quad (3.9)$$

FedAvg allows to increase the computation on the clients by repeatedly iterating the local update $\theta^k \leftarrow \theta^k - \eta \nabla F_k(\theta^k)$ prior to the aggregation step. The parameters that control the computation workload are:

- C , the fraction of clients participating in each round;
- E , the training passes each client does on its local dataset during each round, referred to as the number of local epochs; and
- B , the local mini-batch size, with $B = \infty$ representing the full dataset as a single mini-batch.

The number of local updates per round is En_k/B , where n_k is the number of local examples on client k . FedAvg is a generalisation of FedSGD since the latter is a special case of the former, where $E = 1$ and $B = \infty$. The pseudo-code for FedAvg is given in Algorithm 1.

Algorithm 1 Federated Averaging (FedAvg) [10]. Here, K corresponds to the total number of clients, indexed by k , C is the fraction of clients participating in the federation, E corresponds to the number of local epochs, B is the local mini-batch size, T refers the number of rounds, n_k is the number of examples in client k , $n = \sum_k n_k$, and η is the learning rate.

Server executes:

```

1: initialise  $\theta_0$ 
2: for each round  $t = 0, \dots, T - 1$  do
3:    $m \leftarrow \max(C \cdot K, 1)$ 
4:    $\mathcal{S}_t \leftarrow$  (random set of  $m$  clients)
5:   for each client  $k \in \mathcal{S}_t$  in parallel do
6:      $\theta_{t+1}^k \leftarrow \text{CLIENTUPDATE}(k, \theta_t)$ 
7:   end for
8:    $\theta_{t+1} \leftarrow \sum_{k \in \mathcal{S}_t} \frac{n_k}{n} \theta_{t+1}^k$  ▷ Weighted average of the resulting models
9: end for
```

CLIENTUPDATE(k, θ):

▷ Run on client k

```

1:  $\mathcal{B} \leftarrow$  (split  $\mathcal{P}_k$  into batches of size  $B$ )
2: for each local epoch  $e = 0, \dots, E - 1$  do ▷ Increase client-side computation
3:   for batch  $b \in \mathcal{B}$  do
4:      $\theta \leftarrow \theta - \eta \nabla \ell(b; \theta)$ 
5:   end for
6: end for
7: return  $\theta$  to server
```

3.3.3 Alternative Methods in Federated Optimisation

Within the various alternative methods to FedAvg mentioned in Chapter 2, this dissertation focuses on implementing the following two: FedProx and FedOpt. Building upon the foundation of FedAvg, these algorithms incorporate targeted modifications to address the convergence challenges it encounters in certain settings, such as handling heterogeneous data. In addition, FedOpt tackles the issue of adaptivity deficiency by incorporating adaptive optimisers in federated settings.

This section provides background information on these methods and is grounded in the respective publications [52, 53].

FedProx

FedProx [52] addresses both statistical and system heterogeneity issues. It is a generalisation of FedAvg that allows for variable amounts of computation locally across clients based on their available systems resources. Unlike FedAvg, clients are not forced to run the same number of local epochs, E . To achieve this, each of the local objectives may be solved **inexactly**. This notion is defined below as an extension from the definition of γ -inexact solution since FedProx adjusts multiple γ for distinct clients at different rounds. Thus, Definition 3.4 introduces γ_t^k -inexactness for client k at iteration t .

Definition 3.4 (γ_t^k -inexact solution [52]). Given a function $h_k(\theta; \theta_t) = F_k(\theta) + \frac{\mu}{2}\|\theta - \theta_t\|^2$, and $\gamma \in [0, 1]$, θ^* is said to be a γ_t^k -inexact solution of $\min_{\theta} h_k(\theta; \theta_t)$ if

$$\|\nabla h_k(\theta^*; \theta_t)\| \leq \gamma_t^k \|\nabla h_k(\theta_t; \theta_t)\|,$$

where $\nabla h_k(\theta; \theta_t) = \nabla F_k(\theta) + \mu(\theta - \theta_t)$. The parameter γ_t^k quantifies how much local computation is performed on client k to solve the local subproblem in the t -th round. Smaller values correspond to higher accuracy.

Nevertheless, while the variable amount of local updates can help mitigate the negative effects of system heterogeneity, too many local updates may also lead the models to diverge due to the underlying heterogeneous data. To limit this impact, FedProx includes a **proximal term** in the local subproblem. Instead of solely minimising the local function $F_k(\cdot)$, each client k minimises the objective h_k :

$$h_k(\theta; \theta_t) = F_k(\theta) + \underbrace{\frac{\mu}{2}\|\theta - \theta_t\|^2}_{\text{proximal term}}, \quad (3.10)$$

where $\mu \geq 0$ is the penalty constant.

The proximal term acts as a regulariser and tackles statistical heterogeneity by constraining the local updates to be closer to the initial shared global model, without the need to specify the number of local epochs. At the same time, it enables the seamless incorporation of varied amounts of local computation caused by system heterogeneity. Algorithm 2 summarises the steps of FedProx.

It is important to note that FedProx only makes slight modifications to FedAvg, making it simple to incorporate it into existing packages. In fact, FedAvg is a specific case of FedProx with $\mu = 0$, a variant

Algorithm 2 FedProx [52]. In comparison to FedAvg (Algorithm 1), FedProx introduces two additional variables: γ and the penalty constant μ .

```

1: Input:  $T, K, C, \mu, \gamma, \theta_0$ 
2: for round  $t = 0, \dots, T - 1$  do
3:   Server selects a subset  $S_t$  of  $CK$  clients at random
4:   Server sends  $\theta_t$  to all selected clients
5:   Each client  $k \in S_t$  finds a  $\theta_{t+1}^k$  which is a  $\gamma_t^k$ -inexact minimiser of:
6:      $\theta_{t+1}^k \approx \arg \min_{\theta} h_k(\theta; \theta_t) = F_k(\theta) + \frac{\mu}{2} \|\theta - \theta_t\|^2$ 
7:   Each client  $k \in S_t$  sends  $\theta_{t+1}^k$  back to the server
8:   Server aggregates the updates as  $\theta_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{n} \theta_{t+1}^k$ 
9: end for

```

of SGD specifically as the local solver, and a constant γ (corresponding to E) for all clients throughout every round, i.e., there is no concept of system heterogeneity. The method proposed by Li et al. [52] is far broader in this sense, enabling the utilisation of any local solver on individual clients, including potentially non-iterative ones.

FedOpt

FedOpt, introduced by Reddi et al. [53], is a framework that integrates adaptivity in FL. This method allows for both client and server optimisations: while clients use a client optimiser to minimise loss on their local data, the server updates its global model by applying a gradient-based server optimiser to the average of the clients' model updates.

As a result, adaptivity can be implemented by employing adaptive optimisers as client or server optimisers. At the same time, by emphasising adaptive server optimisation, it is possible to use adaptive learning rates without increasing communication costs or client storage, while ensuring compatibility with cross-device FL.

Considering the FedAvg's update and a subset of clients S_t at round t , it is possible to reformulate (3.9) as

$$\theta_{t+1} = \sum_{k \in S_t} \frac{n_k}{n} \theta_{t+1}^k = \theta_t - \sum_{k \in S_t} \frac{n_k}{n} (\theta_t - \theta_{t+1}^k),$$

where $\Delta_t^k = \theta_{t+1}^k - \theta_t$ and $\Delta_t = \sum_{k \in S_t} \frac{n_k}{n} \Delta_t^k$. Hence, the server update in FedAvg is equivalent to applying SGD to $-\Delta_t$ with learning rate $\eta = 1$. This expression illustrates that there are multiple options for η and that one could also employ optimisers on the clients aside from SGD, or opt for an alternative update approach on the server. This family of algorithms, which is a generalisation of FedAvg, are denoted as FedOpt.

Algorithm 3 provides the pseudo-code for FedOpt with different adaptive server optimisations (Adaptive Gradient (Adagrad), Yogi, and Adam) and a mini-batch GD client optimiser. The parameter τ determines the degree of adaptivity of the algorithms, where smaller values reflect higher adaptivity.

It is important to note that this method can be extended to adaptive optimisers other than those mentioned above and that the server and clients learning rates, η and η_l respectively, may vary with the communication rounds in order to accommodate learning rate schedules.

Algorithm 3 FedOpt: FedAdagrad, FedYogi, and FedAdam [53]. Here, η and η_l correspond to the server and clients learning rates, respectively, β_1 and β_2 are the decay parameters, and τ controls the algorithms' degree of adaptivity. The remaining variables maintain the same meaning as in Algorithm 1.

```

1: Initialisation:  $T, K, C, E, \eta_l, \eta, \theta_0, v_{-1} \geq \tau^2, m_{-1}, \beta_1, \beta_2 \in [0, 1)$ 
2: for round  $t = 0, \dots, T - 1$  do
3:   Server selects a subset  $\mathcal{S}_t$  of  $CK$  clients at random
4:   for each client  $k \in \mathcal{S}_t$  in parallel do
5:      $\theta_{t+1}^k \leftarrow \text{CLIENTOPT}(k, \theta_t, \eta_l)$  ▷ Client optimisation
6:      $\Delta_t^k = \theta_{t+1}^k - \theta_t$ 
7:   end for
8:    $\Delta_t = \sum_{k \in \mathcal{S}_t} \frac{n_k}{n} \Delta_t^k$ 
9:    $\theta_{t+1}, m_t, v_t \leftarrow \text{SERVEROPT}(\theta_t, m_{t-1}, v_{t-1}, \Delta_t, \eta, \beta_1, \beta_2)$  ▷ Server optimisation
10: end for

```

CLIENTOPT(k, θ, η_l): ▷ Run on client k

```

1:  $\mathcal{B} \leftarrow$  (split  $\mathcal{P}_k$  into batches of size  $B$ )
2: for each local epoch  $e = 0, \dots, E - 1$  do
3:   for batch  $b \in \mathcal{B}$  do
4:      $\theta \leftarrow \theta - \eta_l \nabla \ell_k(b; \theta)$  ▷ SGD optimiser
5:   end for
6: end for
7: return  $\theta$  to server

```

SERVEROPT($\theta_t, m_{t-1}, v_{t-1}, \Delta_t, \eta, \beta_1, \beta_2$):

```

1:  $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \Delta_t$ 
2:  $v_t = v_{t-1} + \Delta_t^2$  (FedAdagrad)
3:  $v_t = v_{t-1} - (1 - \beta_2) \Delta_t^2 \text{sign}(v_{t-1} - \Delta_t^2)$  (FedYogi)
4:  $v_t = \beta_2 v_{t-1} + (1 - \beta_2) \Delta_t^2$  (FedAdam)
5:  $\theta_{t+1} = \theta_t + \eta \frac{m_t}{\sqrt{v_t} + \tau}$  ▷ Server update

```

3.4 Differential Privacy

As stated in Chapter 2, differential privacy is the most widely used privacy-preserving approach in FL and aims to add some randomness to the released federated model, making it harder to determine the specific contribution of any individual sample or user. As such, this is the privacy technique implemented in this dissertation.

According to Definition 3.5, DP is assessed using the privacy parameters $\varepsilon > 0$ and $\delta \in [0, 1)$, with smaller values indicating higher levels of privacy. It is imperative to emphasise that the definition of DP remains independent of whether it pertains to user-level or sample-level implementations. The sole distinction between them resides in the concept of adjacency. In sample-level DP, two datasets are deemed adjacent if they differ in only one sample. Conversely, in user-level DP, D' is derived from D by including or excluding all records from a single client.

Definition 3.5 (Differential Privacy [93]). A randomised algorithm $\mathcal{M}$ with domain $\mathcal{D}$ is (ε, δ) -differentially private if for all $S \subseteq \text{Range}(\mathcal{M})$ and for all adjacent datasets D and $D' \in \mathcal{D}$ it holds that

$$P[\mathcal{M}(D) \in S] \leq \exp(\varepsilon)P[\mathcal{M}(D') \in S] + \delta, \quad \varepsilon > 0, \delta \in [0, 1),$$

where ε is a bound on the privacy loss of $\mathcal{M}$ [94] that can be computed to strike a balance between privacy and accuracy and quantifies the level of privacy risk arising from the disclosure of information derived from confidential data [95]. The parameter δ accounts for the probability that plain ε -DP is broken. This pair of parameters (ε, δ) jointly characterise the privacy budget.

To ensure (ε, δ) -DP for numerical data, one can employ a Gaussian mechanism (GM). The GM approximates a real-valued function $f : \mathcal{D} \rightarrow \mathbb{R}$ with a differentially private mechanism, achieved by adding zero-mean Gaussian noise calibrated to f 's ℓ_2 -sensitivity. This sensitivity is defined as follows:

Definition 3.6 (ℓ_2 -sensitivity [93]). The ℓ_2 -sensitivity of a function $f : \mathcal{D} \rightarrow \mathbb{R}$, where D and D' are adjacent datasets, is given by:

$$\Delta_2(f) = \max_{D, D' \in \mathcal{D}} \|f(D) - f(D')\|_2,$$

where $\|\cdot\|_2$ represents the L2-norm.

For instance, the Gaussian noise mechanism is defined as $\mathcal{M}(D) = f(D) + \mathcal{N}(0, \sigma^2 \Delta_2^2(f))$ where $\mathcal{N}(0, \alpha^2)$ denotes a normal distribution with zero mean and standard deviation α [96].

3.4.1 Sample-Level Differential Privacy

In Section 2.3.1, the difference between user-level DP and sample-level DP was first introduced. In summary, the key distinction lies in where the perturbation (i.e., the noise) is added to achieve DP. User-level DP, the most used DP technique in FL, adds noise on the server side during the aggregation step, protecting the privacy of individual clients. Contrarily, sample-level DP focuses on adding noise directly to the clients' gradients or other local updates during training, before sending them to the server, thus ensuring the protection of a single instance's contribution in learning a model.

The choice between the two approaches depends on the specific privacy and security requirements of the FL system. In the context of this work, where the clients involved in the federation represent FIs, the primary focus is on protecting the privacy of individual customers (i.e., persons) within each FI. Therefore, sample-level DP is the selected technique as it aligns to ensure the privacy of individual data samples within FIs, rather than protect the FIs' contributions to the federation.

One might opt for applying DP directly to the raw data for its protection. However, adding noise does have implications for accuracy, and the later DP is applied in the pipeline, the more advantageous it becomes. Hence, a prevalent strategy in ML models involves employing the DP version of the SGD optimiser.

Introduced by Abadi et al. [96], Differentially Private Stochastic Gradient Descent (DP-SGD) modifies SGD to ensure DP on every model parameters update. Algorithm 4 outlines this method for training a model with parameters θ by minimising the empirical loss function $\mathcal{L}(\theta)$. At each step of SGD, rather than computing the average of gradients over a batch of samples, DP-SGD computes per-instance gradients, applies an ℓ_2 norm clipping, aggregates these individual gradients into a batch gradient, and adds Gaussian noise to protect privacy. Figure 3.3 provides a visual representation of this process. Following a GD iteration, it is also necessary to compute the privacy budget of the mechanism based on the information held by the privacy accountant.

Algorithm 4 Differentially Private Stochastic Gradient Descent (DP-SGD) [96].

```

1: Input: Examples  $\{x_1, \dots, x_N\}$ , loss function  $\mathcal{L}(\theta) = \frac{1}{N} \sum_i \mathcal{L}(x_i, \theta)$ . Parameters: learning rate  $\eta$ ,
   noise scale  $\sigma$ , group size  $L$ , gradient norm bound  $\Gamma$ .
2: Initialise  $\theta_0$  randomly
3: for  $t \in [T]$  do
4:   Take a random sample  $L_t$  with sampling probability  $L/N$ 
5:   For each  $i \in L_t$ , compute  $\mathbf{g}_t(x_i) \leftarrow \nabla_{\theta_t} \mathcal{L}(x_i; \theta_t)$  ▷ Compute gradient
6:    $\bar{\mathbf{g}}_t(x_i) \leftarrow \mathbf{g}_t(x_i) / \max\left(1, \frac{\|\mathbf{g}_t(x_i)\|_2}{\Gamma}\right)$  ▷ Gradient clipping
7:    $\tilde{\mathbf{g}}_t \leftarrow \frac{1}{L} \left( \sum_i \bar{\mathbf{g}}_t(x_i) + \mathcal{N}(0, \sigma^2 \Gamma^2 \mathbf{I}) \right)$  ▷ Add noise
8:    $\theta_{t+1} \leftarrow \theta_t - \eta \tilde{\mathbf{g}}_t$  ▷ Gradient descent
9: end for
10: Output:  $\theta_T$  and compute the overall privacy budget  $(\varepsilon, \delta)$  using a privacy accountant method.
```

To establish the differential privacy guarantee of Algorithm 4, it is essential to constrain the influence of each example on $\tilde{\mathbf{g}}_t$. Given the absence of *a priori* bound on the gradients, DP-SGD employs an ℓ_2 norm clipping on each gradient, for a clipping threshold Γ . This ensures that if $\|\mathbf{g}\|_2 \leq \Gamma$, $\mathbf{g}$ remains unchanged, whereas if $\|\mathbf{g}\|_2 > \Gamma$, $\mathbf{g}$ it gets scaled down to be of norm Γ .

Furthermore, in Algorithm 4, all the parameters are combined into a single input θ of the loss function $\mathcal{L}(\cdot)$. However, in multi-layer NNs, each layer is considered separately, as shown in Figure 3.3, affording the flexibility to configure distinct clipping thresholds Γ and noise scales σ for different layers. Moreover, Γ and σ may also vary with the number of training steps t .

The privacy accountant method enables the calculation of the overall privacy budget by computing the privacy cost at each access to the training data and accumulating it as the training advances. Abadi et al. [96] also propose a privacy accountant mechanism, called Moments Accountant and provided in Theorem 3.1, that offers a tight bound on the privacy loss.

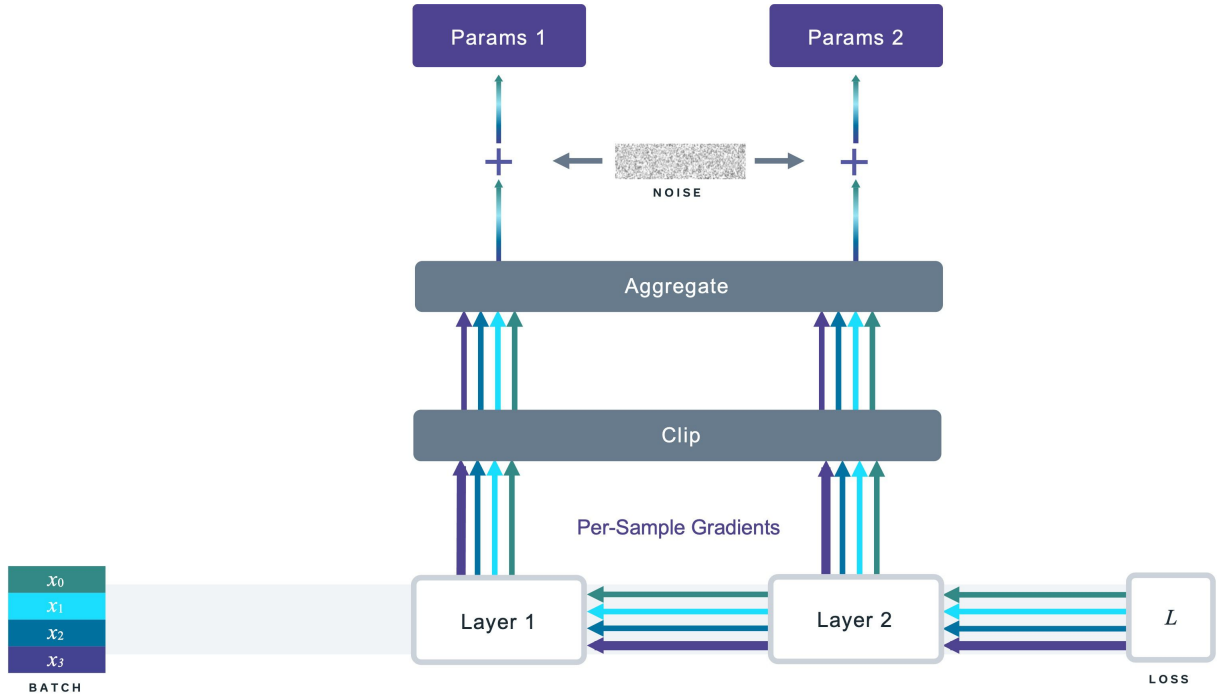


Figure 3.3: Visual depiction of the DP-SGD algorithm. Single-coloured lines represent per-sample gradients, the line width reflects their respective norms, and the multi-coloured lines symbolise the combined gradients [97].

Theorem 3.1 (Moments Accountant [96]). There exist constants c_1 and c_2 so that given the sampling probability $q = L/N$ and the number of steps T , for any $\varepsilon < c_1 q^2 T$, Algorithm 4 is (ε, δ) -differentially private for any $\delta > 0$ if

$$\sigma > c_2 \frac{q \sqrt{T \log(1/\delta)}}{\varepsilon}.$$

Chapter 4

Centralised vs. Federated Learning on IID Settings

Feedzai is a SaaS company that provides a platform for managing financial risk, with a primary emphasis on preventing financial fraud. Financial fraud can take various forms, such as transaction fraud, bank account opening fraud, or even money laundering. The goal of this work is to develop an FL approach to detect financial fraud by considering both the performance and fairness of the resulting ML models. In this chapter, we develop an FL model and conduct a performance comparison with a centralised version, which has access to the entire dataset, starting within the context of IID settings.

4.1 Bank Account Fraud Dataset Suite

The focal point of this thesis centres on financial fraud detection, involving the analysis of tabular data for binary classification tasks. To accomplish this, the Bank Account Fraud (BAF) suite of tabular datasets, introduced by Jesus et al. [98], is used. It is a synthetic replication of a real-world online bank account opening fraud detection dataset, generated using a Conditional Tabular Generative Adversarial Network.

In account opening, fraudsters endeavour to impersonate legitimate individuals through identity theft or fabricate fictitious personas to gain access to banking services. If their fraudulent attempts prove successful, they either exploit the assigned line of credit to its maximum extent or employ the account for receiving illicit payments. In light of the impossibility of confirming the true identities of fraudsters, the FIs assume complete responsibility for all related costs.

The BAF suite is composed of six datasets, where each instance represents an individual application, labelled as negative (legitimate) or positive (fraudulent). The temporal distribution shifts and inherent bias conditions enable the evaluation of both the performance and fairness of different ML models. The data contains eight months of information (February to September).

To achieve a more comprehensive understanding of the impacts of biased data and distribution shifts, it is essential to have a dataset entirely free from these factors, serving as a baseline. This baseline

dataset should maintain consistent data distribution over time, and regarding fairness, it should ensure uniform group sizes and fraud rates among groups defined by the sensitive attribute. Therefore, a new variant of BAF is generated to align with the goals of this study.

4.1.1 BAF IID

To create an IID and unbiased BAF dataset version, BAF Variant II is adapted due to its prevalence disparity while keeping group sizes consistent. It is important to note that every variant exhibits distribution drift. To achieve the IID property, the temporal aspect was eliminated by shuffling the data and randomly splitting it into training, validation, and test sets (68%, 15%, and 17%, respectively). This process guarantees that the fraud rate remains uniform across all three sets. To establish unbiasedness, the relation between the sensitive attribute and both the remaining features and the label was also removed, by binarising the sensitive attribute-related feature and shuffling it.

In order to verify the IID and bias-free nature of the created dataset, a classifier was trained for two different tasks: predicting the data split and predicting the binarised sensitive attribute. The obtained AUC score of 0.5 for both tasks, indicative of random classifier behaviour, confirms that the dataset does not exhibit statistical relations between data splits or the sensitive attribute and the remaining data. Consequently, the BAF IID dataset remains devoid of distribution shifts and is established as unbiased. Table 4.1 provides an overview of this dataset, which is the dataset used at this stage of the work. Detailed information concerning fairness is supplied in Chapter 6.

Table 4.1: Summary of the BAF IID dataset.

Dataset split	Number of records	Fraud prevalence	Number of features
Training	680 000	1.10%	29
Validation	150 000	1.09%	29
Test	170 000	1.13%	29
Full BAF IID dataset	1 000 000	1.10%	29

4.2 Methodology

4.2.1 Centralised Model

This section delves into the implementation of the centralised model, which serves as a baseline for the comparative analysis with our proposed FL approach. The centralised approach uses a conventional DL model, trained on the entirety of the available dataset. This stands in opposition to FL, where the training process is decentralised and distributed across multiple clients.

Data pre-processing is a crucial step in constructing a robust centralised model. In this work, it is essential to ensure that the BAF dataset employed is optimally prepared for ML tasks. This dataset consists

of tabular data, encompassing 24 numerical and 5 categorical features. Therefore, the effectiveness of the centralised model depends on how we address these specific data characteristics.

Data Pre-Processing

The two primary aspects of data pre-processing that we focus on in this work are numerical feature normalisation and categorical feature encoding.

The main challenge when working with numerical features revolves around addressing the disparities in scales across different variables. Ensuring that features exhibit similar distributions is advantageous for accelerating the training process and preventing potential entrapment in local minima during optimisation. In this work, we employ z-score normalisation. This method transforms numerical features to have zero mean and a standard deviation of one. The z-score normalisation process can be expressed mathematically as

$$z_i = \frac{x_i - \bar{x}_i}{s_i},$$

where s_i is the standard deviation of feature vector x_i , and $\bar{x}_i$ its mean.

When dealing with categorical features, a practical approach involves mapping them into continuous representations. Hence, a target encoding technique is employed for this purpose. It consists of replacing each feature with the posterior probability of the target for each given category.

Through the implementation of these data pre-processing strategies, we ensure that the centralised model is suitably prepared to handle a diverse range of data, improving its adaptability and performance.

Implementation

As previously discussed in Section 3.2, the centralised model in this work is an MLP, due to its strong learning capabilities and proven effectiveness in fraud detection. The choice of hyperparameters is described in Appendix A. The hyperparameter optimisation target is the recall at 5% FPR. At that FPR level, the best recall obtained on the validation set was 48.69%. Table 4.2 summarises the MLP architecture of the best model. The selected batch size is 64, with a batch fraud prevalence of 8.50%. The model is trained with an early stopping of seven epochs of patience, using the Adam optimiser with a learning rate of 0.005, and it is optimised to minimise (3.6), i.e., the BCE loss function.

All DL algorithms employed in this thesis, including the centralised MLP, were implemented using the Python programming language [99], with the PyTorch library [100] serving as the primary framework.

Table 4.2: Multi-layer perceptron architecture for the IID settings.

Layer	Neurons	Activation function	Dropout rate
Input	29	–	–
2 × Hidden	4	Tanh (3.4)	0.0561
Output	1	Sigmoid (3.5)	–

4.2.2 Federated Learning Approach

The primary objective of this thesis is to design an FL approach tailored for the detection of fraudulent applications in bank account opening. In this context, the clients participating in the federation are representative of FIs, thereby establishing a scenario of cross-silo FL, as mentioned in Section 2.3, characterised by a limited number of clients. To delve into the complexities of federated optimisation, it is imperative to outline the data distribution across these clients.

Data Partitioning Across Clients

In the context of IID settings, we explore two distinct approaches for partitioning the data among clients: the example-partitioned and the feature-partitioned methods.

The first strategy consists of distributing the training, validation, and test data uniformly at random among the various clients, which ensures that each client receives an equal number of samples in their respective datasets. Consequently, this approach establishes a fully IID setting, both within individual clients and across the collective group. We term this the **uniform data distribution** approach. In our experiments, we engaged varying numbers of clients in the federation, specifically, 5, 8, and 16 clients.

On the other hand, the feature-partitioned method involves distributing data among clients based on a specific feature, often a categorical one. Each client is allocated the samples for which a particular feature exhibits a specific value. Consequently, the maximum number of clients possible corresponds to the number of distinct categories associated with the selected feature. This approach is denoted as the **stratified data distribution**. Within individual clients, the data remains IID, since the original dataset is also IID; however, it exhibits non-IID characteristics when distributed across clients. In our experimental setup, we split the BAF IID dataset based on the 'device_os' feature, comprising five distinct categorical values, thereby allowing the participation of 5 clients within the federation.

Table 4.3 depicts the number of samples and the fraud prevalence in each individual client's training set after both uniform and stratified distribution of the BAF IID dataset. There is a clear disparity in the training set sizes among each client for the stratified data distribution approach. Moreover, it is evident that the IID assumption does not hold across clients for the same distribution method, with the third client exhibiting a significantly higher fraud rate than the remaining clients. Contrarily, the size of the training set and the fraud prevalence remain constant across clients for the uniform data distribution.

After the data distribution among clients, it is imperative to note that the same pre-processing techniques employed in the centralised model are consistently applied within each dataset. Specifically, these techniques include z-score normalisation and target encoding.

Federated Averaging

In our pursuit of constructing an FL-based FDS within IID settings, we adopt the FedAvg methodology, recognised as one of the most prevalent and established FL frameworks. The algorithmic representation of this method is elaborated in Algorithm 1, comprehensively detailed in Section 3.3.2.

Table 4.3: Training set size and fraud prevalence across clients for both uniform and stratified distribution of the BAF IID dataset. Five clients are selected in the case of uniform data distribution for a fair comparison.

	Uniform Data Distribution		Stratified Data Distribution		
Client and set	Size	Fraud prevalence	Category 'device_os'	Size	Fraud prevalence
Client 1 – Training	136 000	1.13%	Other	203 659	0.51%
Client 2 – Training	136 000	1.08%	Linux	230 570	0.48%
Client 3 – Training	136 000	1.07%	Windows	206 818	2.33%
Client 4 – Training	136 000	1.08%	Macintosh	33 439	1.32%
Client 5 – Training	136 000	1.13%	X11	5 514	1.27%
BAF IID – Training	680 000	1.10%	—	680 000	1.10%
BAF IID – Test	170 000	1.13%	—	170 000	1.13%

To implement FedAvg and other FL strategies in this work, we employ Flower [101], a novel FL framework specifically designed to advance FL research. Flower offers framework-agnostic ways to federate ML pipelines, allowing us to choose which framework to use for local training. Therefore, it facilitates the seamless federation of our DL models, employed in PyTorch.

The central server is composed of a Client Manager, which manages a set of Client Proxy objects, and a Strategy, responsible for configuring the communication rounds, sending and receiving updates to and from clients, and performing aggregation. The clients are created using the Virtual Client Engine, a tool that enables the virtualisation of Flower Clients to maximise utilisation of the available hardware. Figure 4.1 illustrates the Flower core framework architecture with the Virtual Client Engine.

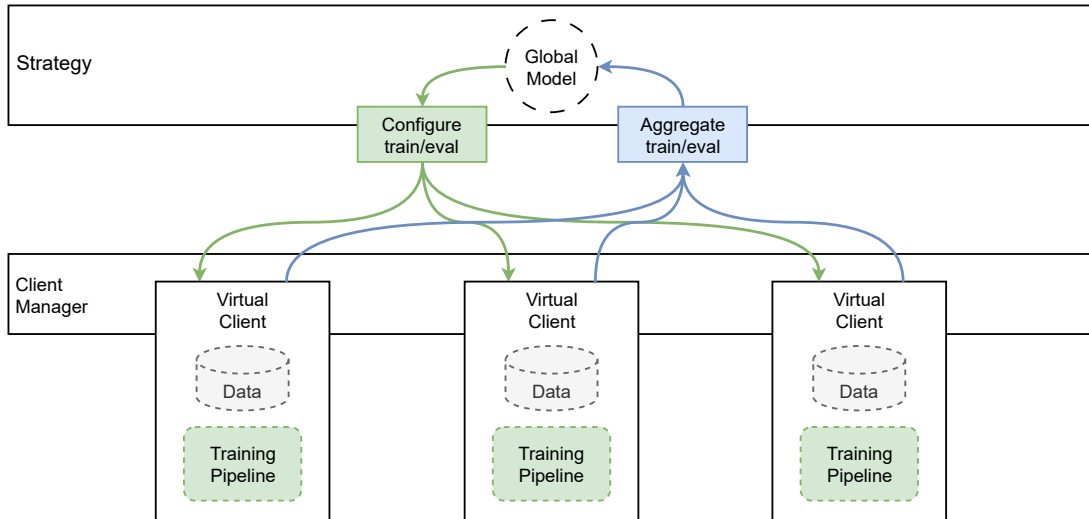


Figure 4.1: Flower core framework architecture with Virtual Client Engine (adapted from [101]). Virtual clients consume close to zero resources when inactive and only load model and data into memory when the client is selected for training or evaluation.

In this implementation, the shared global model takes the form of an MLP, configured with the identical architecture as the centralised model, detailed in Table 4.2. Moreover, the training process within

each client mirrors the centralised approach’s implementation, including the use of identical hyperparameters, as outlined in Section 4.2.1.

This study investigates the influence of varying the amount of local computation by conducting experiments using five different values of local epochs, E , ranging from 1 to 5. To account for the impact of different numbers of clients participating in the federation, we employ four distinct client configurations when using uniform data distribution, encompassing $K = 5, 8$, and 16 clients. In the case of stratified data distribution, a fixed total of $K = 5$ clients is used. It should be noted that, in each model, the fraction of clients taking part in each round, C , is set at one. A summary of the 20 distinct configurations of the FedAvg model implemented in this chapter is presented in Table 4.4.

Table 4.4: Different configurations of the FedAvg model for the experiments on IID settings. For each number of available clients, K , the number of local epochs, E , varies within a range of 1 to 5, resulting in 5 different models for the same number of clients. For all models, C is fixed at 1.

Data distribution	Model	Total # of clients (K)	Local epochs (E)
Uniform	1 – 5	5	1 – 5
	6 – 10	8	1 – 5
	11 – 15	16	1 – 5
Stratified	16 – 20	5	1 – 5

This comprehensive investigation encompasses various distinct models, achieved through the manipulation of the number of local epochs and participating clients, thereby offering in-depth insights into the dynamics of FL across diverse scenarios. The execution of the federated training procedure adheres to the sequential steps delineated in Section 3.3.1 and visually illustrated in Figure 3.2.

Assumptions and Limitations

Some limitations are notable in the FL approaches implemented throughout this dissertation. First, in the context of cross-silo FL settings, where the number of clients is reduced, we work under the assumption that all clients are consistently available and actively contribute to training at all rounds. Thus, we effectively fix $C = 1$.

Secondly, we assume a perfect scenario with no communication failures between the server and the clients, which may not reflect real-world communication conditions.

Lastly, it is important to note that the use of FL primarily occurs in a simulated environment. Although each client owns its local data, we store the entire dataset on a single machine, and each client accesses its respective partition using the Flower Virtual Client Engine. This means that clients do not have distinct IP addresses, which could make the training scenario more realistic.

Environment and Hardware Configuration

All experiments conducted in this study adhered to the following technical specifications. The software environment utilised for the experiments included Python 3.8.16, Torch 2.0.1, Flwr 1.4.0, Aequis

0.42.0, and Opacus 1.4.0. In terms of hardware configurations, the study was conducted on a system powered by an Intel(R) Core(TM) i9-7920X CPU running at 2.90GHz, coupled with an NVIDIA GeForce RTX 2080 Ti GPU equipped with 11 264MiB of memory.

4.3 Results

4.3.1 Performance Metrics

Bank account opening datasets, such as BAF IID, exhibit pronounced class imbalance, with fraudulent instances constituting only about 1% of the data. Consequently, selecting a suitable performance metric is crucial. As discussed in Section 3.1.1, accuracy proves inadequate in this context, as it can be increased by classifying all instances as legitimate. Therefore, we rely on the AUC score, which quantifies the model's ability to distinguish between classes, with values closer to one indicating better performance.

A complementary approach involves applying the Neyman-Pearson criteria, also detailed in Section 3.1.1. In the realm of bank account opening fraud detection, a true positive denotes the correct identification of a fraudulent application, preventing the fraudster's attempt to manipulate the FI, while a false positive signifies the misclassification of a legitimate application as fraudulent, potentially causing inconvenience to a regular customer.

Given the negative consequences of false positives, which may lead to customer dissatisfaction, bank account providers are averse to exceeding a predefined level for the FPR. Therefore, the optimisation goal in this task is to maximise the detection of fraudulent applications (recall) while ensuring a predefined 5% FPR limit is not surpassed.

To assess the overall performance of the federated models discussed throughout this thesis, it is important to note that the evaluations conducted are considered as 'global evaluations'. In this context, global evaluations mean that the test sets and predictions from all clients are aggregated and collectively evaluated. This approach provides a holistic view of the models' performance across the entire FL system.

4.3.2 Federated Averaging

This section presents the results of our study on the performance of the FedAvg models within the two distinct IID settings. The FL models trained on data uniformly distributed among clients are referred to as 'IID uniform experiments', while those trained on data with a stratified distribution are denoted 'IID stratified experiments'. Table 4.5 offers a detailed comparison of the key distinctions between these experiments, encompassing the relationships between $\mathbb{P}(Y | X)$ in the training and test sets, as well as $\mathbb{P}(Y | X, \text{client})$.

In both sets of experiments, we examine the impact of altering the number of local epochs, E , within each client's training process. Additionally, within the context of IID uniform experiments, we assess the consequences of increasing the number of participating clients in the federation, K .

Table 4.5: Key differences between both IID experiments.

Experiments		
IID Uniform	$\mathbb{P}_{\text{train}}(Y X) = \mathbb{P}_{\text{test}}(Y X)$	$\mathbb{P}(Y X, \text{client} = i) = \mathbb{P}(Y X, \text{client} = j), \forall i \neq j$
IID Stratified	$\mathbb{P}_{\text{train}}(Y X) = \mathbb{P}_{\text{test}}(Y X)$	$\mathbb{P}(Y X, \text{client} = i) \neq \mathbb{P}(Y X, \text{client} = j), \forall i \neq j$

IID Uniform Experiments

Figure 4.2 illustrates the evolution along the communication rounds of the AUC score and recall at 5% FPR, using the FedAvg algorithm in the context of the IID uniform setting, while varying K and E . The dashed line shows the performance of the centralised MLP model, trained on the entire BAF IID dataset, which serves as a baseline. Specifically, the centralised model achieved an AUC score of 0.8742 and a recall of 47.86% at 5% FPR.

The principal inference drawn from these results is that the FedAvg approach consistently demonstrates comparable or superior performance to the centralised model in terms of both AUC score and recall, across all values of E and K , when applied in fully IID settings. Nevertheless, it is imperative to emphasise that the differences displayed are not significant, since the graphical representations are characterised by a reduced scale.

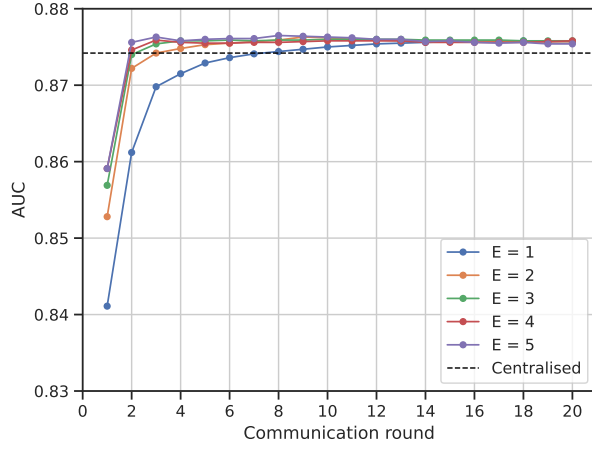
Furthermore, it is evident, as anticipated, that when maintaining a constant number of participating clients, performance convergence accelerates as the number of local training epochs increases. As outlined in Section 3.3.2, the quantity of local updates in FedAvg is given by TEn_k/B , where T indicates the number of communication rounds, n_k denotes the number of local examples on client k and B represents the batch size. Intuitively, holding B and n_k constant, a model trained with a larger number of local epochs requires fewer communication rounds to perform the same number of local updates, thereby achieving equivalent performance more efficiently.

Finally, as K increases, a greater number of communication rounds is necessary to achieve stability in model performance, owing to the decrease in n_k with increasing K . Particularly, for $K = 8$ and $K = 16$, it is clear that FedAvg requires more than the 20 rounds presented to attain the optimal AUC and recall values.

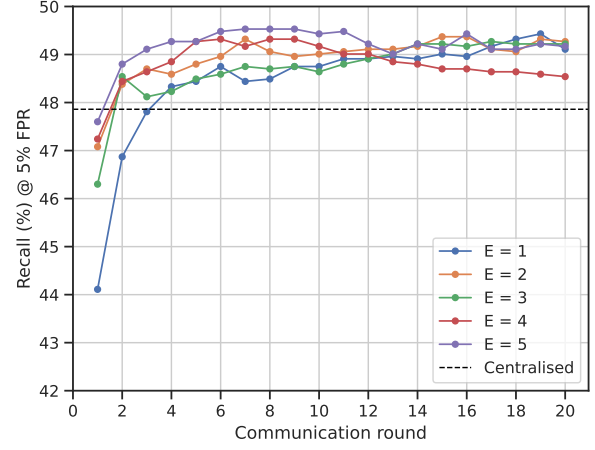
IID Stratified Experiments

To enhance our understanding of the FedAvg algorithm’s performance in situations deviating from uniform data distribution, we also assess the approach in stratified data distribution settings. The AUC score and recall for every communication round in these settings are shown in Figure 4.3, in comparison to the centralised model.

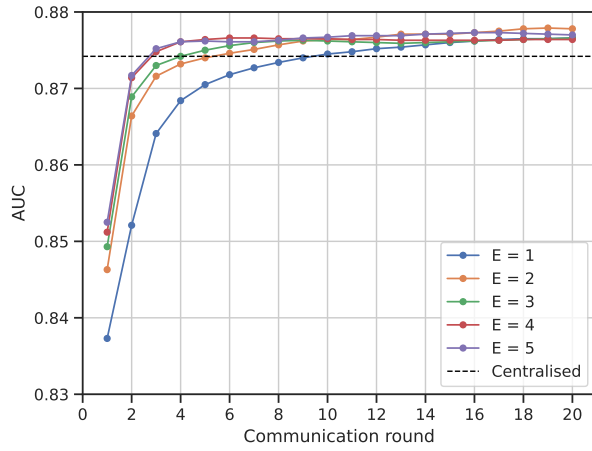
The initial observation to note is a decrease in performance, evident in both the AUC score and recall metrics, as compared to the results observed in the IID uniform experiments. Regarding the AUC score, it is clear that, across all values of E considered, the FedAvg algorithm fails to attain the AUC score achieved by the centralised model. In terms of recall, it is worth mentioning that only when E is set to 3 or 5 does the FedAvg manage to match the performance of the baseline model.



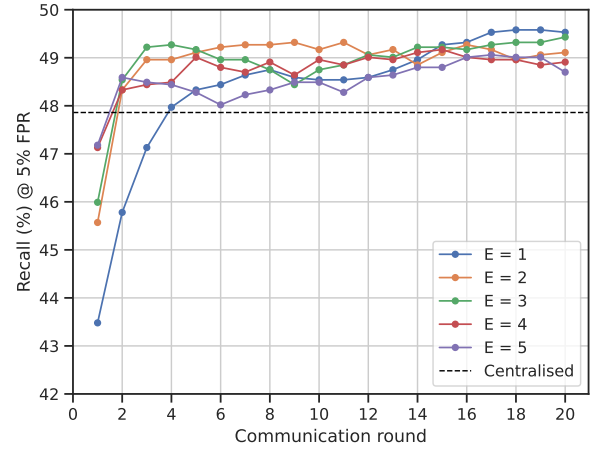
(a) AUC score for $K = 5$ clients



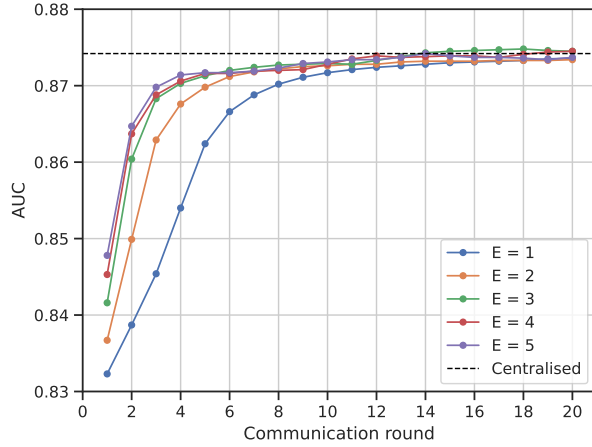
(b) Recall (%) at 5% FPR for $K = 5$ clients



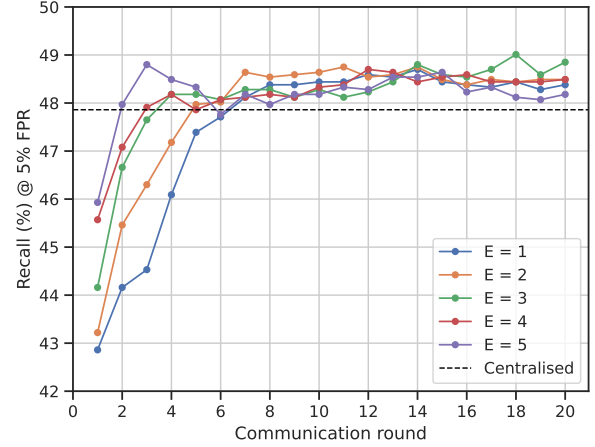
(c) AUC score for $K = 8$ clients



(d) Recall (%) at 5% FPR for $K = 8$ clients



(e) AUC score for $K = 16$ clients



(f) Recall (%) at 5% FPR for $K = 16$ clients

Figure 4.2: Performance evaluation of AUC score and recall at 5% FPR for the FedAvg algorithm on the IID uniform setting, conducted across varying numbers of available clients, K , and observed over multiple local epochs, E .

Furthermore, when considering smaller values of E , such as 1 and 2, FedAvg attains its peak recall performance within the initial three communication rounds. Specifically, for $E = 1$, there is a noticeable decline in the recall, dropping from 47.60% in the third round to 44.99% by the fourteenth round. The

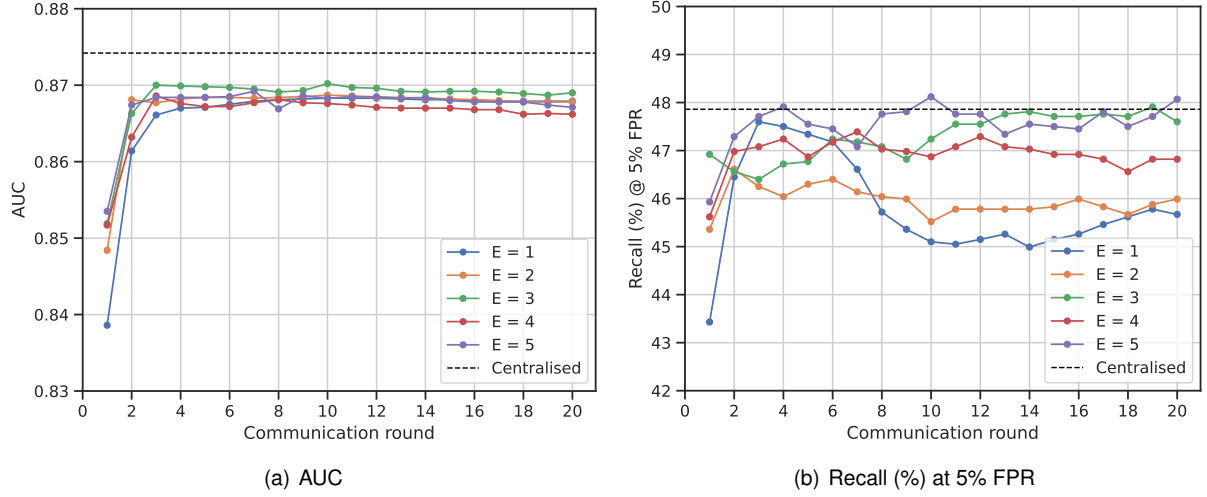


Figure 4.3: Performance evaluation of AUC score and recall at 5% FPR for the FedAvg algorithm on the IID stratified setting, for a fixed number of $K = 5$ clients, and observed over multiple local epochs, E .

highest recall values are achieved with $E = 3$ (47.91%) and with $E = 5$ (48.12%) in the nineteenth and tenth communication rounds, respectively, exceeding the 47.86% recall of the centralised model.

Figure 4.4 presents a formal comparison of the performance between the centralised model and FedAvg in both uniform and stratified IID settings. To ensure an equitable comparison between the IID settings, we maintained $E = 3$ for FedAvg in both cases and for the IID uniform, we set $K = 5$. This configuration ensures a fair evaluation between the two scenarios. The selection of $E = 3$ was based on its achievement of the highest AUC score in the context of the IID stratified experiments.

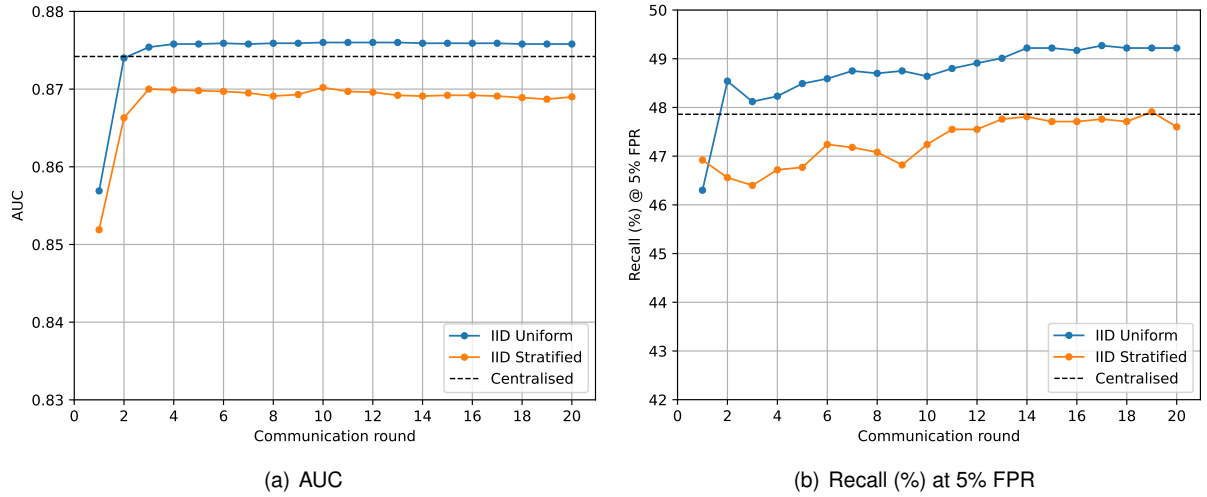


Figure 4.4: Comparison of the performance of the FedAvg algorithm on both uniform and stratified IID settings. In both scenarios, the results presented depict the AUC score and recall at a 5% FPR, with $K = 5$ clients and $E = 3$ local epochs.

Hence, the examination of this figure reveals that the FedAvg's overall performance experiences a decline when transitioning from IID uniform to stratified settings, no longer surpassing the performance of the centralised model. Nevertheless, it is imperative to note that this performance decline is not substantial, as indicated by the marginal differences between the highest AUC scores (0.006) and the

best recall values (1.36 percentage points).

In conclusion, the findings from these experiments establish that the FL approach, particularly through the application of the FedAvg algorithm, offers a compelling and viable alternative to the centralised approach for the detection of bank account opening fraud within IID settings. This method proves effective in scenarios characterised by both uniform and stratified data distributions, enabling the training of a shared model with decentralised data while preserving performance to a remarkable extent. While IID stratified settings offer a closer semblance to real-world conditions, actual fraud detection data often displays complete non-IID characteristics, both within and across clients. Consequently, the next chapter explores the domain of fully non-IID data distributions to assess the FL model performance within the most realistic scenario.

Chapter 5

Real-World Federated Learning

The preceding chapter established the foundations of our approach to combat financial fraud using FL within the context of IID data settings. We examined the potential of FL to enhance the performance with respect to a centralised ML model, particularly in the context of detecting bank account opening fraud. However, the real-world landscape of fraud detection seldom adheres to the assumptions of IID data.

In this chapter, we transition from the controlled environment of IID data to the complex and nuanced realm of non-IID data distributions, both within and across clients. Our objective remains consistent: to develop an FL approach for financial fraud detection. Nonetheless, we now face the additional challenge of addressing the disparities in data distribution across a diverse range of clients. In light of these challenges, this chapter is dedicated to providing insights into the adaptability and efficacy of FL within the complex framework of non-IID data and explores the practical implications and strategies necessary to confront these data complexities.

5.1 BAF Base Dataset

To achieve our goal, we choose the Base Variant of the BAF suite, simply called the BAF Base dataset. This choice aligns with the goal as it faithfully replicates the data distribution drift over time, arising from both natural occurrences such as changes in client behaviour and the adaptive strategies employed by fraudsters to trick the model. Moreover, it accurately reflects the primary biases inherent in bank account opening data.

The fraud prevalence throughout the whole BAF Base is 1.10%; it is a highly unbalanced dataset. In addition, this prevalence suffers fluctuations over time and reaches its peak in the last months. To maintain this non-IID nature of the data, we partition it into training, validation, and test sets based on the monthly timeframes. Specifically, the first six months constitute the training set, the seventh month serves as the validation set, and the final month is designated as the test set. As a result, each split depicts distinct fraud rates, with the test set registering a higher prevalence compared to both the training and validation sets. Table 5.1 summarises the characteristics of the BAF Base dataset. Once

more, information regarding the fairness of this dataset can be found in Chapter 6, with the current stage primarily dedicated to the analysis of model performance.

Table 5.1: Summary of the BAF Base dataset.

Dataset split	Number of records	Fraud prevalence	Duration	Number of features
Training	794 989	1.03%	6 months	29
Validation	108 168	1.34%	1 month	29
Test	96 843	1.48%	1 month	29
Full BAF Base dataset	1 000 000	1.10%	8 months	29

5.2 Methodology

5.2.1 Centralised Model and FedAvg

In agreement with the methodology outlined in Chapter 4, the data pre-processing, as well as the implementation and training of the centralised MLP in the present settings, follow the same established approach. However, training the model on the BAF Base dataset and optimising hyperparameters resulted in the best MLP achieving a 50.83% recall on the BAF Base validation set. The chosen parameters defining the MLP architecture for non-IID settings are presented in Table 5.2. The remaining configurations consist of a batch size of 256 samples, with a fraud prevalence of 5.77%. Additionally, the Adam optimiser is employed once again, albeit with a reduced learning rate, η , of 0.003, and the BCE (Equation 3.6) serves as the designated loss function for this purpose as well.

Table 5.2: Multi-layer perceptron architecture for the non-IID settings.

Layer	Neurons	Activation function	Dropout rate
Input	29	—	—
$4 \times$ Hidden	4	Tanh (3.4)	0.0083
Output	1	Sigmoid (3.5)	—

To preserve the non-IID setting of the experiments, in addition to using the BAF Base dataset, we use a **stratified data distribution** when allocating data to clients, as described in the second approach outlined in Chapter 4. Utilising the ‘device_os’ feature from the BAF Base dataset to perform the split, we derive a set of five distinct client entities for the subsequent experiments. This approach guarantees a complete non-IID environment, wherein the prevalence of fraud varies across training, validation, and test sets within each client and also differs among clients. Table 5.3 illustrates the sample distribution and fraud prevalence across clients following a stratified distribution of the BAF Base dataset, confirming the non-IID setting of these experiments. Z-score normalisation and target encoding are also employed in each dataset as data pre-processing techniques.

Table 5.3: Training set size and fraud prevalence across clients for stratified distribution of the BAF Base dataset.

Stratified Data Distribution			
Client and set	Category 'device_os'	Size	Fraud prevalence
Client 1 – Training	Other	265 427	0.53%
Client 2 – Training	Linux	267 680	0.46%
Client 3 – Training	Windows	212 925	2.30%
Client 4 – Training	Macintosh	43 011	1.26%
Client 5 – Training	X11	5 946	1.04%
BAF IID – Training	—	794 989	1.10%
BAF IID – Test	—	96 843	1.13%

In these experimental settings, the FedAvg algorithm was also used to assess its performance when confronted with fully non-IID data. The implementation remains consistent with the methodology described in the preceding chapter, but in these non-IID settings, the shared global MLP has the architecture detailed in Table 5.2, and the client-side training follows the hyperparameters specifically chosen for non-IID scenarios. Table 5.4 displays the different configurations of the FedAvg model implemented in these settings.

Table 5.4: Different configurations of the FedAvg model for the experiments on non-IID settings. The BAF Base dataset is split across clients according to a stratified data distribution.

Model	Total # of clients (K)	Local epochs (E)
Model 1	5	1
Model 2	5	2
Model 3	5	3
Model 4	5	4
Model 5	5	5

5.2.2 FedProx and FedOpt

At this point in our research, confronted with the challenge of statistical heterogeneity, we opt to extend our exploration beyond FedAvg. In this phase, we introduce the implementation details of two alternative algorithms, namely, FedOpt and FedProx. These algorithms are explicitly designed to tackle the issue of non-IID data, complementing our prior implementation of FedAvg.

FedProx

FedProx, as elaborated in Section 3.3.3 and described in Algorithm 2, serves as an extension of FedAvg that alleviates the adverse effects of heterogeneity in both system and data. At this stage of

the project, we aim to address the problem of non-IID data rather than system heterogeneity, as all experiments are carried out on the same machine. Hence, rather than adjusting the amount of local computation within each client for each round by manipulating the parameter γ , we ensure, for each model, a consistent number of local epochs, E , for all clients in every round. Therefore, the sole deviation from the FedAvg implementation is the inclusion of the proximal term within the local loss function, as given in Equation 3.10, that acts as a regularisation term by constraining the local updates to be closer to the initial share global model.

In summary, the key parameters of FedProx that affect model performance are the amount of local work, E , and the proximal term scaled by the penalty constant, $\mu \geq 0$. Thus, in these experiments, we implement FedProx under various μ values for two different numbers of epochs, 1 and 5, to comprehend the impact of these parameters. Table 5.5 shows the different implemented configurations of the FedProx algorithm.

Table 5.5: Different configurations of the FedProx model for the experiments on non-IID settings. For each value of μ the number of local epochs, E , takes the values $\{1, 5\}$, resulting in 2 different models for the same μ .

Model	Total # of clients (K)	Penalty constant (μ)	Local epochs (E)
Models 1-2	5	10^{-4}	$\{1, 5\}$
Models 3-4	5	10^{-3}	$\{1, 5\}$
Models 5-6	5	10^{-2}	$\{1, 5\}$
Models 7-8	5	10^{-1}	$\{1, 5\}$
Models 9-10	5	1.0	$\{1, 5\}$

FedOpt

The second alternative implemented method is FedOpt, defined in Section 3.3.3 and detailed in Algorithm 3. This approach incorporates adaptivity into FL, enabling adaptive optimisations for both the client and server.

In this study, we evaluate the performance of three algorithms that constitute FedOpt, specifically FedAdam, FedAdagrad, and FedYogi, in comparison to previously adopted methods. Each strategy employs the same client optimisation approach as FedAvg, using the Adam optimiser with a learning rate of $\eta_l = 0.003$. The server-side implementation of FedAdam and FedYogi employs fixed momentum parameters $\beta_1 = 0.9$ and $\beta_2 = 0.99$. We further explore various configurations by altering the adaptivity τ and the server learning rate η across all three algorithms. We maintain a consistent number of $E = 1$ local epochs for all experimental settings. Table 5.6 summarises the different configurations of the FedOpt strategy implemented.

Table 5.6: Different configurations of the FedOpt model for the experiments on non-IID settings. As per the earlier experiments, there are a total of $K = 5$ available clients after stratified data distribution, and all participate in all communication rounds ($C = 1$). For the different models, the number of local epochs, E , remains fixed at 1. A grid search on the τ and η parameters yields 20 models for each strategy.

Model	Strategy	Adaptivity (τ)	Server learning rate ($\log_{10} \eta$)
Models 1-20	FedAdam	$\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$	$\{-3, -2.5, -2, -1.5, -1\}$
Models 21-40	FedAdagrad	$\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$	$\{-3, -2.5, -2, -1.5, -1\}$
Models 41-60	FedYogi	$\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$	$\{-3, -2.5, -2, -1.5, -1\}$

5.3 Results

In this section, we conduct an extensive performance evaluation of the various FL algorithms applied in a real-world context characterised by non-IID data distribution, both within individual clients and across them. The presented findings highlighted the effectiveness and robustness of these FL algorithms when facing such challenging data scenarios. We consider the same performance metrics used in the previous chapter, detailed in Section 4.3.1.

The fundamental distinction between non-IID experiments and IID counterparts is based on the posterior probability $\mathbb{P}(Y | X)$. In the present experimental configurations, there exists a discrepancy where $\mathbb{P}_{\text{train}}(Y | X) \neq \mathbb{P}_{\text{test}}(Y | X)$, a distinction that was not present in the previous experimental setups. Table 5.7 provides a summary of the key differences between the non-IID and the two IID experiments.

Table 5.7: Key differences between the IID and the non-IID experiments.

Experiments		
IID Uniform	$\mathbb{P}_{\text{train}}(Y X) = \mathbb{P}_{\text{test}}(Y X)$	$\mathbb{P}(Y X, \text{client} = i) = \mathbb{P}(Y X, \text{client} = j), \forall i \neq j$
IID Stratified	$\mathbb{P}_{\text{train}}(Y X) = \mathbb{P}_{\text{test}}(Y X)$	$\mathbb{P}(Y X, \text{client} = i) \neq \mathbb{P}(Y X, \text{client} = j), \forall i \neq j$
Non-IID	$\mathbb{P}_{\text{train}}(Y X) \neq \mathbb{P}_{\text{test}}(Y X)$	$\mathbb{P}(Y X, \text{client} = i) \neq \mathbb{P}(Y X, \text{client} = j), \forall i \neq j$

Figure 5.1 demonstrates the evolution along the communication rounds of both the AUC score and recall at 5% FPR using the FedAvg algorithm, recorded across multiple local epochs, E . This figure additionally showcases the performance of the centralised model, trained on the entire BAF Base dataset, which achieved an AUC score of 0.8816 and a recall of 53.15%.

While the FedAvg algorithm outperforms the centralised model in terms of AUC score during the initial rounds, specifically for $E = 1$ between the fourth and eighth rounds, for $E = 2$ in rounds 3 and 4, and for $E = 4$ in the second round, it becomes evident that divergence begins to occur in subsequent rounds. This trend is observed in both the AUC score and recall and corroborates the findings drawn by Li et al. [52], which demonstrate that FedAvg may experience divergence in the presence of heterogeneous data. Nonetheless, FedAvg achieves the highest recall value of 51.26% in the fourth round and the highest AUC score of 0.8829 in the fifth round, both for $E = 1$.

Furthermore, Figure 5.2 provides a visual representation of the performance of the FedProx algorithm, under two different values of E and varying penalty constants μ . The initial examination of this

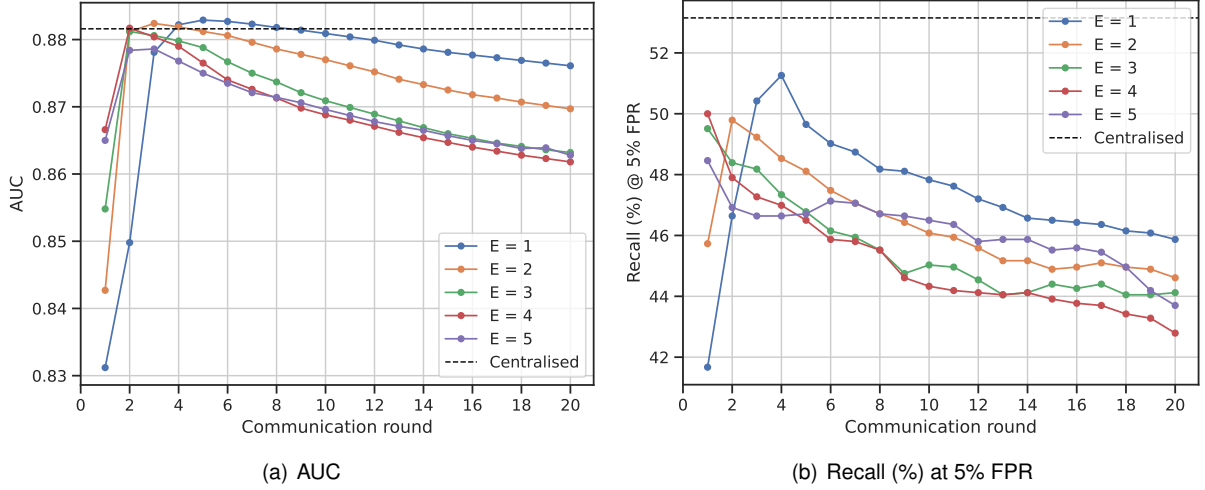


Figure 5.1: Performance evaluation of AUC score and recall at 5% FPR for the FedAvg algorithm non-IID settings, for a fixed number of $K = 5$ clients, and observed over multiple local epochs, E .

figure reveals that the introduction of a positive penalty constant, $\mu > 0$, significantly enhances the convergence of the algorithm in situations characterised by data heterogeneity. Nonetheless, it becomes evident that with a larger value of μ , such as $\mu = 1.0$, the model demonstrates an underfitting pattern. This is due to the imposition of penalties on local updates, constraining them to be closer to the initial shared global model. In contrast, when employing smaller values of μ (for instance, $\mu = 10^{-4}$), the behaviour of FedProx closely resembles that of FedAvg. This outcome aligns with expectations, as FedAvg can be considered a specific case of FedProx when $\mu = 0$.

Moreover, when comparing the performance of FedProx with $E = 1$ and $E = 5$, it is noteworthy that, although the differences between the two settings may not be substantial, the FedProx's performance indeed shows improvement with the increase of E , as anticipated. Consequently, this algorithm achieves a maximum recall value equivalent to that of FedAvg and slightly exceeds FedAvg's maximum AUC score by reaching a value of 0.8852, all while maintaining stability. Overall, the results on the FedProx performance suggest that the adapted subproblem presented in this strategy could be advantageous to practical federated settings with diverse statistical heterogeneity.

With regard to the FedOpt family of algorithms, the performance of the FedAdam strategy along the communication rounds can be found in Figures 5.3 and 5.4. These figures present the results for multiple values of the parameters η and τ , respectively. We have opted to display the results for FedAdam exclusively in this context, as both FedAdagrad and FedYogi demonstrated comparable performance patterns, making it unnecessary to include them separately.

Concerning the impact of the server learning rate, η , smaller values, such as $\eta = 10^{-3}$, mean that the server is making only minor adjustments based on the received updates from the participating clients during each communication round. This consequently slows down the convergence process, as illustrated in Figure 5.3. On the other hand, larger values of η , exemplified by $\eta = 0.1$, may induce an oscillatory behaviour within the initial 50 rounds. The optimal choice of the learning rate η among those used for the FedAdam and FedYogi algorithms is $\eta = 10^{-1.5}$, while for the FedAdagrad algorithm, the

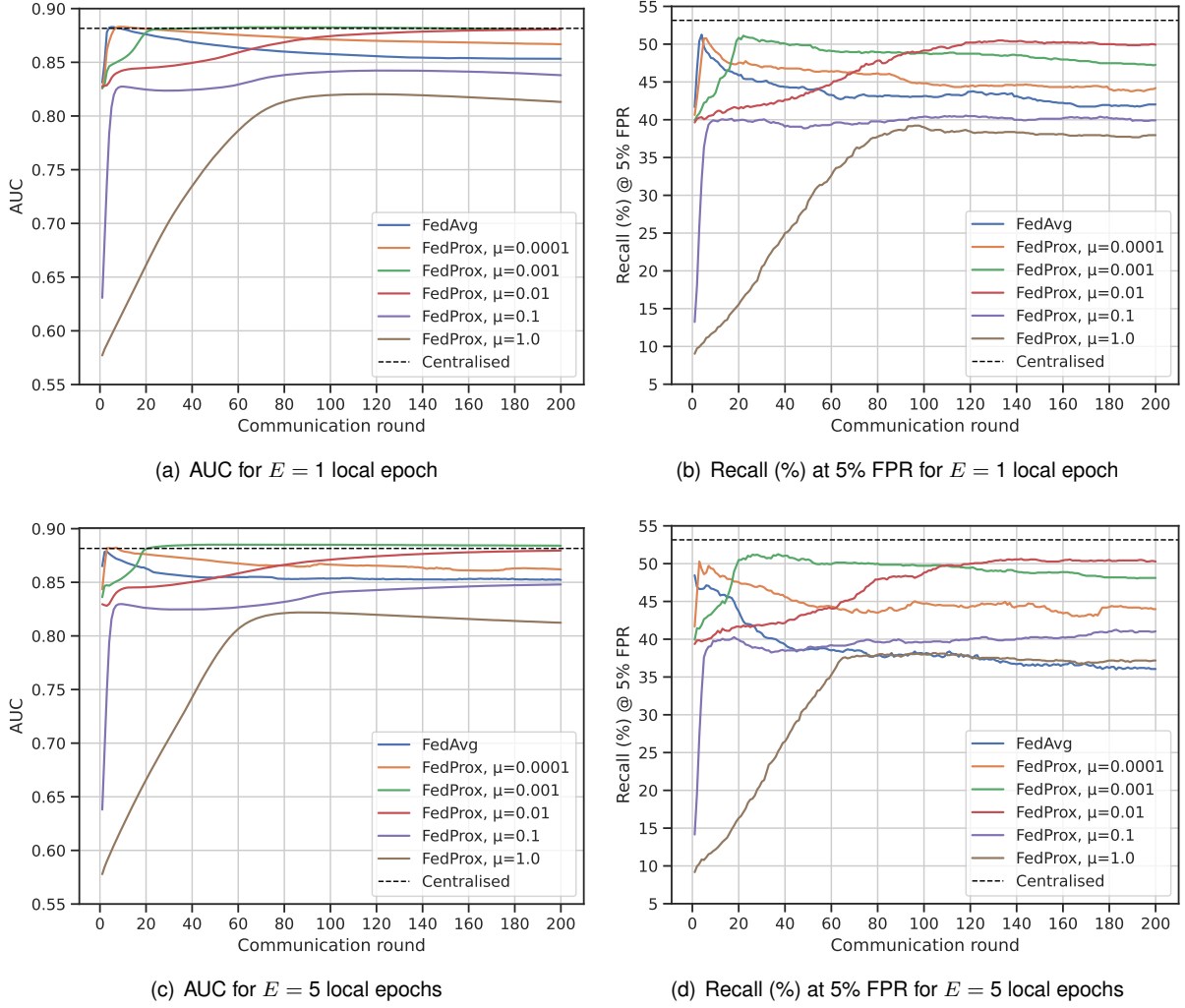
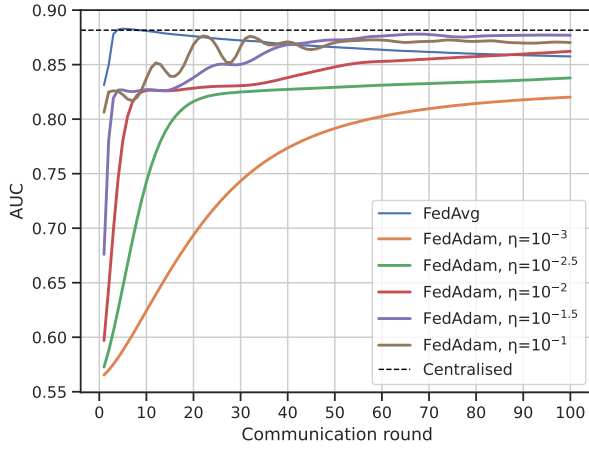


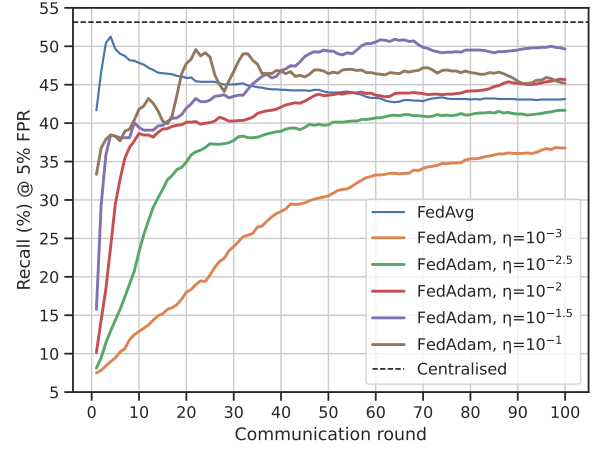
Figure 5.2: Performance evaluation of AUC score and recall at 5% FPR for the FedProx algorithm on non-IID settings, for two distinct values of local epochs, E , observed over multiple penalty constant values, μ . The number of available clients, K , is set to 5.

most effective learning rate is $\eta = 10^{-1}$. Moreover, it is clear from Figure 5.4 that the model's performance remains unaffected by the level of adaptivity, denoted by the parameter τ . The overall results for this family of algorithms demonstrate that incorporating adaptivity into the FL framework effectively addresses the convergence challenges faced by state-of-the-art FedAvg when working with heterogeneous data.

Figure 5.5 provides a comprehensive summary of the performance evaluation of the five algorithms employed in this study, for the best μ , η , and τ parameters. To ensure equitable and easier comparisons, we maintain a consistent number of local epochs, $E = 1$, as this configuration leads to the optimal performance for the FedAvg method. Lastly, we conclude that when transitioning from an ideal IID setting to a more realistic non-IID scenario, the FedAvg algorithm exhibits a tendency to diverge. In such cases, alternatives like FedProx and FedOpt prove to be viable options, as they ensure performance convergence while remaining competitive with the centralised model.

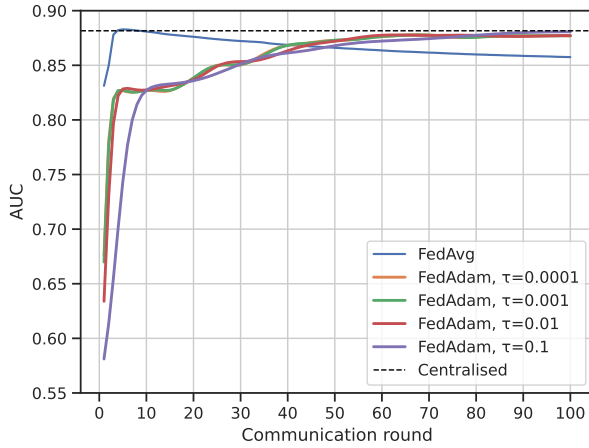


(a) AUC

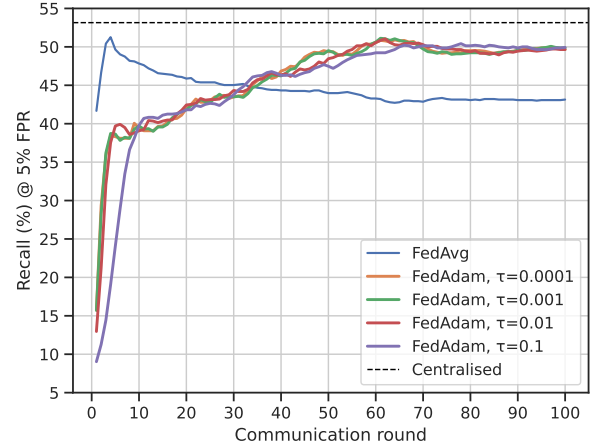


(b) Recall (%) at 5% FPR

Figure 5.3: Performance evaluation of AUC score and recall at 5% FPR for the FedAdam (FedOpt) algorithm non-IID settings, for multiple server learning rates, η , and adaptivity $\tau = 10^{-4}$. The number of available clients, K , is set to 5 and the number of local epochs, E , is fixed at 1.



(a) AUC



(b) Recall (%) at 5% FPR

Figure 5.4: Performance evaluation of AUC score and recall at 5% FPR for the FedAdam (FedOpt) algorithm on non-IID settings, for multiple τ and server learning rate $\log_{10} \eta = -1.5$. The number of available clients, K , is set to 5 and the number of local epochs, E , is fixed at 1.

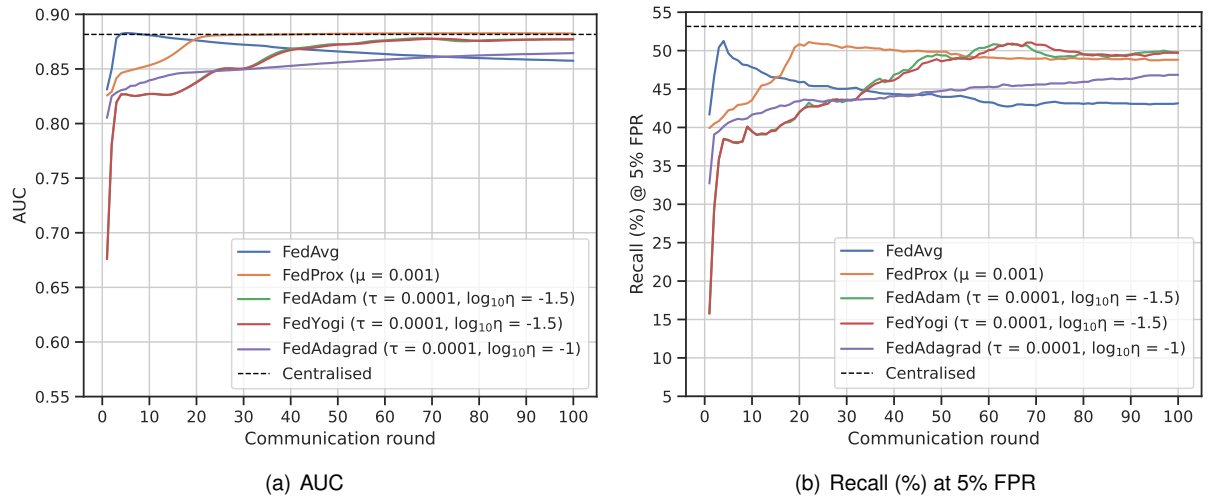


Figure 5.5: Comparison of the performance of FedAvg, FedProx, and FedOpt algorithms on non-IID settings. For all strategies, the number of available clients, K , is set to 5 and the number of local epochs, E , is fixed at 1.

Chapter 6

Fairness and Privacy in Federated Learning

In the preceding chapters, our focus was primarily on the performance evaluation of the implemented models. Nevertheless, FL raises additional concerns requiring attention, specifically regarding fairness and privacy. Therefore, this chapter delves into the domain of responsible AI, addressing the associated fairness and ethical considerations. A meticulous analysis is conducted to evaluate the fairness of the FL algorithms used, taking into account both IID and non-IID settings. Moreover, the privacy challenge within FL is addressed by incorporating DP into the training process to safeguard sensitive data. The resulting impacts on the models' performance and fairness are also assessed.

6.1 Fairness Evaluation

To delve into fairness, it is necessary to consider how its impact should be accounted for. When determining the amount in a line of credit, one factor to consider is the applicant's age, with older individuals often associated with higher values. Hence, there tends to be a higher proportion of fraudulent applications in this group, as it offers fraudsters the opportunity for larger profits. Conversely, the amount of requests is greater among younger individuals, contrasting with the fraud prevalence. Therefore, a relevant fairness concern arises in the task of detecting fraud in bank account applications: ML models tend to raise more false fraud alerts for older customers (≥ 50 years), thus reducing their access to a bank account. This corresponds to an FPR disparity, as introduced in Section 3.1.2.

Here, we study fairness concerning an individual's age within the FL framework. Thus, the fairness of the implemented FL algorithms is thoroughly assessed across both IID and non-IID data settings.

To analyse the inherent bias conditions within the employed datasets, Tables 6.1 and 6.2 provide a fairness overview for the BAF IID and the BAF Base datasets, respectively. The BAF IID dataset was intentionally designed to represent an unbiased version of the BAF Suite. Therefore, as shown in Table 6.1, both groups showcase nearly identical sizes and exhibit relatively similar fraud prevalence. Moreover, the size and the fraud prevalence of the groups remain constant across the training, validation,

Table 6.1: Fairness overview of the BAF IID dataset. Age is the designated sensitive attribute, delineating two distinct sensitive groups: the young (<50 years) referred to as Group 0, and the elderly (≥ 50 years), named Group 1.

Dataset split	Group size		Fraud prevalence	
	Group 0	Group 1	Group 0	Group 1
Training	49.4%	50.6%	1.13%	1.07%
Validation	49.6%	50.4%	1.11%	1.07%
Test	49.4%	50.6%	1.15%	1.11%
Full BAF IID dataset	49.4%	50.6%	1.13%	1.08%

Table 6.2: Fairness overview of the BAF Base dataset. Age is the designated sensitive attribute, delineating two distinct sensitive groups: the young (<50 years) referred to as Group 0, and the elderly (≥ 50 years), named Group 1.

Dataset split	Group size		Fraud prevalence	
	Group 0	Group 1	Group 0	Group 1
Training	81.2%	18.8%	0.75%	2.21%
Validation	80.5%	19.5%	1.01%	2.71%
Test	86.9%	13.1%	1.20%	3.30%
Full BAF Base dataset	81.7%	18.30%	0.83%	2.34%

and test sets. Concerning the BAF Base dataset, two types of bias are evident: disparities in both fraud prevalences and group sizes, as illustrated in Table 6.2. Specifically, the group representing older individuals is approximately one-fourth the size of the other group but demonstrates a roughly three times higher fraud prevalence.

Furthermore, Table 6.3 depicts the disparities in group sizes and fraud prevalences across clients in non-IID settings, after stratified data distribution of the BAF Base dataset. The disparities are consistent with the overall disparities in the entire BAF Base. Particularly, Client 3 presents a slightly lower disparity in both metrics when compared to the others.

Table 6.3: Fairness overview across clients on non-IID settings.

Client	Category 'device_os'	Group size		Fraud prevalence	
		Group 0	Group 1	Group 0	Group 1
Client 1	Other	87.45%	12.55%	0.47%	1.31% ($\times 2.79$)
Client 2	Linux	80.96%	19.04%	0.40%	0.99% ($\times 2.48$)
Client 3	Windows	74.56%	25.44%	1.87%	4.22% ($\times 2.26$)
Client 4	Macintosh	85.39%	14.61%	1.14%	2.89% ($\times 2.53$)
Client 5	X11	75.53%	24.47%	0.79%	2.15% ($\times 2.72$)

6.1.1 Fairness metrics

In Section 3.1.2, multiple observational fairness metrics were introduced and discussed. In this work, the aim is to ensure that the implemented FDSs do not exhibit undue discrimination against protected sub-groups within the population. To achieve this goal, metrics like equality of opportunity and predictive equality are usually adopted.

Ensuring equality of opportunity entails that the FIs accurately identify fraudsters without bias toward either group. Nonetheless, this is not the main ethical concern. Conversely, banking service providers are concerned with the prospect of incorrectly denying access to a bank account to individuals from any sensitive group, recognising the potential consequences this denial could have on their lives. Accordingly, the primary fairness metric in this particular situation, and the one to which we will direct our attention, is predictive equality, measured through the FPR parity (Equation 3.3). According to Definition 3.3, predictive equality corresponds to FPR parity equal to one.

As mentioned in Section 4.3.1, where we discussed the performance metrics used, we adopt a similar approach in our fairness analysis. In this context, we also consider a ‘global fairness evaluation’, which involves aggregating predictions and test sets, and collectively evaluating fairness across all clients. Hence, a comprehensive perspective on the fairness of the models within the entire FL system is detailed.

6.1.2 Results

IID Experiments

Within IID settings, where an unbiased version of the BAF Suite is used, the centralised model demonstrated a high FPR parity of 0.8990, as anticipated. This result means that the centralised model successfully maintains predictive equality, ensuring that access to bank account opening remains fair across the two sensitive groups.

Tables 6.4, 6.5, and 6.6 present the FPR parity results of the FedAvg methods employed in IID uniform settings, for $K = 5, 8$, and 16 clients, respectively. Moreover, Table 6.7 displays the FPR parity values for the FedAvg algorithm implemented in IID stratified settings. In these tables, values denoted in bold indicate FPR parity values lower than that of the centralised model. From the analysis of these results, we conclude that the use of the FedAvg algorithm within an unbiased environment maintains and even improves the FPR parity of the resulting ML models when compared to the centralised MLP, across the majority of E examined. Furthermore, when examining the fairness results of the FedAvg method within IID stratified settings, an interesting pattern is evident: across all values of E , the most favourable FPR parity results are consistently observed during the initial five rounds of communication. Specifically, it is noteworthy that, for both $E = 1$ and $E = 2$, the optimal FPR parity value is achieved in the very first round. This coincides with the initial training stage, characterised by its lowest performance, as depicted in Figure 4.3, thereby highlighting the trade-off between fairness and performance.

Table 6.4: FPR parity for the FedAvg algorithm on the IID uniform settings, with $K = 5$ clients. The centralised model, trained on BAF IID, achieved an FPR parity of 0.8990 on the test set.

Epochs	FPR Parity				
	Average	Standard deviation	Last round (20 th)	Best value	
				Value	Round
$E = 1$	0.9080	0.0127	0.9183	0.9185	16
$E = 2$	0.9589	0.0130	0.9592	0.9932	2
$E = 3$	0.8826	0.0184	0.8756	0.9441	1
$E = 4$	0.9154	0.0135	0.9281	0.9318	14
$E = 5$	0.9363	0.0109	0.9447	0.9537	18

Table 6.5: FPR parity for the FedAvg algorithm on the IID uniform settings, with $K = 8$ clients. The centralised model, trained on BAF IID, achieved an FPR parity of 0.8990 on the test set.

Epochs	FPR Parity				
	Average	Standard deviation	Last round (20 th)	Best value	
				Value	Round
$E = 1$	0.8939	0.0351	0.9346	0.9379	19
$E = 2$	0.9437	0.0113	0.9500	0.9571	10
$E = 3$	0.9703	0.0114	0.9570	0.9987	3
$E = 4$	0.9870	0.0183	0.9968	0.9998	14
$E = 5$	0.9362	0.0159	0.9156	0.9796	2

Table 6.6: FPR parity for the FedAvg algorithm on the IID uniform settings, with $K = 16$ clients. The centralised model, trained on BAF IID, achieved an FPR parity of 0.8990 on the test set.

Epochs	FPR Parity				
	Average	Standard deviation	Last round (20 th)	Best value	
				Value	Round
$E = 1$	0.9684	0.0117	0.9535	0.9973	2
$E = 2$	0.9000	0.0100	0.9225	0.9225	20
$E = 3$	0.8906	0.0129	0.9085	0.9085	20
$E = 4$	0.9221	0.0245	0.9429	0.9789	2
$E = 5$	0.9187	0.0338	0.9703	0.9716	19

Non-IID Experiments

In relation to experiments conducted within non-IID settings, specifically using the BAF Base dataset, which has disparities in both fraud prevalence and group sizes, the results confirm a clear decrease in the FPR parity of the centralised model, from 0.8990 to 0.3063. Specifically, the group of younger individuals shows an FPR of 3.87%, whereas the older group has a substantially higher FPR of 12.65%. In contrast to the results observed in the experiments conducted on an unbiased dataset, the FPR within

Table 6.7: FPR parity for the FedAvg algorithm on the IID stratified settings. The centralised model, trained on BAF IID, achieved an FPR parity of 0.8990 on the test set.

Epochs	FPR Parity				
	Average	Standard deviation	Last round (20 th)	Best value	
				Value	Round
$E = 1$	0.9128	0.0238	0.9549	0.9719	1
$E = 2$	0.9678	0.0086	0.9725	0.9852	1
$E = 3$	0.9496	0.0329	0.9431	0.9965	5
$E = 4$	0.8616	0.0198	0.8643	0.9034	4
$E = 5$	0.9306	0.0202	0.9141	0.9730	3

the former group decreased by 0.82 percentage points, while the FPR within the latter increased by 7.44 percentage points. It is clear that the centralised model is exacerbating the bias present in the data, disproportionately affecting the older group.

To assess the fairness of the FL methods in the non-IID experiments, Table 6.8 presents the FPR parity, including average, optimal, and final round values, for the FedAvg, FedProx, and FedOpt (FedAdam, FedYogi, and FedAdagrad) algorithms. We conclude that the fairness of the FL approach is also affected by biased datasets, yet it does not get worse compared to the centralised model.

Table 6.8: FPR parity for FedAvg, FedProx (with $\mu = 0.001$), and FedOpt (with $\tau = 0.0001$) algorithms on non-IID settings. For FedAdam and FedYogi, the parameter $\log_{10} \eta = -1.5$ is used and for FedAdagrad $\log_{10} \eta = -1.0$. For all strategies, the number of local epochs, E , is fixed at 1. The centralised model, trained on BAF Base, achieved an FPR parity of 0.3063 on the test set.

Strategy	FPR Parity				
	Average	Standard deviation	Last round (100 th)	Best value	
				Value	Round
FedAvg	0.3279	0.0135	0.3362	0.3376	42
FedProx	0.3018	0.0105	0.3150	0.3153	98
FedAdam	0.2983	0.0786	0.3026	0.9852	2
FedYogi	0.2971	0.0790	0.2962	0.9897	2
FedAdagrad	0.3063	0.0296	0.2961	0.5946	1

Moreover, Figure 6.1 illustrates the evolution along the communication rounds of both recall and FPR parity for the five algorithms employed. Upon analysing this figure and Table 6.8, it is clear that, unlike its overall performance, the FPR parity of the FedAvg algorithm converges to a value that is marginally superior to those of the centralised model and other FL techniques. We further observe an oscillating behaviour of the FPR parity of the FedAdam and FedYogi strategies.

In addition, for the FedProx algorithm (with $\mu = 0.001$), we conduct an in-depth examination of the fairness exhibited by individual clients. Clearly, Client 5, having the smallest dataset, demonstrates the poorest average FPR parity among all clients, with a value of 0.1550. Conversely, Client 3, while still exhibiting disparities in its dataset, does not show disparities as severe as the others (as shown in Table

6.3) and reveals the highest average FPR parity, reaching a value of 0.3720.

Finally a comparative analysis of Figures 6.1(a) and 6.1(b) reveals a noteworthy correlation: the peak FPR parity values, attained by the FedAdam and FedYogi methodologies in the second round, align with the worst recall values among all FL algorithms, thus accentuating the inherent trade-off between fairness and performance in the context of FL.

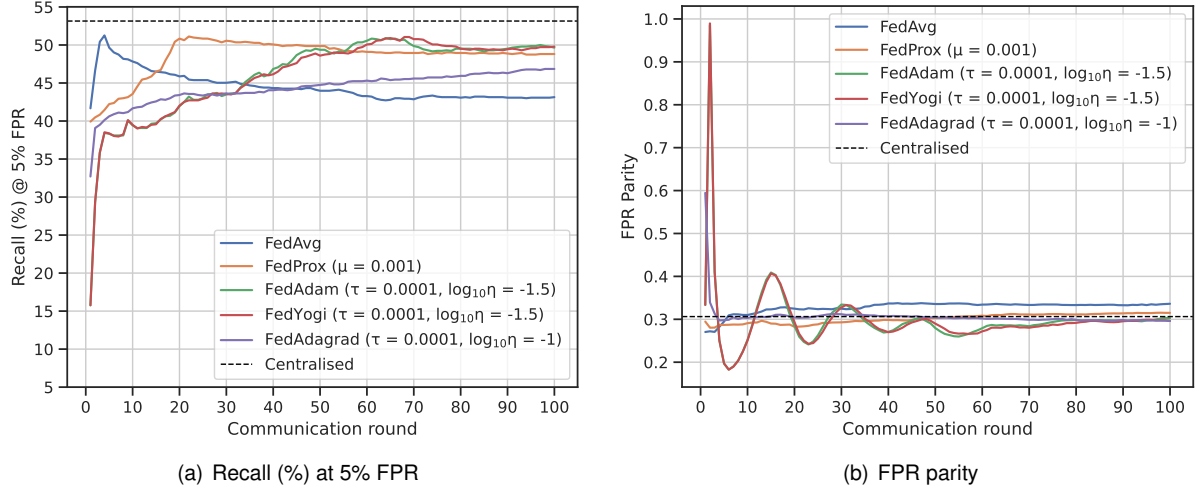


Figure 6.1: Comparison of the performance and fairness of FedAvg, FedProx, and FedOpt (FedAdam, FedYogi, and FedAdagrad) algorithms on non-IID settings. For all strategies, the number of available clients, K , is set to 5 and the number of local epochs, E , is fixed at 1.

6.2 Differentially Private Federated Learning

FL is intended to train consortium models with diverse data sources, ensuring no data exchange. Nevertheless, the collaborative training process may inadvertently expose sensitive information through shared model parameters. Malicious third parties may launch privacy attacks against FL models, to potentially infer raw data by exploiting model access.

This section is dedicated to an in-depth examination of the integration of sample-level DP into the real-world FL framework to protect the confidentiality of data instances. Specifically, the main emphasis is on the implementation of the DP-SGD algorithm, outlined in Section 3.4.1, providing insights into the real-world implications and trade-offs related to the adoption of this privacy-preserving technique.

6.2.1 Implementation

We intend to integrate the DP-SGD approach, defined in Algorithm 4, into the FL models developed on non-IID settings. Hence, we consider the FedProx algorithm with $E = 1$ and $\mu = 0.001$, detailed in Section 5.2.2.

To incorporate this privacy mechanism into the training workflow of the clients throughout the federated training process, the Opacus PyTorch Library [97] is used. Opacus takes the PyTorch model, optimiser, and data loader objects, along with the privacy parameters, and generates differentially pri-

vate equivalents. To align with the requirements of the DP-SGD algorithm, the model is wrapped to enable per-sample gradient computation. Additionally, the optimiser is responsible for clipping gradients and adding noise, and the data loader is transformed into a Poisson sampling-based loader. Within the Poisson sampling method, also known as uniform batch sampling, each data point is independently included in the batch with a probability equal to the sampling rate. To keep track of the privacy budget, Opacus provides an accountant method based on Rényi Differential Privacy [102].

Our primary objective is to seamlessly integrate the DP-SGD algorithm into the FL framework and subsequently evaluate its influence on the performance, fairness, and privacy level of the resultant models. To achieve this, we deliberately vary the values of three key privacy hyperparameters: the target δ , the noise scale σ , and the gradient clipping threshold Γ .

We adopt a systematic grid search approach to comprehensively cover the entire spectrum of feasible parameter combinations. Specifically, we select four distinct values for each of these hyperparameters. This results in a total of 64 unique combinations of the privacy mechanism’s hyperparameters. Table A.2 in Appendix A displays the privacy parameters search space.

Subsequently, the FL approach, using FedProx, is subjected to training on the BAF Base dataset. Each of the 64 distinct hyperparameter combinations is used, allowing for an extensive exploration of the impact of DP-SGD integration. This comprehensive evaluation process is undertaken to gain insights into the implications of varying privacy parameter values on the model’s performance, fairness, and privacy characteristics.

Furthermore, for a comprehensive visual representation of the entire workflow encompassing the implementations discussed in this thesis, readers are encouraged to refer to Appendix B. In Figure B.1, a flowchart offers a concise overview of the various stages of the workflow and the interactions between its components. This visual aid aims at enhancing the understanding of the implemented methodologies and their integration within the dissertation framework.

6.2.2 Results

Evaluation metrics

In this section, the metrics used for assessing performance and fairness follow the approach previously outlined in Sections 4.3.1 and 6.1.1. Specifically, our evaluation methodology entails a global assessment, which involves aggregating results from all participating clients, and focuses on performance metrics such as AUC and recall at 5% FPR. For fairness evaluation, we prioritise examining the FPR parity as a critical indicator to access predictive equality.

To measure the privacy level of the models, we leverage the Opacus accountant method, which provides the associated ϵ value for a specified target δ . However, in the context of this study, where the Opacus library is used within an FL framework implemented in a simulated environment, it becomes impractical to maintain a running computation of the cumulative privacy budget. This limitation arises from the fact that the simulation generates new clients in each round of communication. Consequently, our approach involves reporting the ϵ value on a per-round basis.

Differentially Private FedProx

This section presents the results of the performance, fairness and privacy level of the FedProx algorithm reinforced with DP, which we will henceforth refer to as differentially private FedProx (DP-FedProx).

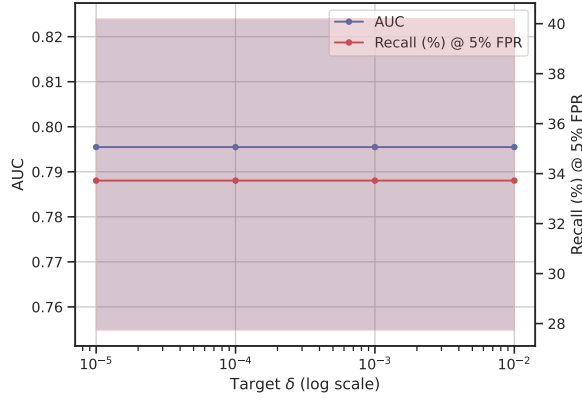
To evaluate the influence of each privacy parameter, σ , Γ , and δ on the performance, fairness, and privacy level, ϵ , of the DP-FedProx method, we follow an approach in which one of these hyperparameters is fixed at a specific value while the other two systematically vary within the corresponding search space. This results in 16 distinct models for each combination of privacy parameters, as there were four possible values for each of them. For each of these models, from the 100 rounds of training and evaluation, we select the round that yielded the highest AUC score as the representative round for that model. Subsequently, we assess the AUC score, recall at 5% FPR, FPR parity, and ϵ metrics of these rounds. For each metric, we compute the average, maximum, and minimum values across the 16 models. By repeating this process for each of the four values of the fixed privacy parameter, we comprehensively evaluate its impact on the metric of interest.

Figures 6.2, 6.3, and 6.4 depict the influence of varying the privacy parameters δ , Γ , and σ , respectively, on the performance, fairness and privacy level of the DP-FedProx method. Within these figures, the lines represent the metrics' average values, while the shaded regions denote the range between the maximum and minimum values observed.

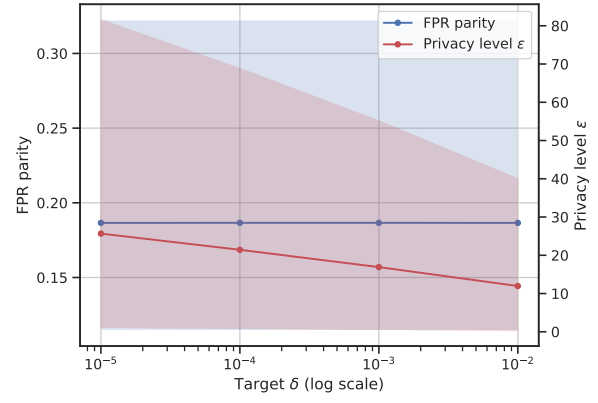
Analysing Figure 6.2, we conclude that the target δ does not affect the performance (measured by AUC score and recall at 5% FPR) or fairness of the DP-FedProx method. Moreover, δ and ϵ have an inverse relationship, which is expected from Definition 3.5. We reiterate that the (ϵ, δ) pair corresponds to the privacy budget, with lower values of both parameters corresponding to enhanced levels of privacy. Nonetheless, frameworks such as Opacus provide users with the capability to specify a desired privacy budget in terms of (ϵ, δ) , and internally compute the corresponding value of σ to attain the specified privacy goal.

From Figure 6.3, we observe that, in contrast to parameter δ , Γ solely affects the performance and fairness of the DP-FedProx method, while exhibiting no impact on ϵ . It should be noted, however, that this effect of Γ on AUC, recall, and FPR parity is minimal, given the scale of the graphics, and becomes negligible for $\Gamma > 2.0$. In this regard, Abadi et al. [96] explained that the gradient clipping threshold Γ has two opposite effects: on the one hand, if Γ is very small, it may cause the average clipped gradient to deviate significantly from the true gradient's direction; on the other hand, increasing Γ forces the addition of more noise to the gradients, and thus the model parameters, since the noise is added based on $\sigma\Gamma$. Therefore, a good approach for determining an appropriate value for Γ is by taking the median of the norms of the unclipped gradients throughout the training process.

As anticipated, Figure 6.4 shows that σ is the only privacy parameter of the three that affects all metrics. Our analysis leads to the conclusion that increasing the noise level results in lower predictive performance, affecting both the AUC score and recall. Simultaneously, this increase in noise enhances the level of privacy, achieved by reducing ϵ , thus highlighting the inherent trade-off between privacy and performance in the realm of DP. Moreover, the addition of noise introduces an element of randomness into the models, which could lead to an expected increase in the FPR parity for higher values of σ .

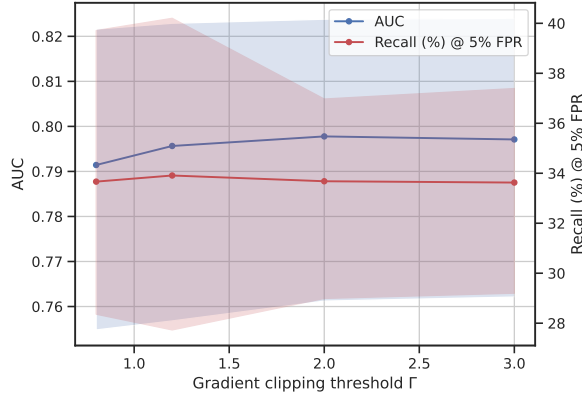


(a) AUC and recall (%) @ 5% FPR

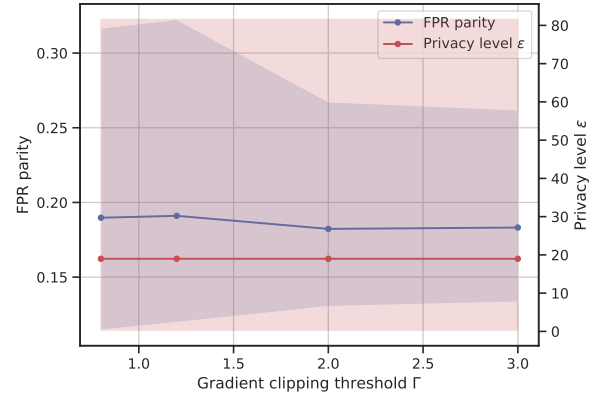


(b) FPR parity and privacy level ϵ

Figure 6.2: Influence of the target δ on the performance, fairness and privacy level of the DP-FedProx algorithm.

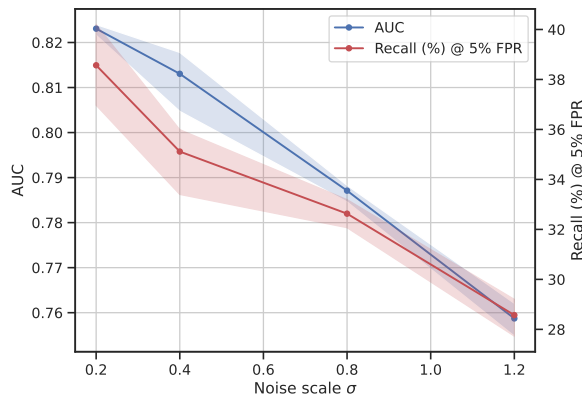


(a) AUC and recall (%) @ 5% FPR

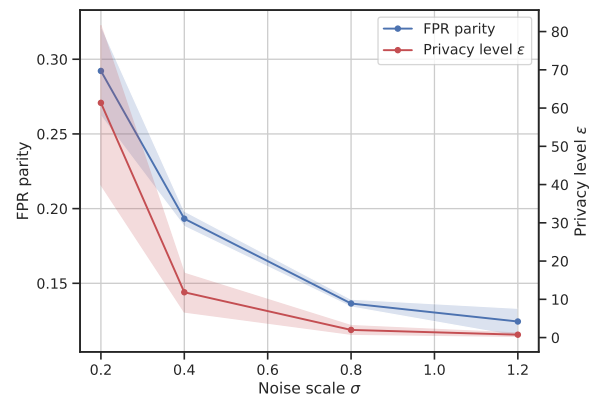


(b) FPR parity and privacy level ϵ

Figure 6.3: Influence of the gradient clipping threshold, Γ , on the performance, fairness and privacy level of the DP-FedProx algorithm.



(a) AUC and recall (%) @ 5% FPR



(b) FPR parity and privacy level ϵ

Figure 6.4: Influence of the noise scale, σ , on the performance, fairness and privacy level of the DP-FedProx algorithm.

However, Figure 6.4(b) does not support this anticipation. Specifically, raising the noise scale from $\sigma = 0.2$ to $\sigma = 1.2$, we observe a reduction of 1.17 percentage points in the FPR for the first group, while

the FPR for the second sensitive group increases 7.98 percentage points. Furthermore, our findings are corroborated by the work of Farrand et al. [103], who conclude that in highly imbalanced settings, the introduction of DP is likely to have an adverse effect on fairness, to the extent that it may lead to a significant degradation in the model's performance, resulting in increased randomness and, consequently, enhanced fairness. To further investigate and visualise this phenomenon, we plan to conduct future research employing a broader spectrum of higher σ values and examining their influence on the FPR parity.

Chapter 7

Conclusions

The rise of machine learning (ML) has resulted in the extensive incorporation of artificial intelligence (AI) into high-stakes decision-making settings, due to its scalability, generalisation, and consistency. Financial fraud detection is a predictive task that involves the analysis of tabular data for binary classification. Feedzai leverages ML and AI to develop effective fraud detection systems (FDSs) that predict this type of crime, mitigating its negative consequences. Given the similarities in fraud patterns and data schemas among different financial institutions, one strategy to enhance FDSs could involve data sharing between these institutions, enabling the development of centralised joint learning models that can effectively detect a broader range of fraudulent activities. However, regulatory guidelines on data privacy make such an exchange unfeasible. Federated learning (FL), a subfield of ML, emerges as a viable alternative to centralised learning, using multiple data sources to train a consortium model, often under the orchestration of a central server.

Nevertheless, the application of FL in financial fraud detection remains under-researched, especially when it comes to understanding the intricate relationship between predictive performance, fairness and privacy in dynamic operational settings. In this thesis, we conduct an empirical analysis of different FL techniques in realistic scenarios, focusing on the development of an FL approach for bank account opening fraud detection. Our strategy considers both performance and fairness as crucial factors. Furthermore, we introduce differential privacy (DP) into the FL framework to explore the impact of this privacy-preserving technique on the resulting models.

In Chapter 4, a comparison between the performance of the FedAvg algorithm and a centralised multi-layer perceptron (MLP) within independent and identically distributed (IID) data settings is conducted. Two techniques for data partition among clients are employed: uniform and stratified data distributions. In both scenarios, the FedAvg method proved to be able to compete with the centralised approach, achieving similar performance.

Nevertheless, when moving towards a more realistic scenario involving non-IID data settings, as detailed in Chapter 5, the FedAvg method is shown to diverge. In these situations, algorithms such as FedProx and FedOpt (including FedAdam, FedYogi, and FedAdagrad) emerge as promising alternatives, offering convergence while maintaining competitiveness with the centralised model.

Focusing on ethical and legal concerns such as fairness and privacy, in Chapter 6, we firstly conclude that, within an unbiased environment, the FedAvg algorithm maintains, and in some cases even improves, the false positive rate (FPR) parity of the resulting ML models when compared to the centralised MLP. On the other hand, biased data negatively affects the predictive equality of both the centralised and FL approaches. However, despite divergence in performance, the FPR parity of the FedAvg algorithm converges to a value that is marginally superior to those of the centralised model and other FL techniques. Moreover, it is essential to highlight the inherent fairness-performance trade-off within the context of FL.

Regarding DP, our work explores the impact of three distinct privacy parameters — namely, the target δ , gradient clipping threshold Γ , and noise scale σ — on the performance, fairness, and privacy level of the differentially private FedProx (DP-FedProx) approach. Among these parameters, Γ solely affects model performance and fairness, whereas δ specifically influences ε . The parameter σ , on the other hand, affects all metrics. Our results reveal that an increase in noise has a detrimental effect on performance, while simultaneously enhancing privacy. Unexpectedly, this noise augmentation also leads to a decrease in the FPR parity. Nevertheless, this finding is corroborated by existing literature that suggests that DP may negatively impact fairness to the extent that it significantly degrades the model's performance, resulting in increased randomness and, consequently, greater fairness.

To conclude, with this dissertation, we can claim that FL holds promise as a viable approach when confronted with legal or regulatory constraints that impede the centralisation of all data.

7.1 Future Work

For future work, we intend to incorporate a fairness-enhancing mechanism into our FL approach, aiming to improve the fairness of our models in scenarios characterised by non-IID and biased data distributions, without compromising performance.

Moreover, within the realm of differentially private FL, we plan to investigate the impact of increased levels of noise on the FPR parity of the DP-FedProx, expecting to observe an increase in fairness. Furthermore, we intend to transition from the use of Flower's Virtual Client Engine to a more realistic environment, wherein each client has a distinct IP address. This shift will facilitate the meticulous tracking of the cumulative privacy budget, offering a more comprehensive understanding of the privacy level of our models. We also recommend the integration of additional privacy-preserving techniques, such as secure multiparty computation, simultaneously with DP to ensure robust privacy protection without compromising the performance of our models.

Finally, it would be of the utmost importance to rigorously evaluate the effectiveness of the implemented approaches using authentic, real-world data sourced from some of Feedzai's clients.

Bibliography

- [1] E. W. Ngai, Y. Hu, Y. H. Wong, Y. Chen, and X. Sun. The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature. *Decision support systems*, 50(3):559–569, 2011. doi: 10.1016/j.dss.2010.08.006.
- [2] S. M. D. A. C. Jayatilake, G. U. Ganegoda, et al. Involvement of machine learning tools in healthcare decision making. *Journal of healthcare engineering*, 2021, 2021. doi: 10.1155/2021/6679512.
- [3] Z. Ni. Reframe the field of aerospace engineering via machine learning: Application and comparison. In *Journal of Physics: Conference Series*, volume 2386, page 012031. IOP Publishing, 2022. doi: 10.1088/1742-6596/2386/1/012031.
- [4] R. W. Campbell. Artificial intelligence in the courtroom: The delivery of justice in the age of machine learning. *Colo. Tech. LJ*, 18:323, 2020. doi: 10.2139/ssrn.4425791.
- [5] A. Stevens, P. Deruyck, Z. Van Veldhoven, and J. Vanthienen. Explainability and fairness in machine learning: Improve fair end-to-end lending for kiva. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 1241–1248. IEEE, 2020. doi: 10.1109/SSCI47803.2020.9308371.
- [6] J. Angwin, J. Larson, S. Mattu, and L. Kirchner. Machine bias. In *Ethics of data and analytics*, pages 254–264. Auerbach Publications, 2022. doi: 10.1201/9781003278290-37.
- [7] J. Pombal, A. F. Cruz, J. Bravo, P. Saleiro, M. A. Figueiredo, and P. Bizarro. Understanding unfairness in fraud detection through model and data bias interactions. *arXiv preprint arXiv:2207.06273*, 2022.
- [8] P. A. Grabowicz, N. Perello, and A. Mishra. Marrying fairness and explainability in supervised learning. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, pages 1905–1916, 2022. doi: 10.1145/3531146.3533236.
- [9] W. Yang, Y. Zhang, K. Ye, L. Li, and C.-Z. Xu. Ffd: A federated learning based method for credit card fraud detection. In *Big Data–BigData 2019: 8th International Congress, Held as Part of the Services Conference Federation, SCF 2019, San Diego, CA, USA, June 25–30, 2019, Proceedings 8*, pages 18–32. Springer, 2019. doi: 10.1007/978-3-030-23551-2_2.

- [10] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017 (original version on arxiv Feb. 2016).
- [11] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, et al. Advances and open problems in federated learning. *Foundations and Trends® in Machine Learning*, 14(1–2):1–210, 2021. doi: 10.1561/22000000083.
- [12] A. Anandakrishnan, S. Kumar, A. Statnikov, T. Faruquie, and D. Xu. Anomaly detection in finance: editors’ introduction. In *KDD 2017 Workshop on Anomaly Detection in Finance*, pages 1–7. PMLR, 2018.
- [13] K. Fu, D. Cheng, Y. Tu, and L. Zhang. Credit card fraud detection using convolutional neural networks. In *Neural Information Processing: 23rd International Conference, ICONIP 2016, Kyoto, Japan, October 16–21, 2016, Proceedings, Part III 23*, pages 483–490. Springer, 2016. doi: 10.1007/978-3-319-46675-0_53.
- [14] R. Desrousseaux, G. Bernard, and J.-J. Mariage. Identify theft detection on e-banking account opening. In *IJCCI*, pages 556–563, 2019. doi: 10.5220/0008648605560563.
- [15] P. Rajpurkar, J. Irvin, K. Zhu, B. Yang, H. Mehta, T. Duan, D. Ding, A. Bagul, C. Langlotz, K. Shpanskaya, M. P. Lungren, and A. Y. Ng. Chexnet: Radiologist-level pneumonia detection on chest x-rays with deep learning, 2017.
- [16] A. Bharat, N. Pooja, and R. A. Reddy. Using machine learning algorithms for breast cancer risk prediction and diagnosis. In *2018 3rd International conference on circuits, control, communication and computing (I4C)*, pages 1–4. IEEE, 2018. doi: 10.1109/CIMCA.2018.8739696.
- [17] S. Kushwaha, S. Bahl, A. K. Bagha, K. S. Parmar, M. Javaid, A. Haleem, and R. P. Singh. Significant applications of machine learning for covid-19 pandemic. *Journal of Industrial Integration and Management*, 5(04):453–479, 2020. doi: 10.1142/S2424862220500268.
- [18] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2828–2837, 2019. doi: 10.1145/3292500.3330672.
- [19] D. Azevedo, B. Ribeiro, and A. Cardoso. Web-based tool for predicting the remaining useful lifetime of aircraft components. In *2019 5th Experiment International Conference (exp. at’19)*, pages 231–232. IEEE, 2019. doi: 10.1109/EXPAT.2019.8876503.
- [20] G. Acciarini, F. Pinto, F. Letizia, J. A. Martinez-Heras, K. Merz, C. Bridges, and A. G. Baydin. Kessler: A machine learning library for spacecraft collision avoidance. In *8th European Conference on Space Debris*, pages 1–9, 2021.

- [21] B. Cankaya, K. Topuz, D. Delen, and A. Glassman. Evidence-based managerial decision-making with machine learning: The case of bayesian inference in aviation incidents. *Omega*, page 102906, 2023. doi: 10.1016/j.omega.2023.102906.
- [22] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30, 2017.
- [23] Z. C. Lipton. The mythos of model interpretability: In machine learning, the concept of interpretability is both important and slippery. *Queue*, 16(3):31–57, 2018.
- [24] S. Barocas and A. D. Selbst. Big data’s disparate impact. *California law review*, pages 671–732, 2016. doi: 10.2139/ssrn.2477899.
- [25] R. Bartlett, A. Morse, R. Stanton, and N. Wallace. Consumer-lending discrimination in the fintech era. *Journal of Financial Economics*, 143(1):30–56, 2022. doi: 10.1016/j.jfineco.2021.05.047.
- [26] S. Barocas, M. Hardt, and A. Narayanan. Fairness and machine learning. 2019, 2020.
- [27] C. Dwork, M. Hardt, T. Pitassi, O. Reingold, and R. Zemel. Fairness through awareness. In *Proceedings of the 3rd innovations in theoretical computer science conference*, pages 214–226, 2012. doi: 10.1145/2090236.2090255.
- [28] M. Hardt, E. Price, and N. Srebro. Equality of opportunity in supervised learning. *Advances in neural information processing systems*, 29, 2016.
- [29] S. Corbett-Davies, E. Pierson, A. Feller, S. Goel, and A. Huq. Algorithmic decision making and the cost of fairness. In *Proceedings of the 23rd acm sigkdd international conference on knowledge discovery and data mining*, pages 797–806, 2017. doi: 10.1145/3097983.3098095.
- [30] N. Martinez, M. Bertran, and G. Sapiro. Minimax pareto fairness: A multi objective perspective. In *International Conference on Machine Learning*, pages 6755–6764. PMLR, 2020.
- [31] A. Papadaki, N. Martinez, M. Bertran, G. Sapiro, and M. Rodrigues. Minimax demographic group fairness in federated learning. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, pages 142–159, 2022. doi: 10.1145/3531146.3533081.
- [32] P. Saleiro, B. Kuester, L. Hinkson, J. London, A. Stevens, A. Anisfeld, K. T. Rodolfa, and R. Ghani. Aequitas: A bias and fairness audit toolkit. *arXiv preprint arXiv:1811.05577*, 2018.
- [33] R. Ghani, K. T. Rodolfa, P. Saleiro, and S. Jesus. Addressing bias and fairness in machine learning: A practical guide and hands-on tutorial. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5779–5780, 2023. doi: 10.1145/3580305.3599180.
- [34] H. Zhao and G. J. Gordon. Inherent tradeoffs in learning fair representations. *The Journal of Machine Learning Research*, 23(1):2527–2552, 2022.

- [35] H. Lamba, K. T. Rodolfa, and R. Ghani. An empirical comparison of bias reduction methods on real-world problems in high-stakes policy settings. *ACM SIGKDD Explorations Newsletter*, 23(1): 69–85, 2021. doi: 10.1145/3468507.3468518.
- [36] S. Hajian and J. Domingo-Ferrer. A methodology for direct and indirect discrimination prevention in data mining. *IEEE transactions on knowledge and data engineering*, 25(7):1445–1459, 2012. doi: 10.1109/TKDE.2012.72.
- [37] B. Salimi, L. Rodriguez, B. Howe, and D. Suciu. Interventional fairness: Causal database repair for algorithmic fairness. In *Proceedings of the 2019 International Conference on Management of Data*, pages 793–810, 2019. doi: 10.1145/3299869.3319901.
- [38] F. Kamiran and T. Calders. Data preprocessing techniques for classification without discrimination. *Knowledge and information systems*, 33(1):1–33, 2012. doi: 10.1007/s10115-011-0463-8.
- [39] N. Kilbertus, M. Rojas Carulla, G. Parascandolo, M. Hardt, D. Janzing, and B. Schölkopf. Avoiding discrimination through causal reasoning. *Advances in neural information processing systems*, 30, 2017.
- [40] T. Salazar, M. S. Santos, H. Araújo, and P. H. Abreu. Fawos: Fairness-aware oversampling algorithm based on distributions of sensitive attributes. *IEEE Access*, 9:81370–81379, 2021. doi: 10.1109/ACCESS.2021.3084121.
- [41] J. Pombal, P. Saleiro, M. A. Figueiredo, and P. Bizarro. Fairness-aware data valuation for supervised learning. *arXiv preprint arXiv:2303.16963*, 2023.
- [42] T. Kamishima, S. Akaho, and J. Sakuma. Fairness-aware learning through regularization approach. In *2011 IEEE 11th International Conference on Data Mining Workshops*, pages 643–650. IEEE, 2011. doi: 10.1109/ICDMW.2011.83.
- [43] M. B. Zafar, I. Valera, M. G. Rognier, and K. P. Gummadi. Fairness constraints: Mechanisms for fair classification. In *Artificial intelligence and statistics*, pages 962–970. PMLR, 2017.
- [44] A. F. Cruz, C. Belém, S. Jesus, J. Bravo, P. Saleiro, and P. Bizarro. Fairgbm: Gradient boosting with fairness constraints. *arXiv preprint arXiv:2209.07850*, 2022.
- [45] O. D. Kose and Y. Shen. Fairgat: Fairness-aware graph attention networks. *arXiv preprint arXiv:2303.14591*, 2023.
- [46] X. Han, J. Chi, Y. Chen, Q. Wang, H. Zhao, N. Zou, and X. Hu. Ffb: A fair fairness benchmark for in-processing group fairness methods. *arXiv preprint arXiv:2306.09468*, 2023.
- [47] K. T. Rodolfa, E. Salomon, L. Haynes, I. H. Mendieta, J. Larson, and R. Ghani. Case study: predictive fairness to reduce misdemeanor recidivism through social service interventions. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, pages 142–153, 2020.

- [48] K. T. Rodolfa, H. Lamba, and R. Ghani. Empirical observation of negligible fairness–accuracy trade-offs in machine learning for public policy. *Nature Machine Intelligence*, 3(10):896–904, 2021. doi: 10.1145/3351095.3372863.
- [49] C. Dwork, N. Immorlica, A. T. Kalai, and M. Leiserson. Decoupled classifiers for group-fair and efficient machine learning. In *Conference on fairness, accountability and transparency*, pages 119–133. PMLR, 2018.
- [50] Q. Yang, Y. Liu, T. Chen, and Y. Tong. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 10(2):1–19, 2019. doi: 10.1145/3298981.
- [51] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith. Federated learning: Challenges, methods, and future directions. *IEEE signal processing magazine*, 37(3):50–60, 2020. doi: 10.1109/MSP.2020.2975749.
- [52] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith. Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems*, 2:429–450, 2020.
- [53] S. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan. Adaptive federated optimization. *arXiv preprint arXiv:2003.00295*, 2020.
- [54] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh. Scaffold: Stochastic controlled averaging for federated learning. In *International conference on machine learning*, pages 5132–5143. PMLR, 2020.
- [55] T. Li, S. Hu, A. Beirami, and V. Smith. Ditto: Fair and robust federated learning through personalization. In *International Conference on Machine Learning*, pages 6357–6368. PMLR, 2021.
- [56] L. Yang, J. Huang, W. Lin, and J. Cao. Personalized federated learning on non-iid data via group-based meta-learning. *ACM Transactions on Knowledge Discovery from Data*, 17(4):1–20, 2023. doi: 10.1145/3558005.
- [57] T. Salazar, M. Fernandes, H. Araújo, and P. H. Abreu. Fair-fate: Fair federated learning with momentum. In *International Conference on Computational Science*, pages 524–538. Springer, 2023. doi: 10.1007/978-3-031-35995-8_37.
- [58] Q. Li, Z. Wen, Z. Wu, S. Hu, N. Wang, Y. Li, X. Liu, and B. He. A survey on federated learning systems: Vision, hype and reality for data privacy and protection. *IEEE Transactions on Knowledge and Data Engineering*, 2021. doi: 10.1109/TKDE.2021.3124599.
- [59] Y. Aono, T. Hayashi, L. Wang, S. Moriai, et al. Privacy-preserving deep learning via additively homomorphic encryption. *IEEE transactions on information forensics and security*, 13(5):1333–1345, 2017. doi: 10.1109/TIFS.2017.2787987.

- [60] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017. doi: 10.1145/3133956.3133982.
- [61] C. Dwork. Differential privacy. In *International colloquium on automata, languages, and programming*, pages 1–12. Springer, 2006. doi: 10.1007/11787006_1.
- [62] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang. Learning differentially private recurrent language models. *arXiv preprint arXiv:1710.06963*, 2017.
- [63] R. C. Geyer, T. Klein, and M. Nabi. Differentially private federated learning: A client level perspective. *arXiv preprint arXiv:1712.07557*, 2017.
- [64] M. Noble, A. Bellet, and A. Dieuleveut. Differentially private federated learning on heterogeneous data. In *International Conference on Artificial Intelligence and Statistics*, pages 10110–10145. PMLR, 2022.
- [65] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. Smote: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16:321–357, 2002. doi: 10.1613/jair.953.
- [66] M. Jansson and M. Axelsson. Federated learning used to detect credit card fraud. *LU-CS-EX*, 2020.
- [67] W. Zheng, L. Yan, C. Gou, and F.-Y. Wang. Federated meta-learning for fraudulent credit card detection. In *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, pages 4654–4660, 2021. doi: 10.24963/ijcai.2020/642.
- [68] D. Byrd and A. Polychroniadou. Differentially private secure multi-party computation for federated learning in financial applications. In *Proceedings of the First ACM International Conference on AI in Finance*, pages 1–9, 2020. doi: 10.1145/3383455.3422562.
- [69] R. H. L. Rosero, C. Silva, and B. Ribeiro. Remaining useful life estimation in aircraft components with federated learning. In *PHM Society European Conference*, volume 5, pages 9–9, 2020.
- [70] W. Li, W. Yang, G. Jin, J. Chen, J. Li, R. Huang, and Z. Chen. Clustering federated learning for bearing fault diagnosis in aerospace applications with a self-attention mechanism. *Aerospace*, 9(9):516, 2022. doi: 10.3390/aerospace9090516.
- [71] B. Brik, A. Ksentini, and M. Bouaziz. Federated learning for uavs-enabled wireless networks: Use cases, challenges, and open problems. *IEEE Access*, 8:53841–53849, 2020. doi: 10.1109/ACCESS.2020.2981430.
- [72] H. Zhang and L. Hanzo. Federated learning assisted multi-uav networks. *IEEE Transactions on Vehicular Technology*, 69(11):14104–14109, 2020. doi: 10.1109/TVT.2020.3028011.

- [73] J. Yao and N. Ansari. Secure federated learning by power control for internet of drones. *IEEE Transactions on Cognitive Communications and Networking*, 7(4):1021–1031, 2021. doi: 10.1109/TCCN.2021.3076167.
- [74] Y. Wang, Z. Su, N. Zhang, and A. Benslimane. Learning in the air: Secure federated learning for uav-assisted crowdsensing. *IEEE Transactions on network science and engineering*, 8(2): 1055–1069, 2020. doi: 10.1109/TNSE.2020.3014385.
- [75] Y. Qu, H. Dai, Y. Zhuang, J. Chen, C. Dong, F. Wu, and S. Guo. Decentralized federated learning for uav networks: Architecture, challenges, and opportunities. *IEEE Network*, 35(6):156–162, 2021. doi: 10.1109/MNET.001.2100253.
- [76] N. Moustafa, I. A. Khan, M. Hassanin, D. Ormrod, D. Pi, I. Razzak, and J. Slay. Dfsat: Deep federated learning for identifying cyber threats in iot-based satellite networks. *IEEE Transactions on Industrial Informatics*, 2022. doi: 10.1109/TII.2022.3214652.
- [77] H. Chen, M. Xiao, and Z. Pang. Satellite-based computing networks with federated learning. *IEEE Wireless Communications*, 29(1):78–84, 2022. doi: 10.1109/MWC.008.00353.
- [78] J. So, K. Hsieh, B. Arzani, S. Noghabi, S. Avestimehr, and R. Chandra. Fedspace: An efficient federated learning framework at satellites and ground stations. *arXiv preprint arXiv:2202.01267*, 2022.
- [79] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi. Credit card fraud detection: a realistic modeling and a novel learning strategy. *IEEE transactions on neural networks and learning systems*, 29(8):3784–3797, 2017. doi: 10.1109/TNNLS.2017.2736643.
- [80] C. Elkan. The foundations of cost-sensitive learning. In *International joint conference on artificial intelligence*, volume 17, pages 973–978. Lawrence Erlbaum Associates Ltd, 2001.
- [81] A. Cannon, J. Howse, D. Hush, and C. Scovel. Learning with the neyman-pearson and min-max criteria. *Los Alamos National Laboratory, Tech. Rep. LA-UR*, pages 02–2951, 2002.
- [82] A. Chouldechova. Fair prediction with disparate impact: A study of bias in recidivism prediction instruments. *Big data*, 5(2):153–163, 2017. doi: 10.1089/big.2016.0047.
- [83] T. Speicher, H. Heidari, N. Grgic-Hlaca, K. P. Gummadi, A. Singla, A. Weller, and M. B. Zafar. A unified approach to quantifying algorithmic unfairness: Measuring individual & group unfairness via inequality indices. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2239–2248, 2018. doi: 10.1145/3219819.3220046.
- [84] J. Kleinberg. Inherent trade-offs in algorithmic fairness. In *Abstracts of the 2018 ACM International Conference on Measurement and Modeling of Computer Systems*, pages 40–40, 2018. doi: 10.1145/3219617.3219634.
- [85] R. Patidar, L. Sharma, et al. Credit card fraud detection using neural network. *International Journal of Soft Computing and Engineering (IJSCE)*, 1(32-38), 2011.

- [86] B. Kasasbeh, B. Aldabaybah, and H. Ahmad. Multilayer perceptron artificial neural networks-based model for credit card fraud detection. *Indonesian Journal of Electrical Engineering and Computer Science*, 26(1):362–373, 2022. doi: 10.11591/ijeecs.v26.i1.pp362-373.
- [87] A. M. Mubarek and E. Adali. Multilayer perceptron neural network technique for fraud detection. In *2017 International Conference on Computer Science and Engineering (UBMK)*, pages 383–387. IEEE, 2017. doi: 10.1109/UBMK.2017.8093417.
- [88] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [89] Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.
- [90] J. Singh and R. Banerjee. A study on single and multi-layer perceptron neural network. In *2019 3rd International Conference on Computing Methodologies and Communication (ICCMC)*, pages 35–40. IEEE, 2019. doi: 10.1109/ICCMC.2019.8819775.
- [91] I. Neutelings. Neural networks, Apr 2023. URL https://tikz.net/neural_networks/.
- [92] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [93] C. Dwork, A. Roth, et al. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science*, 9(3–4):211–407, 2014. doi: 10.1561/04000000042.
- [94] C. Dwork, F. McSherry, K. Nissim, and A. Smith. Calibrating noise to sensitivity in private data analysis. *Journal of Privacy and Confidentiality*, 7(3):17–51, 2016. doi: 10.29012/jpc.v7i3.405.
- [95] J. Lee and C. Clifton. How much is enough? choosing ϵ for differential privacy. In *Information Security: 14th International Conference, ISC 2011, Xi'an, China, October 26-29, 2011. Proceedings 14*, pages 325–340. Springer, 2011. doi: 10.1007/978-3-642-24861-0_22.
- [96] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 308–318, 2016. doi: 10.1145/2976749.2978318.
- [97] A. Yousefpour, I. Shilov, A. Sablayrolles, D. Testuggine, K. Prasad, M. Malek, J. Nguyen, S. Ghosh, A. Bharadwaj, J. Zhao, et al. Opacus: User-friendly differential privacy library in pytorch. *arXiv preprint arXiv:2109.12298*, 2021.
- [98] S. Jesus, J. Pombal, D. Alves, A. Cruz, P. Saleiro, R. Ribeiro, J. Gama, and P. Bizarro. Turning the tables: Biased, imbalanced, dynamic tabular datasets for ml evaluation. *Advances in Neural Information Processing Systems*, 35:33563–33575, 2022.
- [99] G. Van Rossum, F. L. Drake, et al. *Python reference manual*, volume 111. Centrum voor Wiskunde en Informatica Amsterdam, 1995.

- [100] Meta AI. Pytorch, 2016. URL <https://pytorch.org/>.
- [101] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. de Gusmão, et al. Flower: A friendly federated learning research framework. *arXiv preprint arXiv:2007.14390*, 2020.
- [102] I. Mironov, K. Talwar, and L. Zhang. Rényi differential privacy of the sampled gaussian mechanism. *arXiv preprint arXiv:1908.10530*, 2019.
- [103] T. Farrand, F. Mireshghallah, S. Singh, and A. Trask. Neither private nor fair: Impact of data imbalance on utility and fairness in differential privacy. In *Proceedings of the 2020 workshop on privacy-preserving machine learning in practice*, pages 15–19, 2020. doi: 10.1145/3411501.3419419.

Appendix A

Hyperparameter Search Spaces

In this appendix, an elucidation is presented regarding the optimisation procedure for the hyperparameters within the centralised MLP model. A random search method was employed to explore the expansive hyperparameter space, as outlined in Table A.1. The MLP underwent training using one hundred distinct hyperparameter combinations, and the selection of the optimal model hinged upon its performance on the validation set. The combination of hyperparameters that led to the model with the highest recall while maintaining a fixed FPR ceiling of 5% was selected. Subsequently, the model's performance was rigorously assessed on the test set, for that hyperparameter combination.

Table A.1: Centralised model (MLP) hyperparameter search space.

Hyperparameter	Values or Interval	Distribution
Number of hidden layers	{1, 2, 3, 4}	
Number of neurons of each layer	{1, 2, 3, 4}	
Hidden activation functions	{"LeakyReLU", "Tanh"}	
Learning rate	[0.00001, 0.01]	Uniform
Dropout rate	[0, 0.1]	Uniform
Batch prevalence	[0.001, 0.0999]	Uniform
Batch size	{64, 128, 256, 512}	
Optimiser	{"Adam", "SGD"}	

Furthermore, this appendix provides Table A.2, which delineates the parameter space concerning the DP hyperparameters. Employing a grid search approach to encompass the full spectrum of potential parameter combinations, it becomes feasible to thoroughly evaluate the individual impacts of each parameter on the performance, fairness, and privacy level of the models.

Table A.2: Differential Privacy hyperparameter search space.

Hyperparameter	Values
Target δ	$\{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$
Noise scale σ	$\{0.2, 0.4, 0.8, 1.2\}$
Gradient clipping threshold Γ	$\{0.8, 1.2, 2.0, 3.0\}$

Appendix B

Dissertation Workflow

Figure B.1 illustrates the workflow employed in this thesis. Two datasets from the BAF suite are selected, and three distinct experiments are conducted. In Chapter 4, a comparison is made between the performance of the FedAvg algorithm and a centralised MLP on IID settings. Additionally, two different experiments are undertaken within these settings: one that considers uniform data distribution across clients for FedAvg, and the other that examines stratified data distribution.

In Chapter 5, we explore FL on non-IID settings using the BAF Base dataset with a stratified data distribution among clients. The performance of multiple FL algorithms is compared, namely FedAvg, FedProx, and FedOpt, which encompasses FedAdam, FedAdagrad, and FedYogi. Moreover, a comparison with the corresponding centralised model is also carried out, although it is not the primary objective of this chapter.

Finally, Chapter 6 introduces observational fairness evaluation and DP to previously studied algorithms. These include the centralised model in both IID and non-IID scenarios, FedAvg on IID settings, and FedProx on the BAF Base. Furthermore, the study investigates how privacy constraints affect the performance and fairness of the corresponding algorithms.

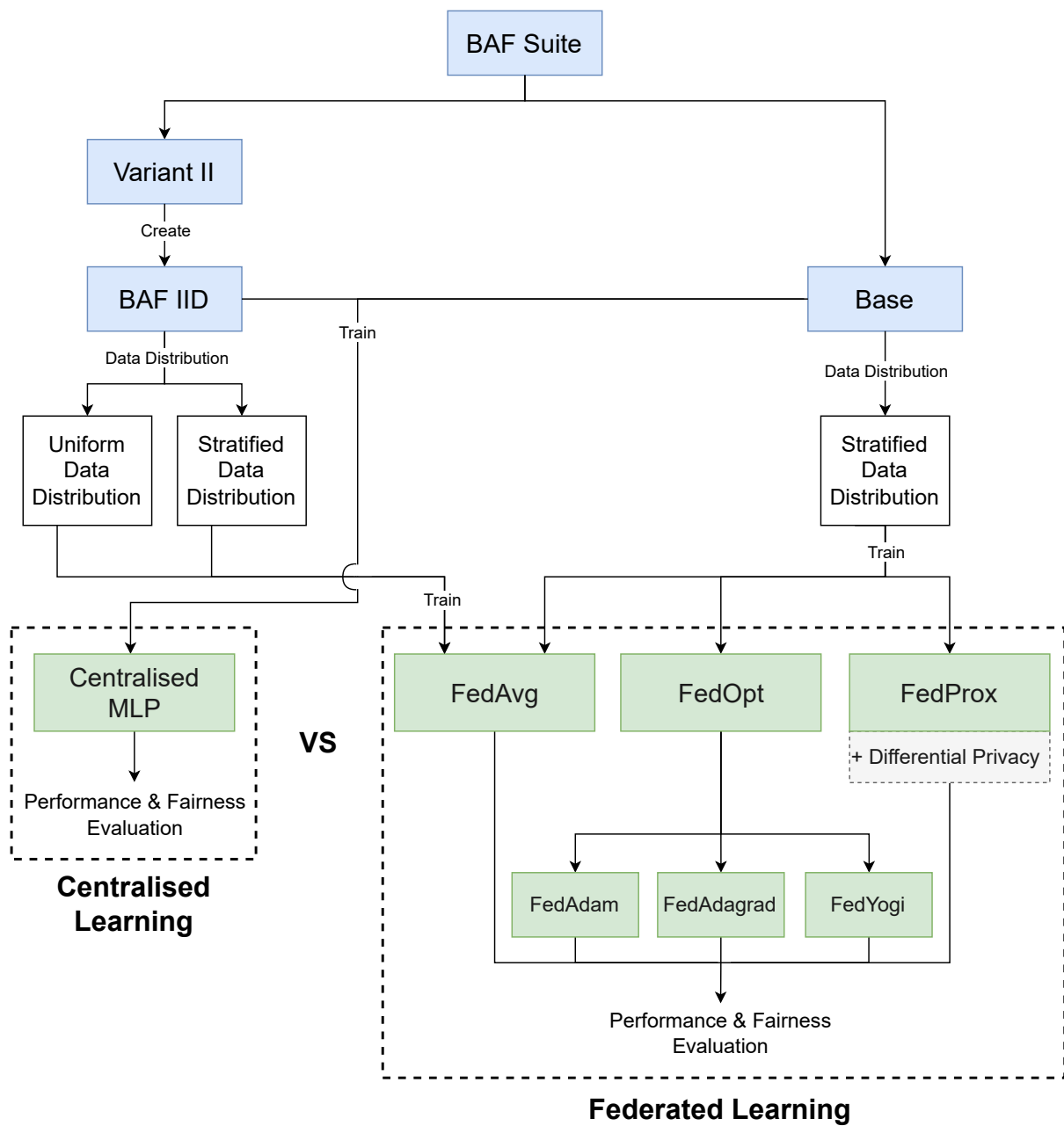


Figure B.1: Flowchart representing this dissertation workflow.